

Strukture podataka i algoritmi

Stabla

dr Danijel Čabarkapa

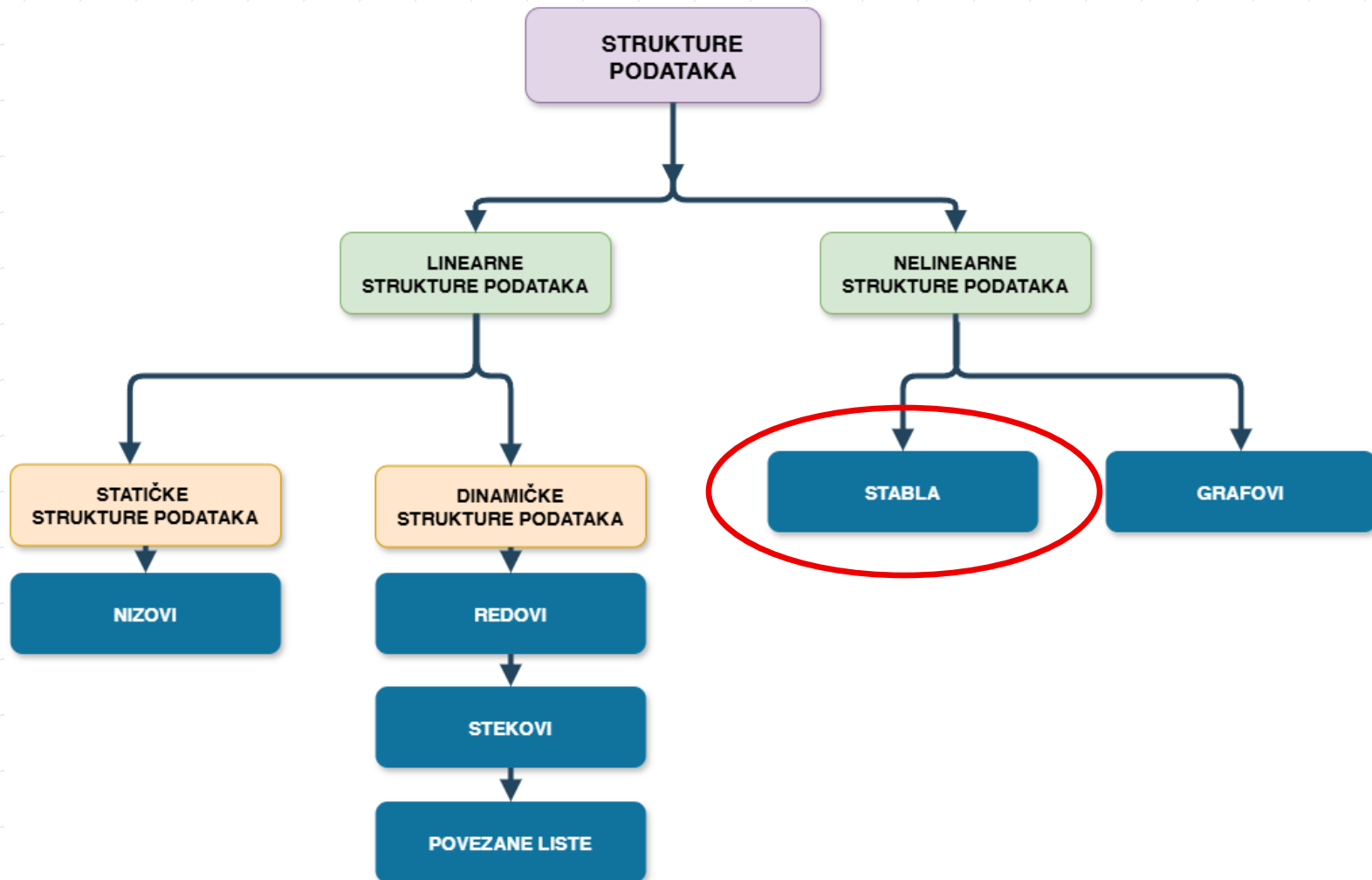
Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Strukture podataka: Stabla
2. Strukture podataka: Binarno stablo, binarni hip
3. Binarna balansirana stabla: AVL i RBT

Klasifikacija struktura podataka

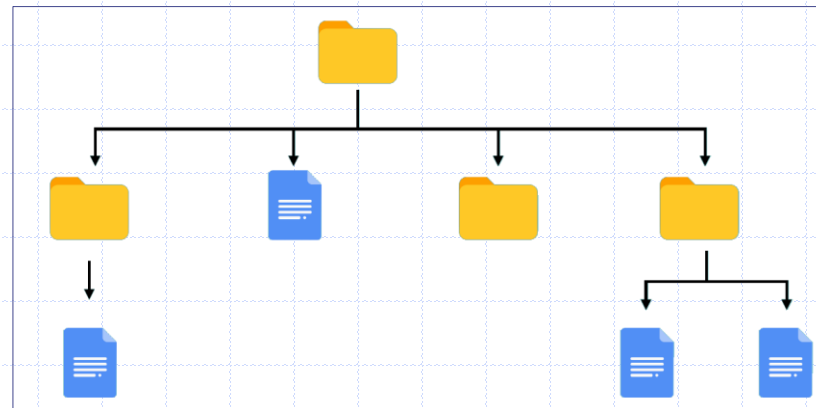
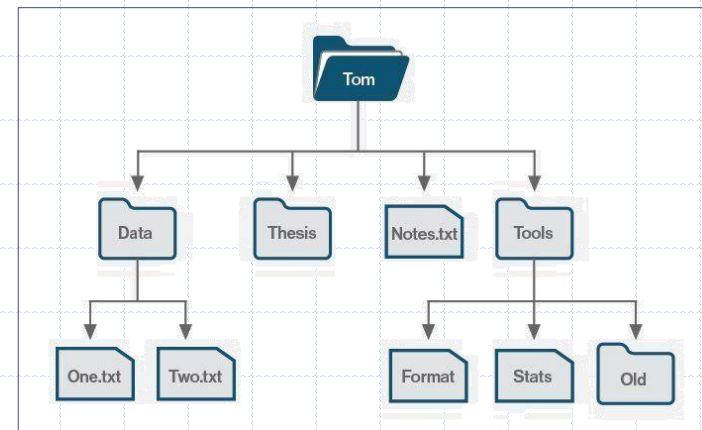
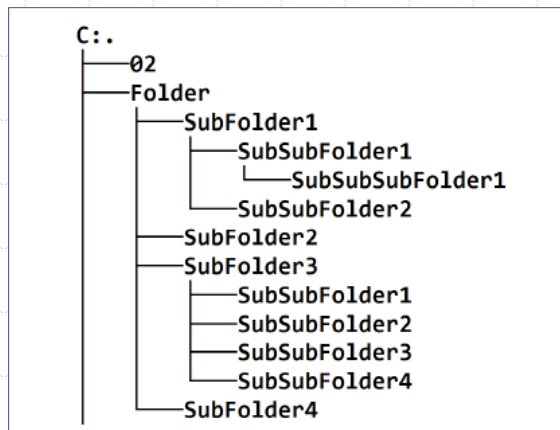


Stabla

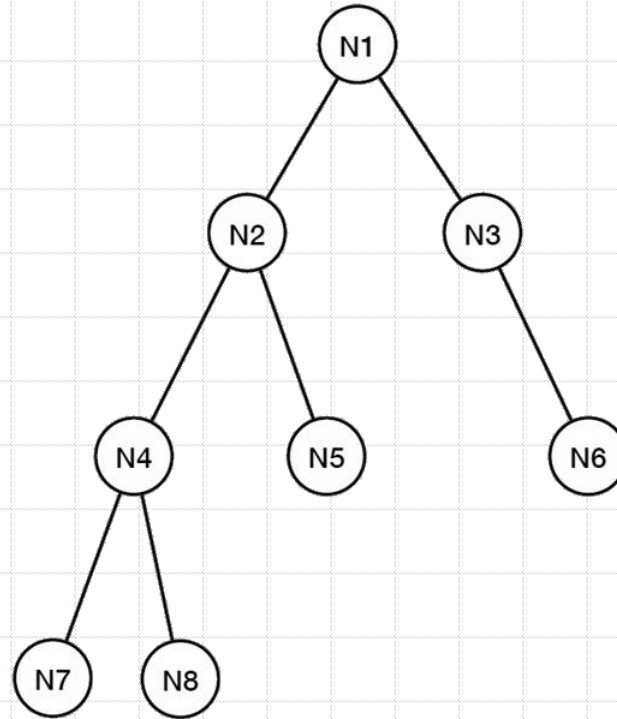
- **Stablo** (drvo, **tree**) je nelinearna struktura podataka kod koje su elementi organizovani u hijerarhijskoj formi
- Iako je po svom obliku jednostavna, ova struktura podataka ima značajnu primenu u računarstvu (svi operativni sistemi imaju organozovane datoteke u formi stabla, aritmetički izrazi se u interpreterima koriste u formi stabla izraza, koriste se za predstavljanje drugih struktura podataka – disjunktni skupovi, redovi sa prioritetima itd...)
- Stablo se može koristiti i kao model nekog procesa ili odnosa među objektima i podacima, a ne samo kao zbirka podataka
- Stabla su hijerarhijski uređeni podaci (postoje prethodnik i sledbenik, ali i više nivoa pdataka)
- Uređeno stablo je objekt koja simulira hijerarhijsku strukturu skupom povezanih čvorova

Stabla - primene

- Struktura stabla omogućava brz i jednostavan pristup podacima
- Čuvanje hijerarhijskih podataka kao što su npr. strukture foldera, organizacione strukture, sadržaji knjiga, XML/HTML podaci itd...

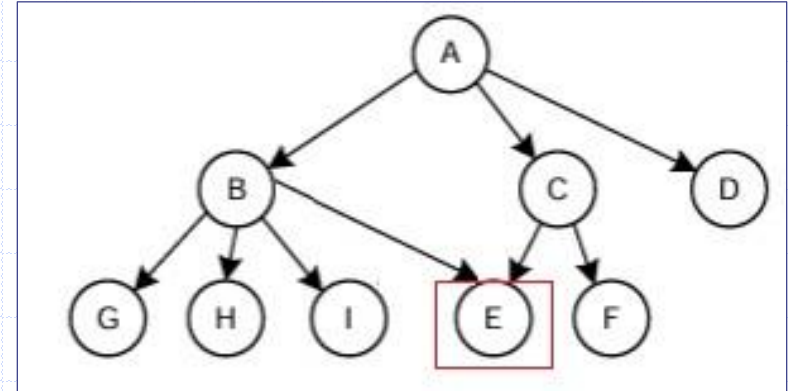
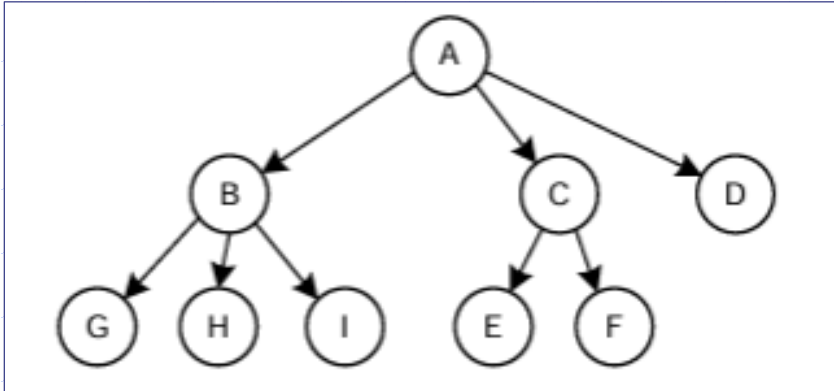


Stabla



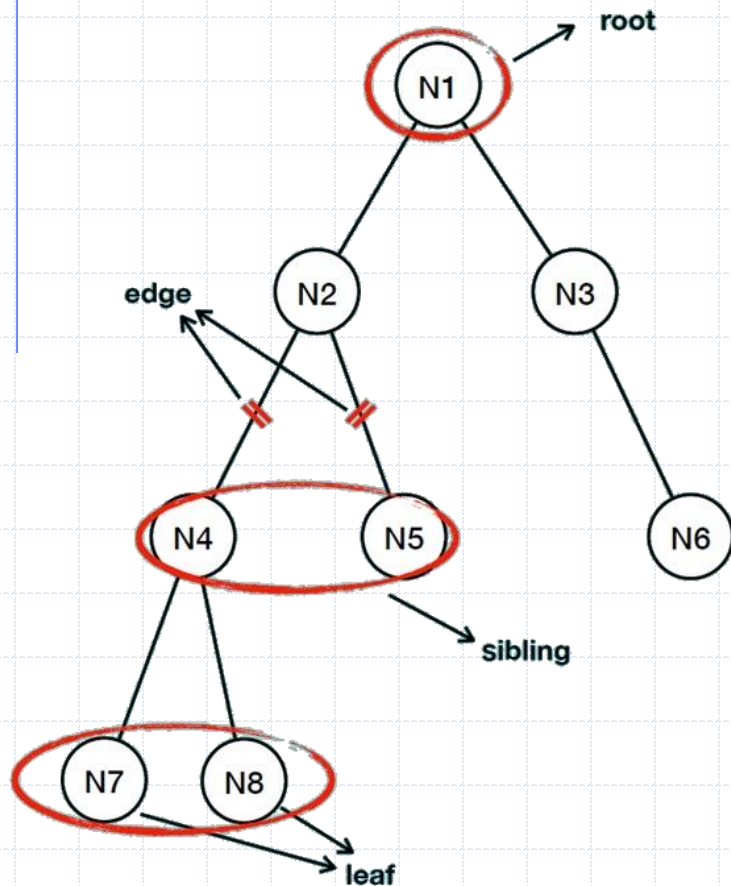
- **Korensko stablo** (*rooted tree*) se sastoji od jednog čvora (N1) koji predstavlja koren (*root*) i skupa drugih čvorova koji se iz njega direktno ili indirektno granaju (N2-N8)
- Stablo se sastoji od **čvorova** (*nodes*) (N1-N8) i grana između njih (*edges*)
- Svaka grana povezuje jedan čvor **roditelj** (*parent*) sa njegovim **potomkom** (*dete, child, successor, descendent*)

Stabla



- Stablo se se sastoji od jednog ili više čvorova koji zadovoljavaju sledeće uslove:
 - Postoji jedan čvor koji predstavlja koren stabla
 - Ostali čvorovi su podeljeni u **n** disjunktних skupova čvorova, tako da svaki skup predstavlja stablo
- (Leva slika) jeste stablo jer je čvor A koren stabla, a ostali čvorovi su podeljeni u 3 disjunktne skupa (B,G,H,I), (C,E,F) i D
- (Desna slike) nije stablo jer je čvor E podeljen

Stabla - terminologija



Koren (Root) – čvor na vrhu stabla, bez roditelja

Link (Edge) – veza, konekcija između čvorova

List (Leaf) – čvorovi bez potomaka (N5, N6, N7, N8)

Sibling – čvorovi od istog roditelja (N4 i N5)

Predak (Ancestor) – predak je roditeljski čvor kao i svi prethodni čvorovi roditelji (N1, N2 i N4 su preci čvora N7)

Sucessors – potomci, svi čvorovi koji se granaju iz jednog čvora

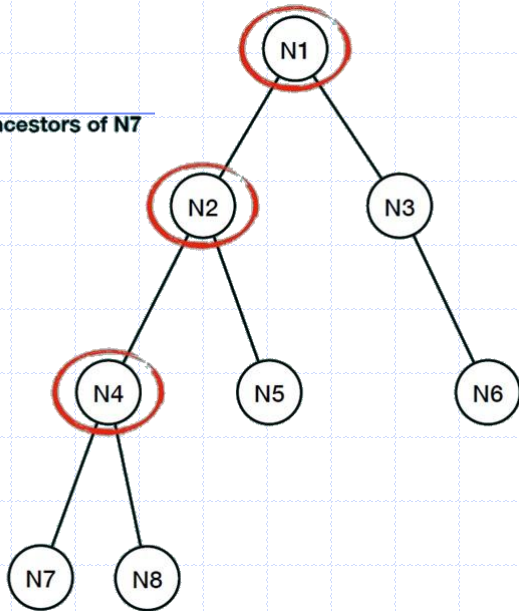
Visina stabla (Height of Tree) – broj čvorova na putu od korena do najdaljeg lista (koren se nalazi na visini 0) ili visina korena

Dubina čvora (Depth of Node) – predstavlja dužinu puta od korena do tog čvora (koren je 0)

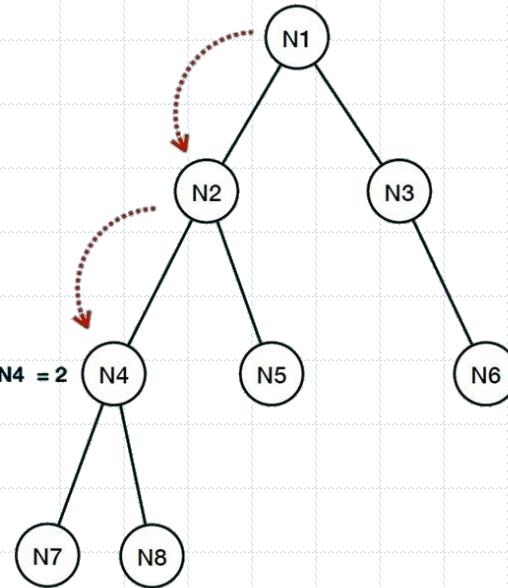
Visina čvora (Height of Node) – dužina putanje od čvora do lista (list je na visini 0)

Stabla - terminologija

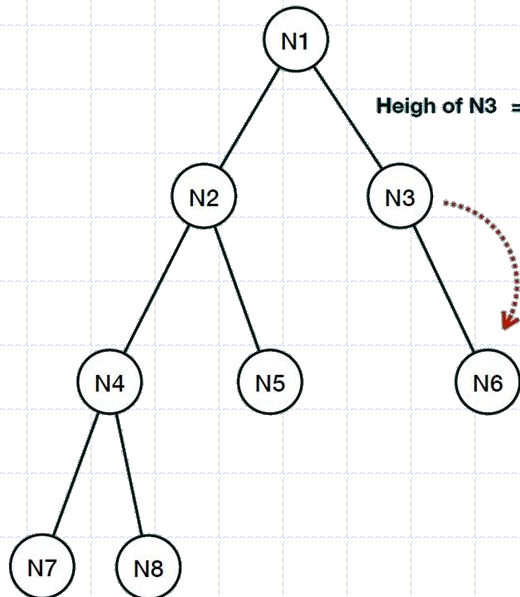
ancestors of N7



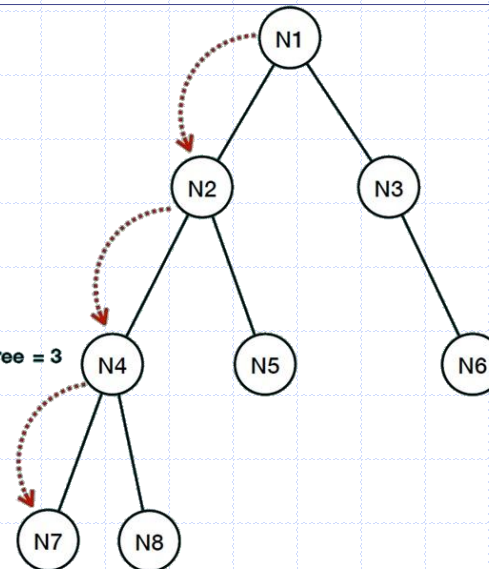
Depth of N4 = 2



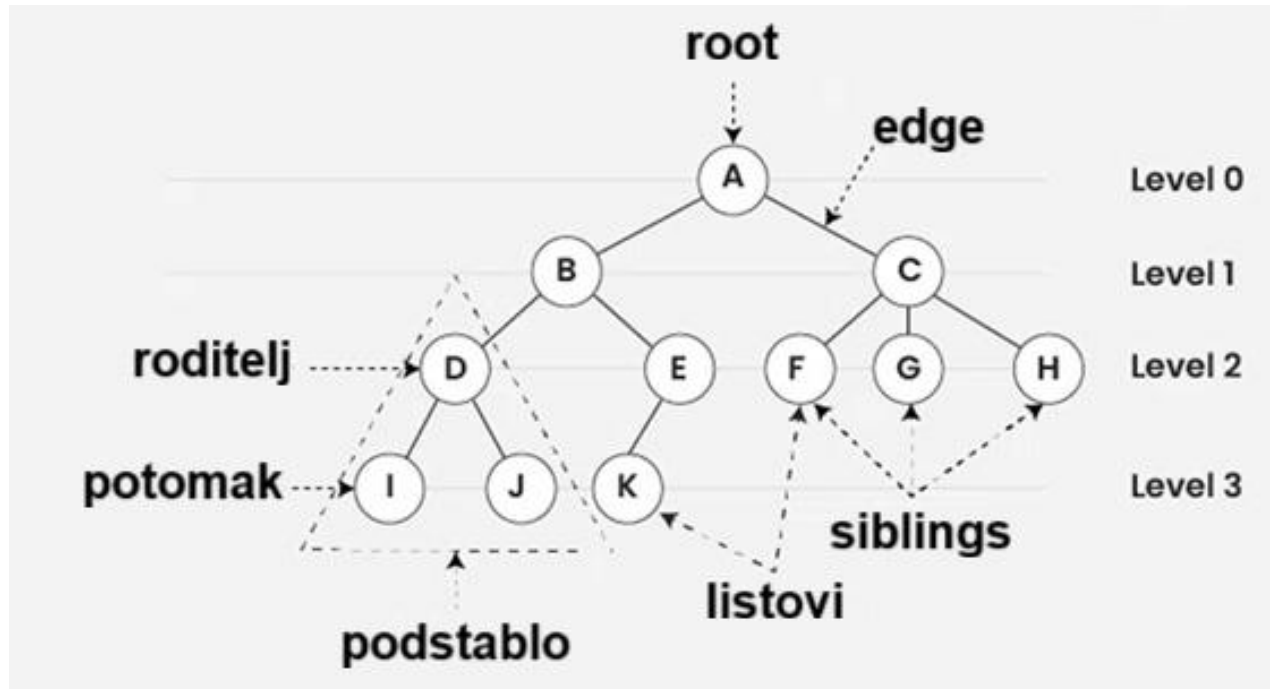
Heigh of N3 = 1



Height of tree = 3

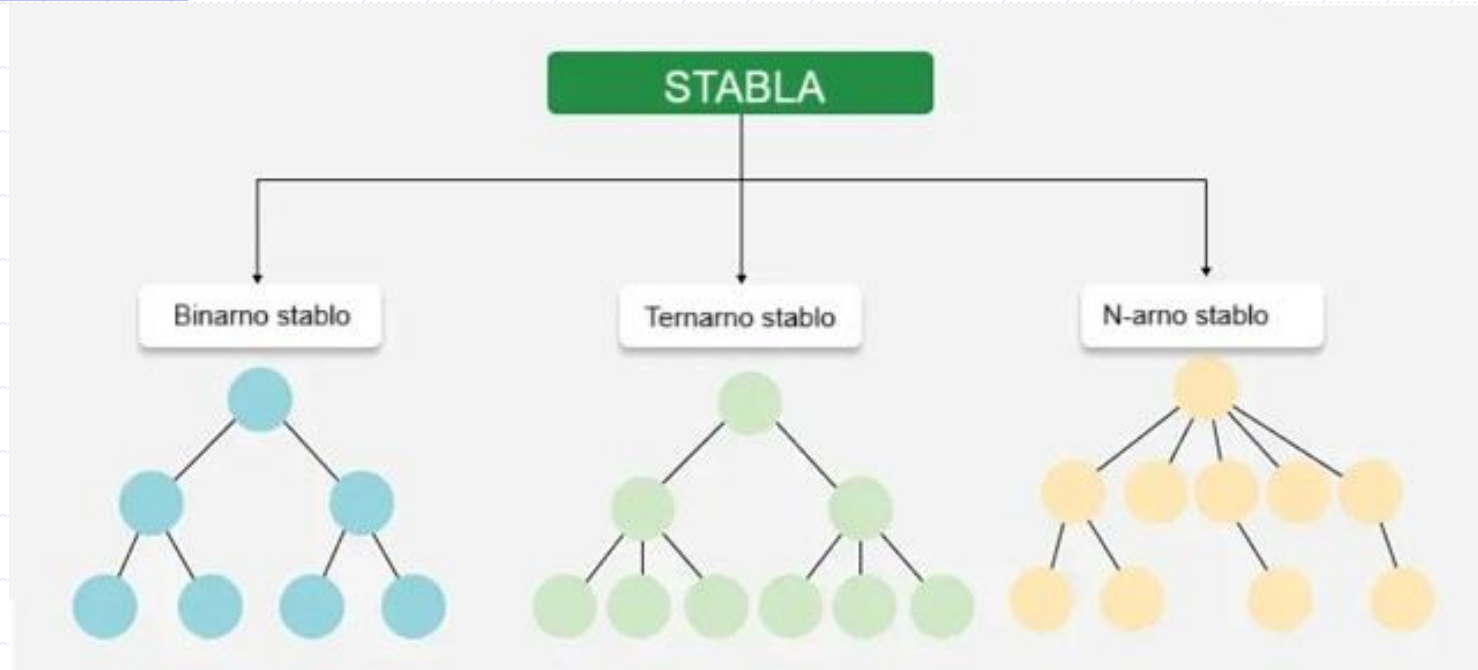


Stabla - terminologija



- Čvorovi koji imaju bar jednog potomka su **unutrašnji** (*internal nodes*) {A, B, C, D, E}, a čvorovi bez potomaka su **spoljašnji** (*external nodes*) {I, J, K, F, G, H}
- **Podstablo** (*subtree*) je stablo koje se sastoji od root čvora i svih njegovih potomaka, zajedno sa nasleđenim granama, {D, I, J}
- **Predak** (*ancestor*) čvora su bilo koji prethodni čvorovi na putanji od korena do tog čvora, {A, B} su čvorovi preci čvora {E}
- **Nivo stabla** (*level*) je skup svih čvorova sa svojstvom da je put od korena do tog čvora tačno određene dužine, koren je na nivou 0, njegovi potomci na nivou 1, njihovi potomci na nivou 2 itd...

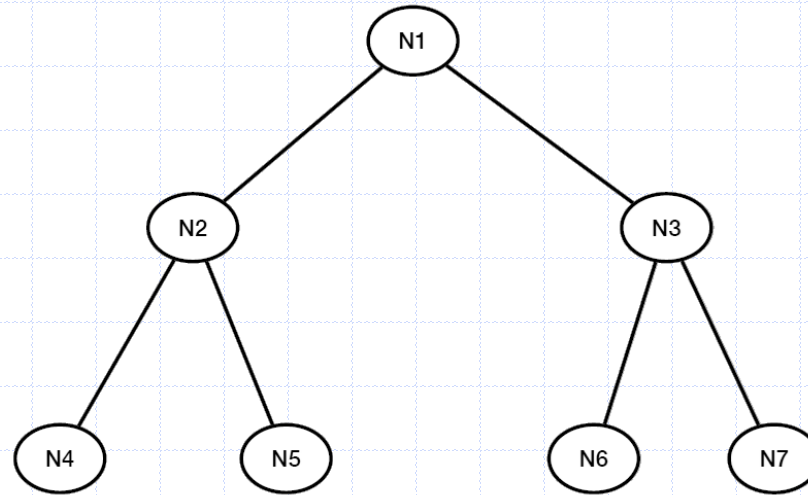
Stabla - podele



- Binarno stablo: Svaki čvor ima najviše dva čvora potomka
- Ternarno stablo: Svaki čvor ima najviše tri čvora potomka
- N-arno stablo: Svaki čvor ima najviše N čvorova potomaka

2. Strukture podataka: Binarno stablo, binarni hip

Binarna stabla

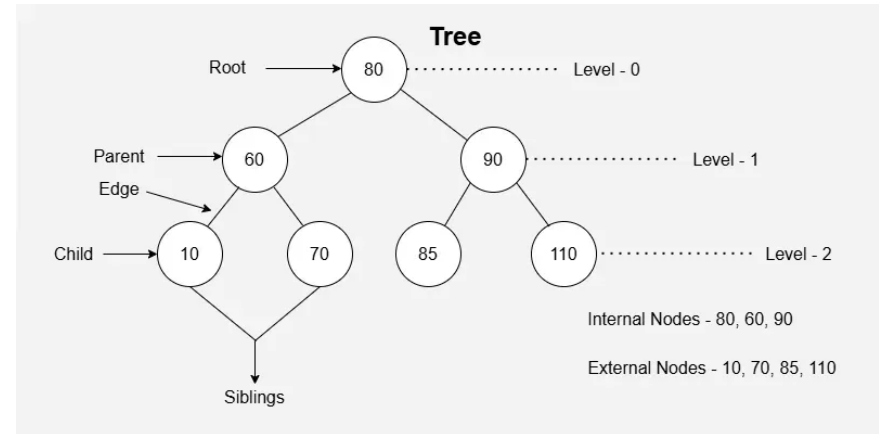
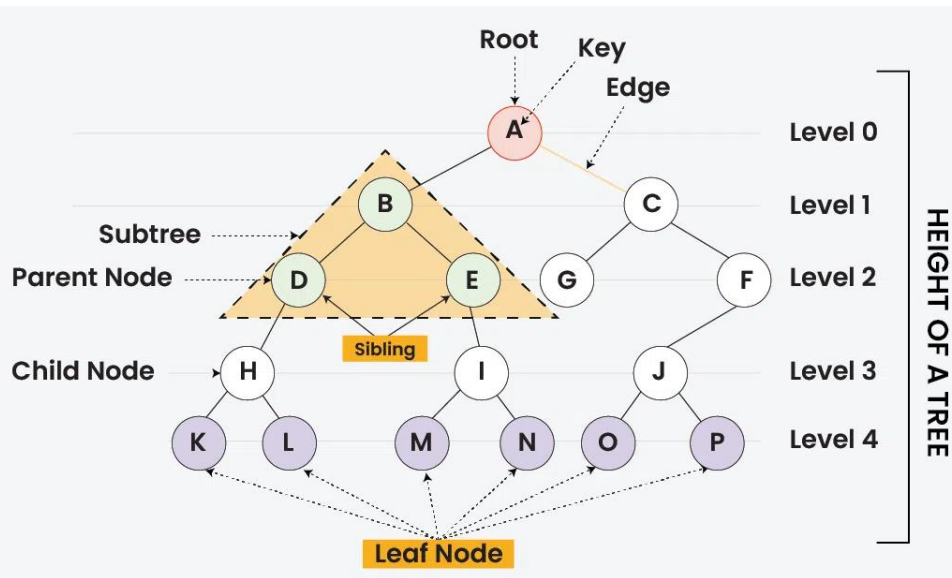


- **Binarno stablo** (Binary Tree) je specijalni slučaj stabla u kome ni jedan čvor nema stepen veći od 2, što znači da svaki čvor ima najviše dva potomka koja se nazivaju levi i desni potomak
- Binarno stablo se često koristi u praksi za efikasno skladištenje i pretraživanje podataka, sa različitim operacijama kao što su umetanje, brisanje i prolazak (traverse) kroz stablo
- Binarno stablo je skup od jednog ili više čvorova takvih da:
 - Postoji jedan čvor koji je koren stabla
 - Preostali čvorovi su podeljeni u dva disjunktne skupa, tako da svaki skup predstavlja binarno stablo. Ova dva skupa se nazivaju levo i desno podstablo

Binarna stabla - terminologija

- Za binarna stabla definišu se sledeći pojmovi:
 - **Visina stabla**: broj čvorova na putu od korena do najdaljeg lista stabla (koren je na visini 0)
 - **Dubina čvora**: predstavlja dužinu puta od korena do tog čvora
 - **Stepen čvora**: predstavlja broj potomaka čvora
 - **Potpuno binarno stablo**: binarno stablo kod kojeg svaki list ima istu dubinu, a svaki unutrašnji čvor dva potomka
 - **Skoro potpuno binarno stablo**: stablo kod kojeg poslednji nivo ne mora biti do kraja popunjen, a svi listovi na tom nivou poređani su jedan do drugog bez praznih mesta, posmatrajući ih s leva na desno
 - **Balansirano binarno stablo**: binarno stablo kod kojeg se visina levog i desnog podstabla svakog čvora ne razlikuje za više od 1
 - **Degenerisano binarno stablo**: binarno stablo u kojem svaki roditeljski čvor ima samo jednog potomka, i u tom slučaju stablo se predstavlja kao povezana lista

Binarna stabla - osobine



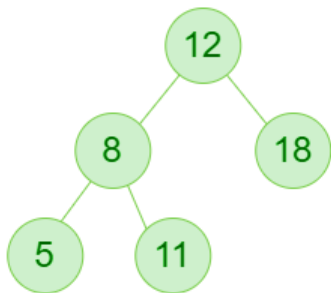
- Maksimalni broj čvorova na nivou **L** binarnog stabla je **2^L**
- Maksimalni broj čvorova binarnog stabla visine **H** je **$2^H - 1$**
- Ukupan broj listova binarnog stabla se računa kao ukupan broj čvorova sa 2 potomka + 1
- U binarnom stablu sa **N** čvorova, minimalna moguća visina ili minimalni broj nivoa je **$\log_2(N+1)$**
- Binarno stablo sa **L** listova ima najmanje **$\lceil \log_2 L \rceil + 1$** nivoa

Binarna stabla - klasifikacija

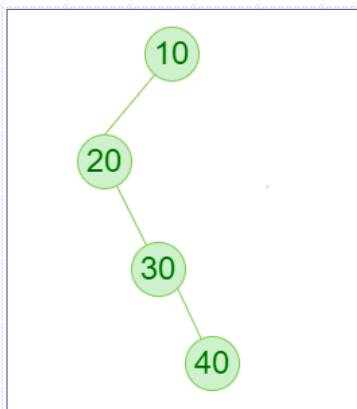
Binarna stabla se klasifikuju u zavisnosti od nekoliko faktora:

- 1) U zavisnosti od broja čvorova potomaka:
 - Potpuno binarno stablo (*Full Binary Tree*)
 - Degenerisano binarno stablo (*Degenerate Binary Tree*)
 - Iskošeno binarno stablo (*Skewed Binary Tree*)
- 2) U zavisnosti od popunjenosti nivoa stabla:
 - Kompletno binarno stablo (*Complete Binary Tree*)
 - Savršeno binarno stablo (*Perfect Binary Tree*)
 - Balansirano binarno stablo (*Balanced Binary Tree*)
- 3) U zavisnosti od vrednosti čvorova stabla:
 - Binarno stablo pretrage (*Binary Search Tree*)
 - AVL Tree
 - Red-Black Tree
 - B Tree
 - B+ Tree
 - Segment Tree

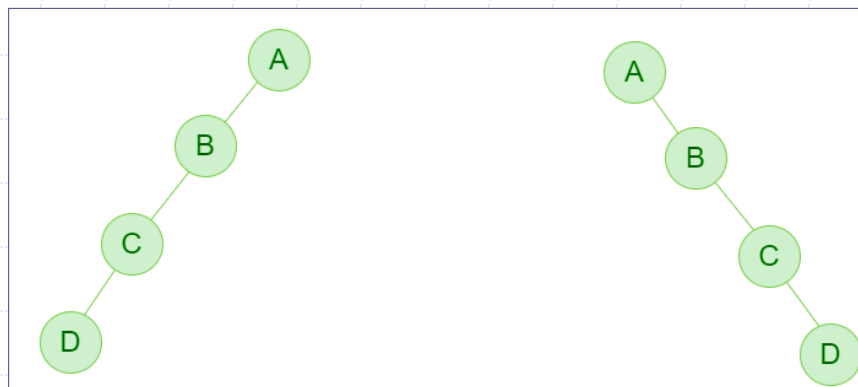
Binarna stabla - klasifikacija



Potpuno binarno stablo je poseban tip binarnog stabla u kome svaki roditeljski čvor/unutrašnji čvor ima dva ili nijednog potomka. Naziva se još i pravilno binarno stablo

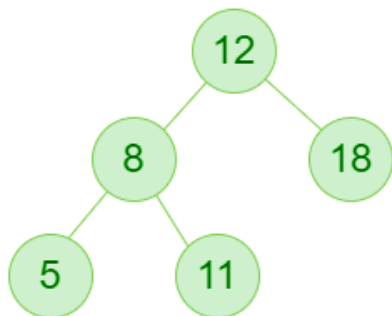


Degenerisano binarno stablo ima formu da svaki unutrašnji čvor (roditelj) ima samo jednog potomka i po pitanju performansi ova stabla su ista kao i povezane liste

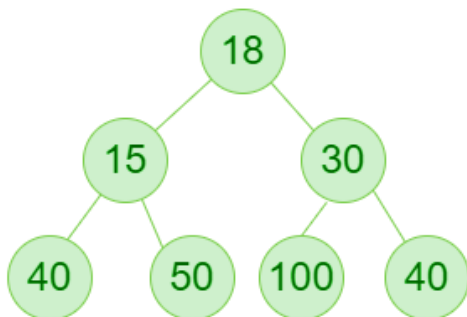


Iskošeno binarno stablo je vrsta degenerisanog stabla u kojem dominiraju ili levi ili desni čvorovi, tako da postoje dve vrste ovog binarnog stabla: levo iskošeno i desno iskošeno binarno stablo

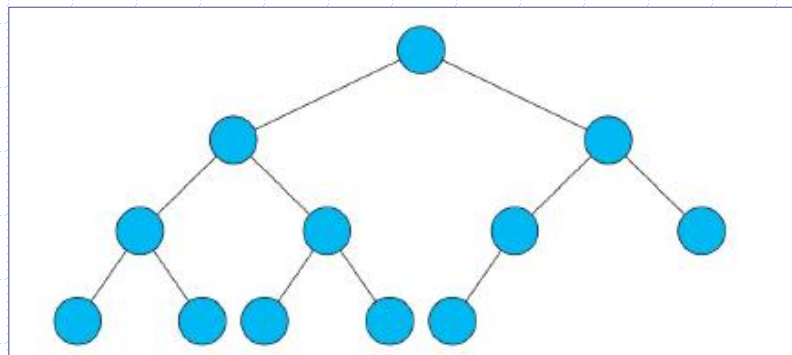
Binarna stabla - klasifikacija



Kompletno binarno stablo je stablo kada su svi nivoi potpuno popunjeni osim možda poslednjeg nivoa, koji treba da ima sve listove sa leve strane. Poslednji list ne mora imati desni sibling čvor.

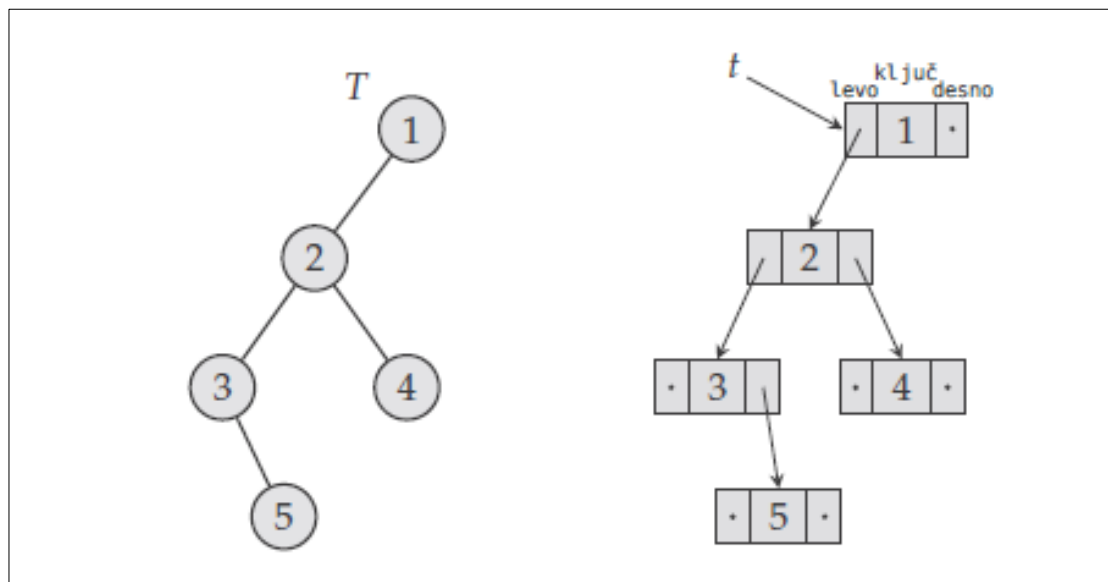


Savršeno binarno stablo je vrsta binarnog stabla u kojem svaki unutrašnji čvor ima tačno dva čvora potomka i svi listovi su na istom nivou



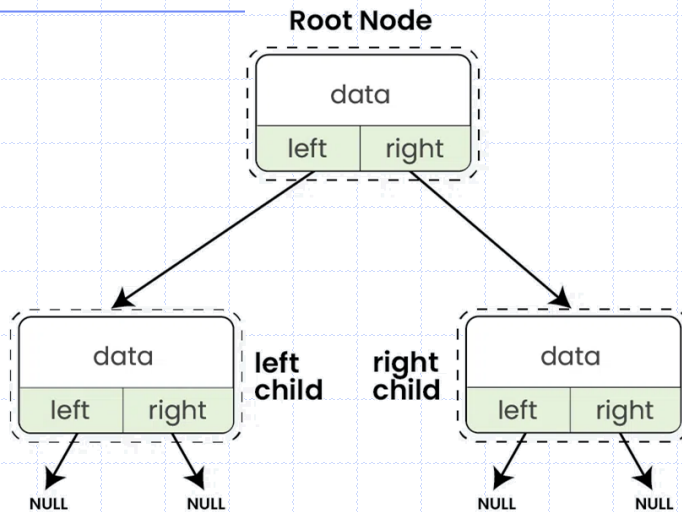
Balansirano binarno stablo je binarno stablo kod kojeg se visina levog i desnog podstabla svakog čvora ne razlikuje za više od 1

Predstavljjanje binarnog stabla



- Predstavljjanje binarnog stabla je pojednostavljeno zbog činjenice da svaki njegov čvor ima **najviše dva potomka**
- U praksi se za predstavljanje binarnih stabala uglavnom koriste pokazivači (*pointers*)
- Binarno stablo je organizovano u obliku povezane strukture podataka u kojoj je svaki čvor stabla predstavljen jednim objektom
- Objekat sadrži polje - ključ čvora (key)
- Stablu se kao celini pristupa preko spoljašnjeg pokazivača koji nije deo stabla i koji pokazuje na koren stabla
- Svaki čvor u binarnom stablu ima tri dela: **podatke**, **pokazivač na čvor levog potomka**, **pokazivač na čvor desnog potomka**

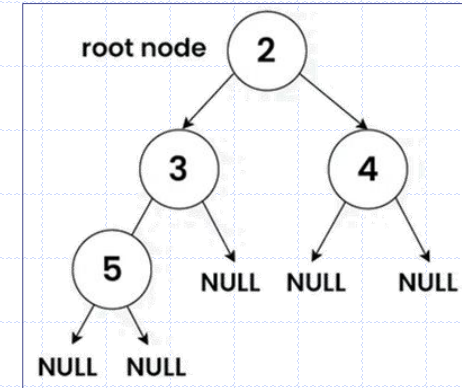
Predstavljanje binarnog stabla



Python klasa koja predstavlja
čvor binarnog stabla

```
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
```

DEFINISANJE ČVORA
STABLA



Kreiranje binarnog stabla sa 4 čvora 2,3,4,5

```
class Node:
    def __init__(self, d):
        self.data = d
        self.left = None
        self.right = None

# Inicijalizacija i alokacija memorije za
# čvorove stabla
firstNode = Node(2)
secondNode = Node(3)
thirdNode = Node(4)
fourthNode = Node(5)

# Povezivanje čvorova binarnog stabla
firstNode.left = secondNode
firstNode.right = thirdNode
secondNode.left = fourthNode
```

KREIRANJE BINARNOG STABLA SA
ČVOROVIMA {2,3,4,5}

Operacije binarnog stabla

Osnovne operacije za rad sa binarnim stablima su:

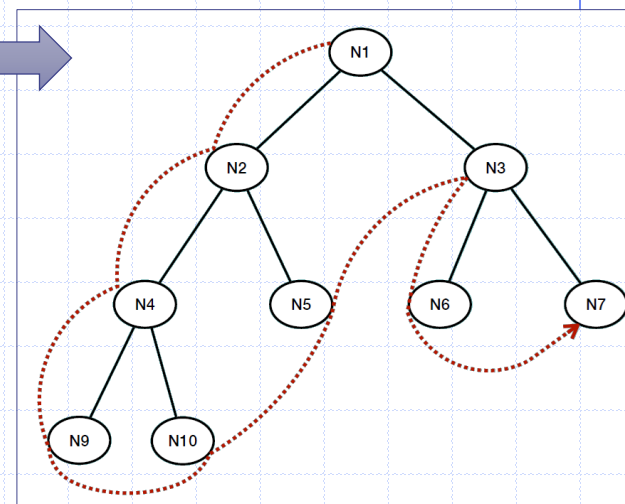
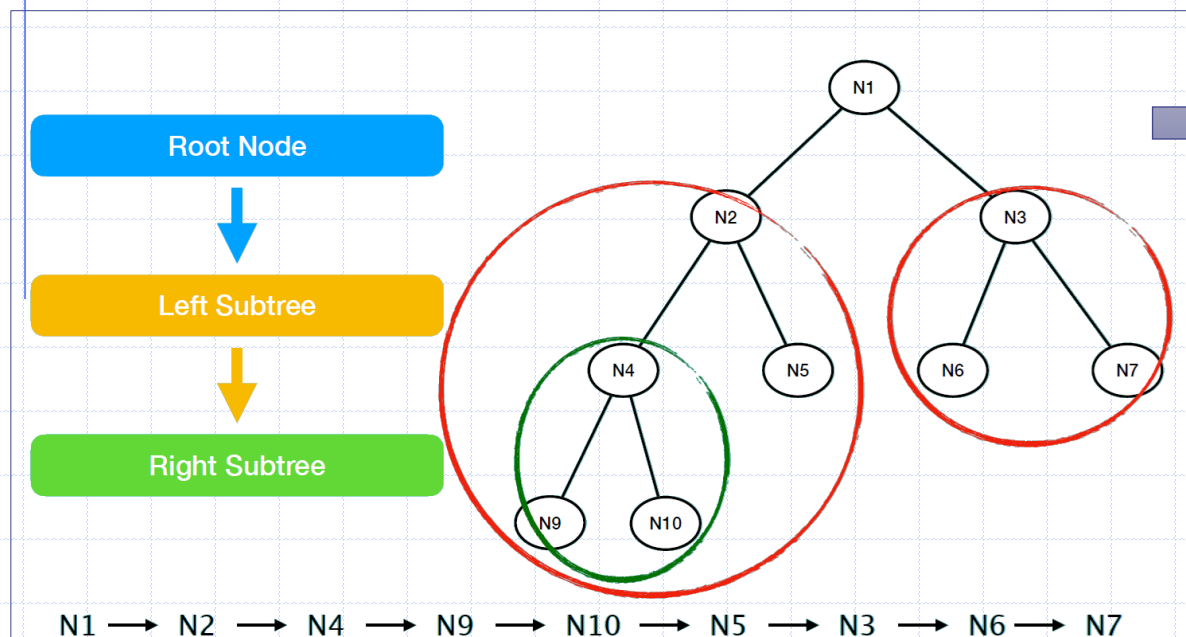
- 1) Obilazak binarnog stabla (traverse)
- 2) Dodavanje novog čvora binarnog stabla (*insert*)
- 3) Pretraživanje binarnog stabla
- 4) Brisanje čvora binarnog stabla

Obilazak binarnog stabla

- Pod obilaskom binarnog stabla (*traverse*) podrazumevamo prolazak kroz sve njegove čvorove određenim redosledom, pri čemu je istovremeno moguće izvršiti i određenu operaciju nad podacima koji su upisani u te čvorove
- Algoritmi za obilazak stabla mogu se klasifikovati u dve kategorije:
 - a) Obilazak po dubini DFS (*Depth-First-Search*), rekurzivno se obilazi stablo po granama kroz što više nivoa
 - b) Obilazak po širini BFS (*Breadth-First Search*) gde se stablo obilazi po nivoima, i implementira korišćenjem redova, naziva se i *Level Order Traversal*
- DFS algoritmi se dele u tri kategorije:
 - 1) PreOrder traversal (prefiksni)
 - 2) PostOrder traversal (postfiksni)
 - 3) InOrder traversal (infiksni)

DFS prefiksni obilazak stabla

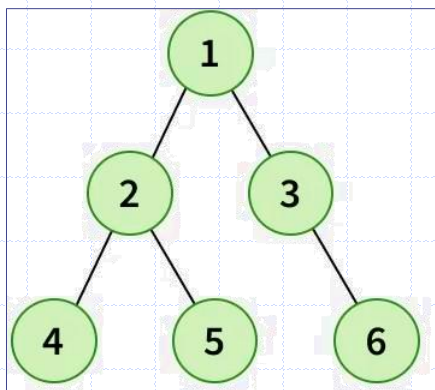
- **Prefiksni obilazak** podrazumeva prvo obilazak root čvora, zatim rekurzivni obilazak levog podstabla, i na kraju rekurzivni obilazak desnog podstabla



- **Algoritam:**
 - Ako je root čvor NULL onda je stablo prazno (return)
 - Procesiranje root čvora (npr. print njegove vrednosti)
 - Rekurzivni prolazak kroz levo podstablo
 - Rekurzivni prolazak kroz desno podstablo

DFS prefiksni obilazak stabla

Primer: Preorder Traversal (Root → Levo → Desno), obilazak 1 → 2 → 4 → 5 → 3 → 6, rezultat obilaska 1 2 4 5 3 6



Struktura čvora binarnog stabla

```
class Node:
```

```
    def __init__(self, v):
```

```
        self.data = v
```

```
        self.left = None
```

```
        self.right = None
```

Funkcija za print preorder obilaska

```
def printPreorder(node):
```

```
    if node is None:
```

```
        return
```

```
# Print podataka čvora
```

```
print(node.data, end=' ')
```

```
# Rekurzivni obilazak levog podstabla
```

```
printPreorder(node.left)
```

```
# Rekurzivni obilazak desnog podstabla
```

```
printPreorder(node.right)
```

```
if __name__ == '__main__':
```

```
    root = Node(1)
```

```
    root.left = Node(2)
```

```
    root.right = Node(3)
```

```
    root.left.left = Node(4)
```

```
    root.left.right = Node(5)
```

```
    root.right.right = Node(6)
```

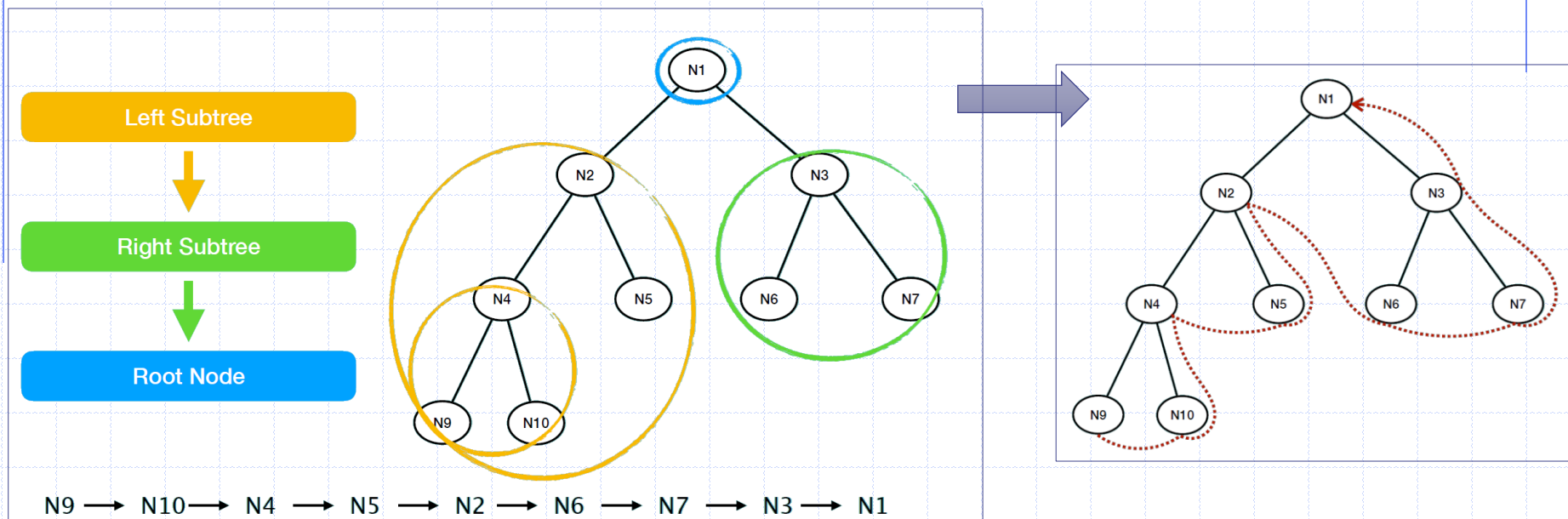
```
    printPreorder(root)
```

Output

```
1 2 4 5 3 6
```

DFS postfiksni obilazak stabla

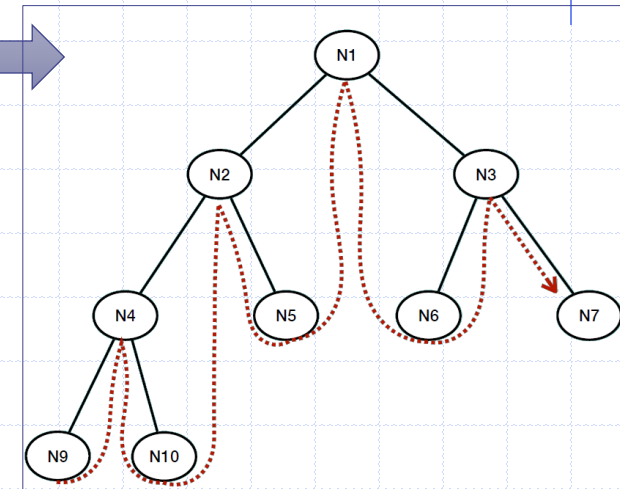
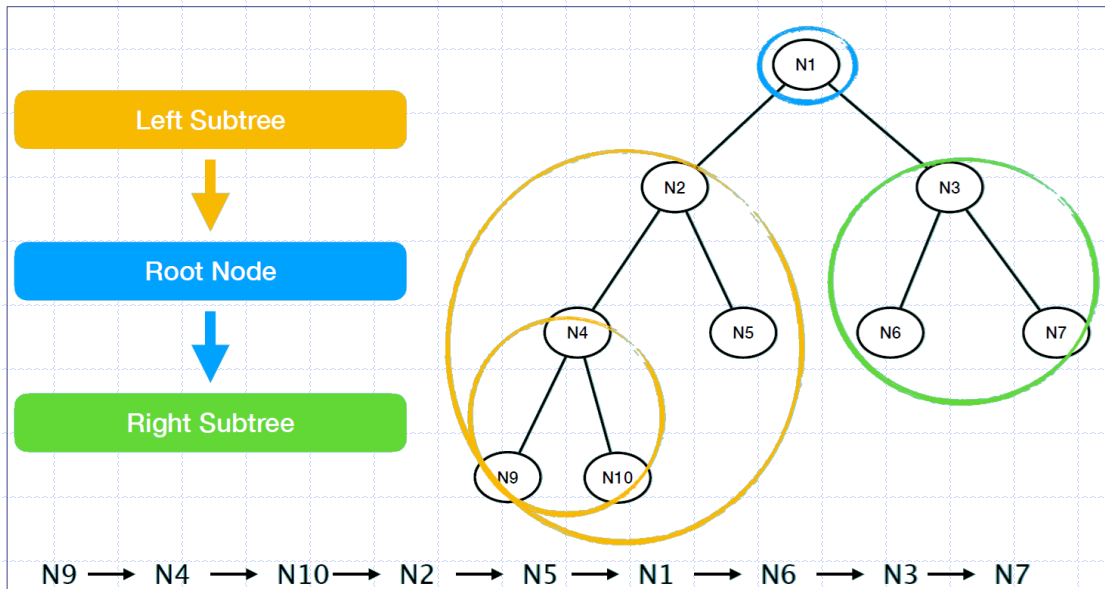
- **Postfiksni obilazak** podrazumeva prvo obilazak levog podstabla, zatim rekurzivni obilazak desnog podstabla, i na kraju obilazak root čvora



- **Algoritam:**
 - Ako je root čvor NULL onda je stablo prazno (return)
 - Rekurzivni prolazak kroz levo podstablo
 - Rekurzivni prolazak kroz desno podstablo
 - Procesiranje root čvora (print njegove vrednosti)

DFS infikсни obilazak stabla

- **Infikсни obilazak** podrazumeva prvo rekurzivni obilazak levog podstabla, zatim obilazak root čvora, i na kraju obilazak desnog podstabla



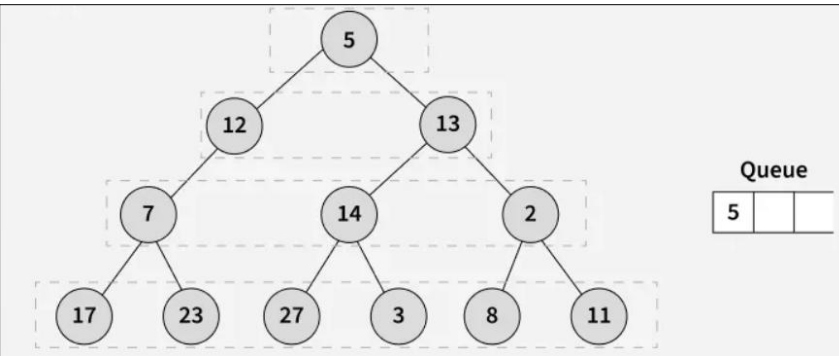
- **Algoritam:**
 - Ako je root čvor NULL onda je stablo prazno (return)
 - Rekurzivni prolazak kroz levo podstablo
 - Procesiranje root čvora (print njegove vrednosti)
 - Rekurzivni prolazak kroz desno podstablo

BFS obilazak stabla

- BFS obilazak stabla u širinu odgovara obilasku stabla po nivoima
- Za implementaciju BFS obilaska stabla u širinu koriste se strukture podataka redovi
- BFS i prefiksni algoritam obilaska su skoro identični, s tim da prefiksni algoritam koristi stek a BFS koristi red za čuvanje vrednosti čvorova stabla
- Algoritmi prefiksnog obilaska i obilaska u širinu primenljivi su za bilo koja stabla, ne samo binarna, a mogu se primeniti i na strukture podataka koje se nazivaju GRAFOVI
- Opšti princip funkcionisanja algoritma obilaska u širinu je sledeći:
 - 1) na početku, u red se stavlja root čvor
 - 2) vrši se obrada čvora koji se nalazi na početku reda, a na kraj reda se smeštaju njegovi potomci
 - 3) iz reda se izbacuje čvor sa početka reda
 - 4) Ukoliko je red prazan, postupak je završen, u suprotnom ide se nazad na korak 2

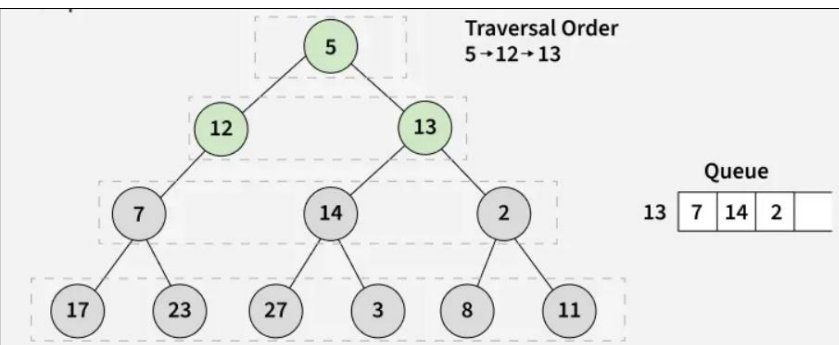
BFS obilazak stabla

BFS obilazak podrazumeva da se čvorovi stabla prelaze nivo po nivo, od vrha do dna tj. od root čvora krećući se nadole, što podrazumeva **FIFO** (*First-In-First-Out*) princip ili „prvi ulazi, prvi izlazi“, uz upotrebu redova kao strukture podataka za izvršavanje obilaska stabla



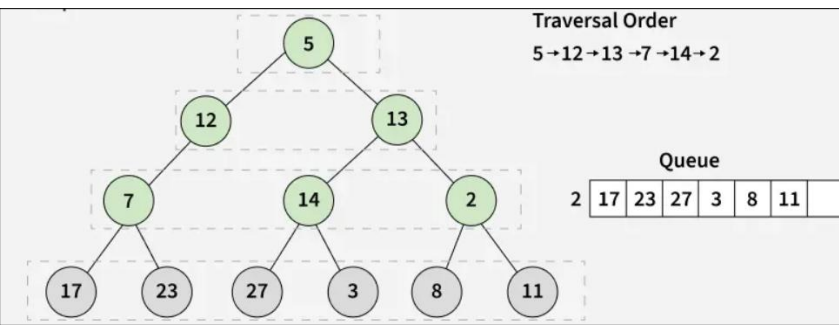
KORAK 1: Dodavanje root čvora 5 na početak reda

... Obilazak stabla se nastavlja ...



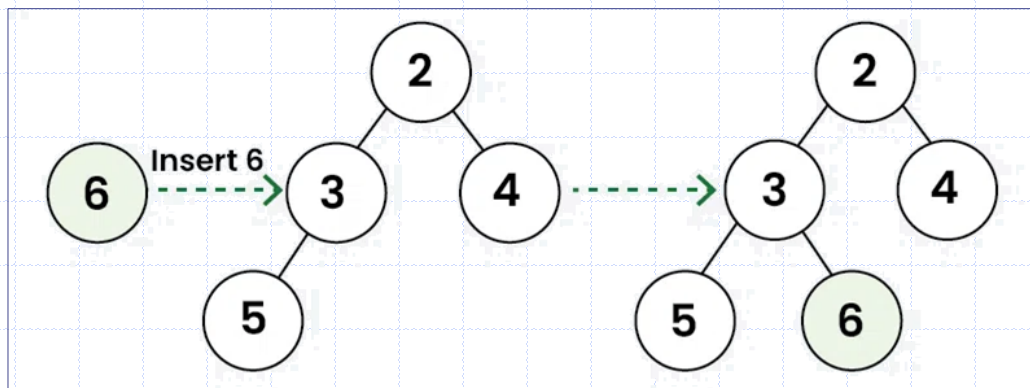
KORAK 4: Izbaciti (*pop*) prvi element **13** iz reda, dodati levi potomak čvor **14**, zatim desni potomak čvor **2** u red respektivno

... Obilazak stabla se nastavlja ...



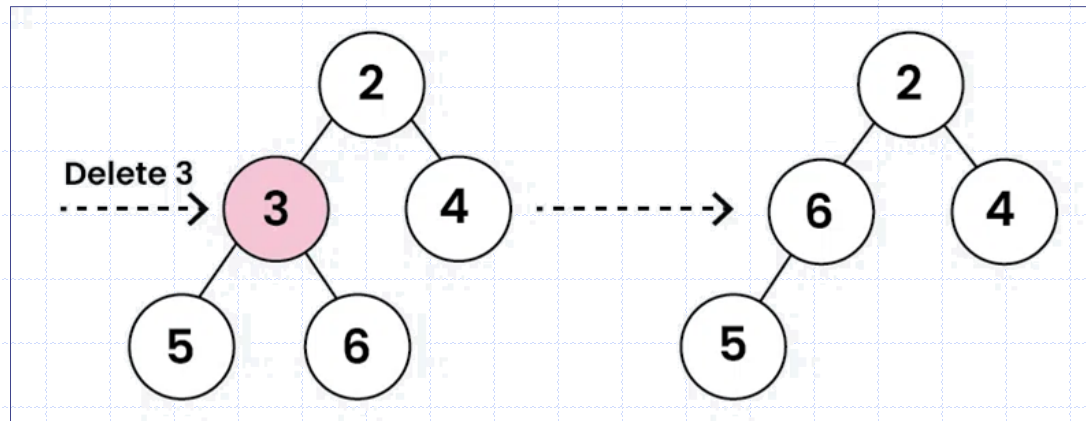
KORAK 7: Izbaciti prvi element **2** iz reda, dodati levi potomak čvor **8**, zatim desni potomak čvor **11** u red respektivno

Dodavanje čvora binarnog stabla



- Da bi se dodao čvor treba pronaći odgovarajući čvor u stablu na koji će se novi čvor nadovezati
- Prvo treba formirati root čvor za slučaj praznog stabla, a dodavanja čvorova uključuju iterativno traženje praznog mesta na svakom nivou stabla
- Kada se pronađe prazan levi ili desni čvor, onda se na to mesto dodaje novi čvor, a podrazumevano dodavanje novog čvora uvek počinje sa levim čvorom
- Rekurzivna operacija dodavanja čvora je sledeća: Ako je **T** prazno stablo, ono postaje stablo koje se sastoji samo od novog čvora **p**. Ako stablo **T** nije prazno, a njegov root čvor ima ključ jednak ključu čvora **p**, tada se **p** već nalazi u stablu; ako stablo **T** nije prazno i njegov root čvor nema istu vrednost kao **p**, tada čvor **p** dodajemo u levo podstablo roota ako je ključ roota veći od ključa čvora **p**, ili dodajemo **p** u desno podstablo roota, ako je ključ roota manji od ključa čvora **p**

Brisanje čvora binarnog stabla



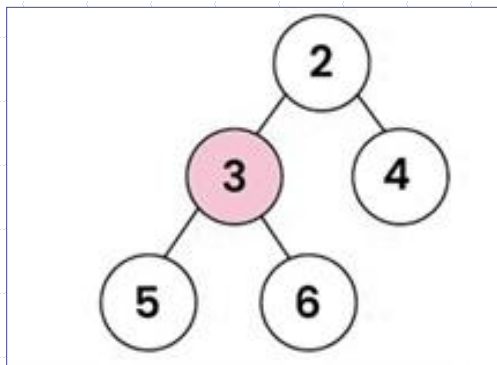
- Brisanje čvora iz binarnog stabla podrazumeva njegovo uklanjanje uz zadržavanje strukture stabla
- Prvo je potrebno pronaći čvor koji treba obrisati nekom od metoda obilaska stabla, a zatim treba zameniti vrednost čvora vrednošću poslednjeg čvora u stablu (pronađenog obilaženjem do krajnjeg desnog lista), a zatim obrisati taj poslednji čvor, čime se ne remeti struktura stabla
- Ako je **p** spoljašnji čvor (list), on se jednostavno briše iz stabla, ali ako je **p** unutrašnji čvor, postoje dva slučaja. Prvi slučaj je kada čvor **p** ima samo jednog potomka i tada se on može zameniti sa njim. Drugi slučaj je kada čvor **p** ima dva potomka, tada se prvo nađe najmanji čvor **q** u desnom podstablu i zatim se zameni čvor **p** čvorom **q** i uklanja se čvor **q** iz desnog podstabla

Pretraživanje binarnog stabla

- Pretraživanje po vrednosti u binarnom stablu znači pronalaženje čvora koji ima određenu vrednost
- Pošto binarna stabla nemaju uređeni raspored čvorova kao binarna stabla pretraživanja, koristi se bilo koja metoda obilaska za pretragu
- Najčešće metode su pretraživanje po dubini (DFS) ili širini (BFS)
- U DFS metodi obilaska počinje se od root čvora, i zatim se vrši pretraživanje čvorova po dubini
- U BFS metodi pretražujemo sve čvorove na trenutnom nivou, pre nego što pređemo na sledeći nivo stabla, nastavlja se postupak dok se pronađe čvor željene vrednosti ili dok se ne stigne do kraja stabla, ako je stablo prazno ili vrednost nije pronađena nakon pretraživanja svih mogućnosti zaključuje se da vrednost čvora ne postoji u stablu

Pretraživanje binarnog stabla

Primer: Za dato binarno stablo sa čvorovima 2, 3, 4, 5 i 6 DFS metodom pretrage pronaći da li postoji čvor sa vrednošću 6 u binarnom stablu



```
class Node:
    def __init__(self, d):
        self.data = d
        self.left = None
        self.right = None

# Funkcija pretrage vrednosti čvora putem DFS
def searchDFS(root, value):
    # Osnovni slučaj: prazno stablo ili došli do leaf čvora
    if root is None:
        return False
    # Ako je vrednost čvora jednaka rootu
    if root.data == value:
        return True
```

```
# Rekurzivna pretraga levih/desnih podstabala
left_res = searchDFS(root.left, value)
right_res = searchDFS(root.right, value)

return left_res or right_res

if __name__ == "__main__":
    root = Node(2)
    root.left = Node(3)
    root.right = Node(4)
    root.left.left = Node(5)
    root.left.right = Node(6)

    value = 6
    if searchDFS(root, value):
        print(f"{value} is found in the binary tree")
    else:
        print(f"{value} is not found in the binary tree")
```

Output

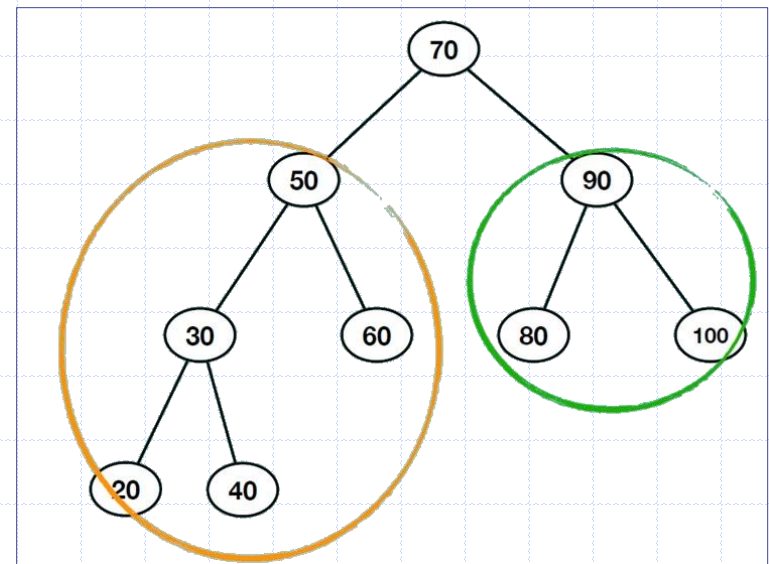
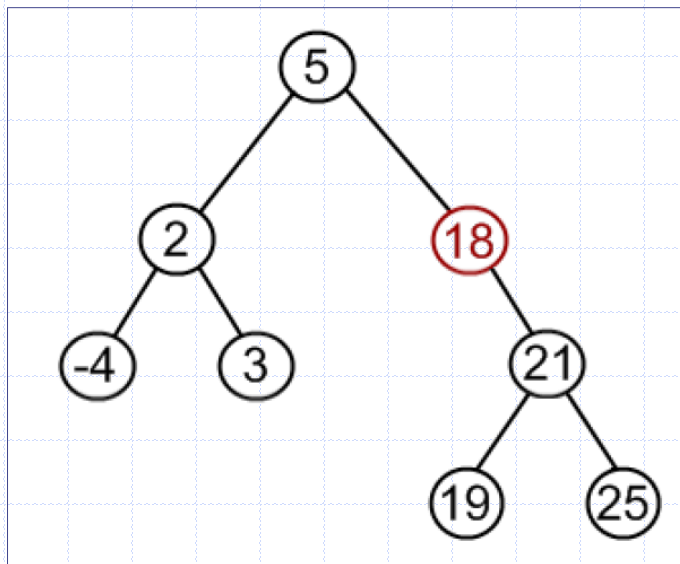
6 is found in the binary tree

Vremenska i prostorna složenost operacija u binarnom stablu

	Vremenska složenost	Prostorna složenost
Obilazak binarnog stabla	$O(n)$	$O(n)$
Dodavanje novog čvora u binarno stablo	$O(n)$	$O(n)$
Pretraživanje binarnog stabla	$O(n)$	$O(n)$
Brisanje čvora binarnog stabla	$O(n)$	$O(n)$

Binarna stabla pretraživanja BST

- **Binarno stablo pretraživanja BST** (*Binary Search Tree*) ili uređeno binarno stablo karakteriše da za svaki čvor **n**, vrednosti svih čvorova iz njegovog levog podstabla su manje ili jednake od vrednosti **n**, a vrednosti svih čvorova iz njegovog desnog podstabla su veće od vrednosti **n**
- BST stabla omogućavaju efikasniju pretragu podataka jer su podaci ispod svakog čvora podeljeni na one koji su "levo" tj. manji, i one koji su "desno" tj. veći od podatka u posmatranom čvoru
- Ta efikasnost se gubi ako je binarno stablo neuravnoteženo (npr. svaki čvor ima samo desnog potomka) ili skoro neuravnoteženo

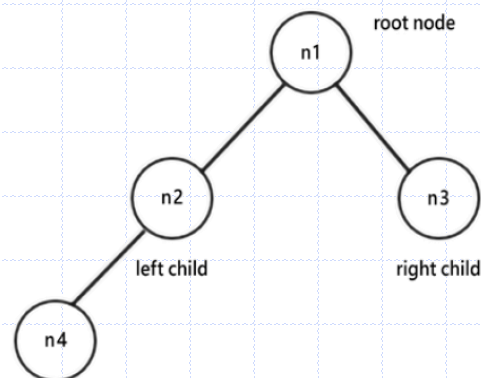


BST stabla - osobine

- **Hijerarhijska struktura:** BST stablo se sastoji od čvorova sa najviše dva potomka, formirajući hijerarhiju sličnu stablu sa root čvorom na vrhu
- **Svojstvo redosleda:** Za svaki čvor BST stabla, sve vrednosti u levom podstablu su manje, a sve vrednosti u desnom podstablu su veće od vrednosti čvora (ključa) i ovo pravilo važi rekurzivno za sva podstabla
- **Efikasnost operacija:** U balansiranom BST stablu, operacije poput pretraživanja, umetanja i brisanja čvorova mogu se izvršiti za vreme $O(\log n)$, dok je za najgori slučaj nebalansirang stabla ovo vreme $O(n)$
- **Svojstvo rekurzivnosti:** Svako levo ili desno podstablo čvora u BST stablu je samo po sebi BST stablo, što omogućava efikasnu primenu rekurzivnih algoritama
- **Praktična primena:** BST stabla se široko koriste u indeksiranju baza podataka, simboličkim tabelama, upitima opsega i osnova su za napredne strukture poput AVL stabala, Red-Black stabala, u situacijama kada treba održavati sortirani tok podataka

BST stabla - operacije

- Kao i kod ostalih, i kod BST stabala se polazi od root čvora, jer se podrazumeva da je pristup stablu spolja omogućen jedino pokazivačem na njegov koren
- Tipične operacije nad BST stablima pretrage su:
 - a) dodavanje novog čvora u stablo
 - b) pretraživanje vrednosti čvora u stablu
 - c) uklanjanje datog čvora iz stabla
- Vreme izvršavanja većine operacija nad BST stablom je proporcionalno visini stabla **h**



```
class Node:
    def __init__(self, data):
        self.data = data
        self.right_child = None
        self.left_child = None
```

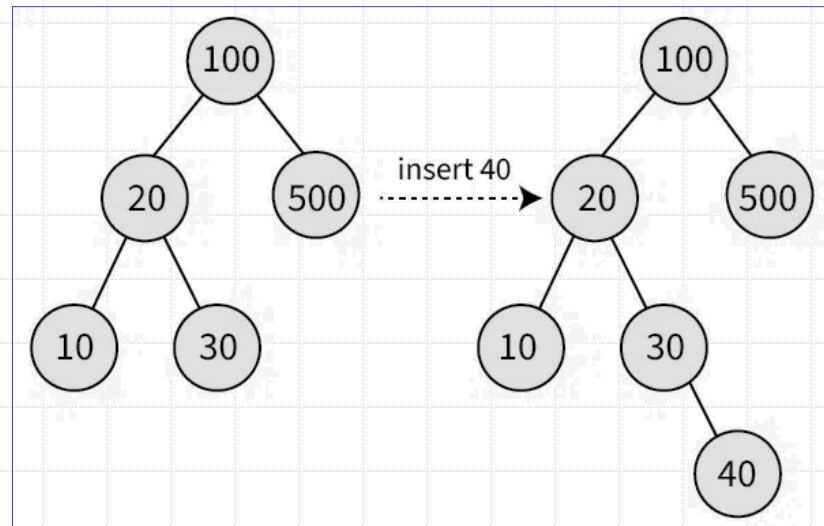
Definisanje klase Node za inicijalizaciju čvora u BST stablu



```
n1 = Node("root node")
n2 = Node("left child node")
n3 = Node("right child node")
n4 = Node("left grandchild node")

n1.left_child = n2
n1.right_child = n3
n2.left_child = n4
```

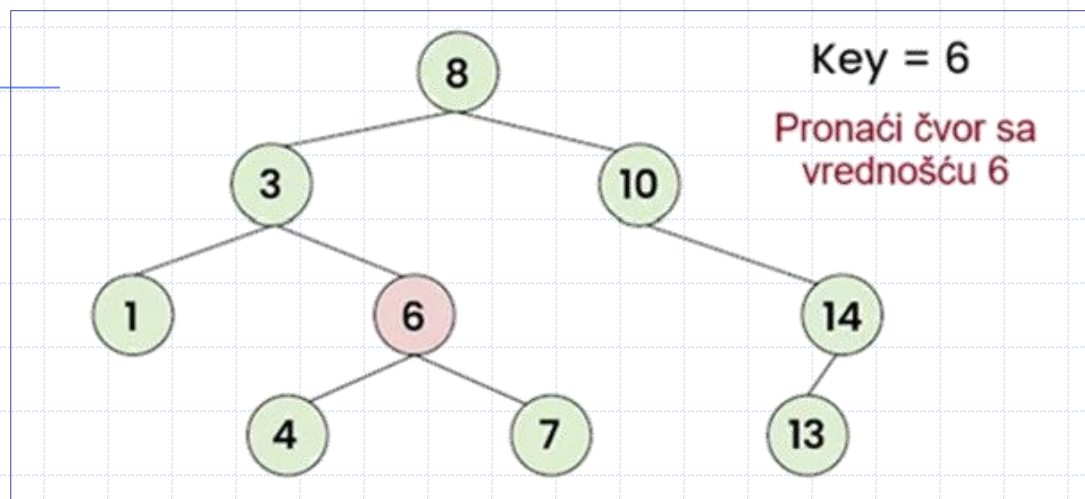
BST stabla - dodavanje čvora



Algoritam (rekurzivni):

- Potrebno je niz brojeva smestiti u uređeno binarno stablo
- Formirati root čvor i prvi broj iz niza smestiti u root
- Za svaku sledeću vrednost **v** iz niza brojeva:
 - Uporediti vrednost **v** sa vrednošću broja u rootu, ako je vrednost **v** manja ili jednaka od vrednosti u rootu, dodati broj u levo podstablo, inače ga dodati u desno
 - Postupak nastaviti sa odgovarajućim podstablom sve dok se ne dođe do lista ili do čvora čije je odgovarajuće podstablo prazno (ima NULL vrednost)
 - Na tom mestu kreirati čvor stabla i broj sa vrednošću **v** smestiti u taj čvor
- Algoritam dodavanja čvora može biti iterativni, i tada se koristi **while** petlja

BST - pretraživanje vrednosti čvora

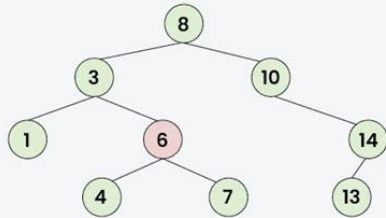


Algoritam (rekurzivni):

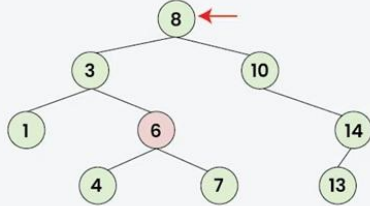
- Operacija nad BST stablom je nalaženje čvora sa vrednošću ključa **X**
- Poredi se vrednost ključa **X** se ključem root čvora, i moguća su tri slučaja:
 - I slučaj: vrednost **X** i root ključa su jednake, pronađen je čvor i pretraga se završava
 - II slučaj: ako je root ključ veći od **X**, prema uslovu binarne pretrage čvor sa ključem **X** može biti samo u levom podstablu roota, pretraga se rekurzivno nastavlja od korena levog stabla
 - III slučaj: ako je root ključ manji od **X**, tada čvor sa ključem **X** može biti samo u desnom podstablu roota i rekurzivna pretraga se nastavlja od korena desnog podstabla

BST - pretraživanje vrednosti čvora

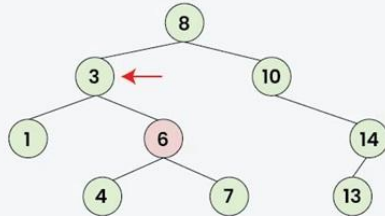
01 | Key = 6



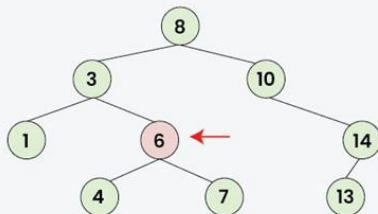
02 | Key se poredi sa rootom tj. 8
Pošto je 6<8, pretraga je u levom podstablu od 8



03 | Key je veći od 3, tj. 6>3
Pretraga u desnom podstablu od 3



04 | 6=Key
Čvor je pronađen

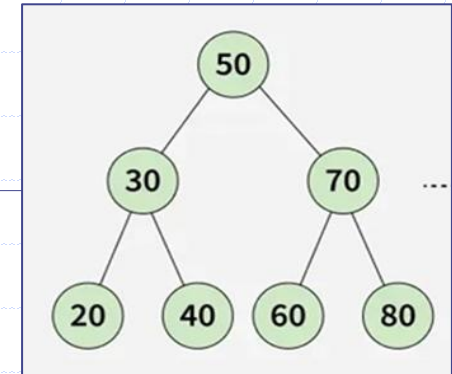


```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

# Funkcija pretrage ključa u BST stablu
def search(root, key):
    # Slučaj: root je null ili key je jednak rootu
    if root is None or root.key == key:
        return root
    # Key je veći od root keya
    if root.key < key:
        return search(root.right, key)
    # Key je manji od root keya
    return search(root.left, key)
```

```
root = Node(50)
root.left = Node(30)
root.right = Node(70)
root.left.left = Node(20)
root.left.right = Node(40)
root.right.left = Node(60)
root.right.right = Node(80)
```

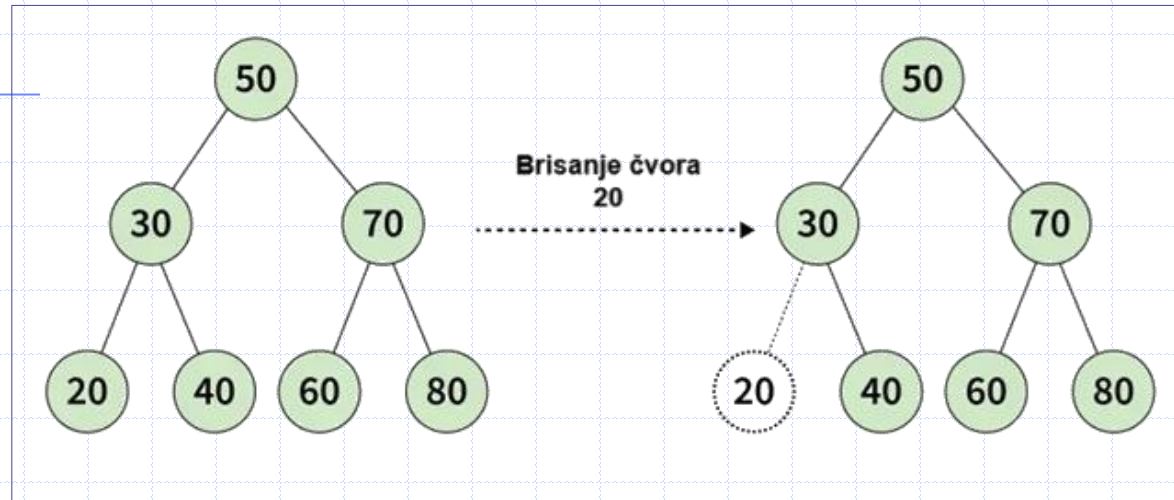
```
# Pretraga ključeva u BST stablu
print("Found" if search(root, 19) else "Not Found")
print("Found" if search(root, 80) else "Not Found")
```



Output

Not Found
Found

BST stablo - brisanje čvora



- Brisanje čvora iz BST stabla podrazumeva njegovo uklanjanje, uz zadržavanje strukture stabla
- Najjednostavniji način je brisanje spoljašnjeg čvora (lista)
- Za unutrašnji čvor postoje dva slučaja:
 - I slučaj: brisanje unutrašnjeg čvora koji ima samo jedan potomak, u tom slučaju kopirati čvor potomka u čvor koji treba da se briše, a zatim treba obrisati taj čvor
 - II slučaj: kada čvor ima dva potomka, taj slučaj je složeniji jer je potrebno obrisati čvor tako da rezultujuće stablo prati princip BST stabla, postupak podrazumeva da čvor sa dva potomka zamenimo njegovim sledbenikom (prema vrednosti) koji će se nalaziti u desnom podstablu čvora

Vremenska i prostorna složenost operacija u BST stablu

	Vremenska složenost	Prostorna složenost
Dodavanje novog čvora u BST stablo	$O(h)/O(n)$	$O(h)/O(1)$
Pretraživanje vrednosti čvora u BST stablu	$O(h)$	$O(h)$
Brisanje čvora BST stabla	$O(h)$	$O(h)$

Hip struktura podataka

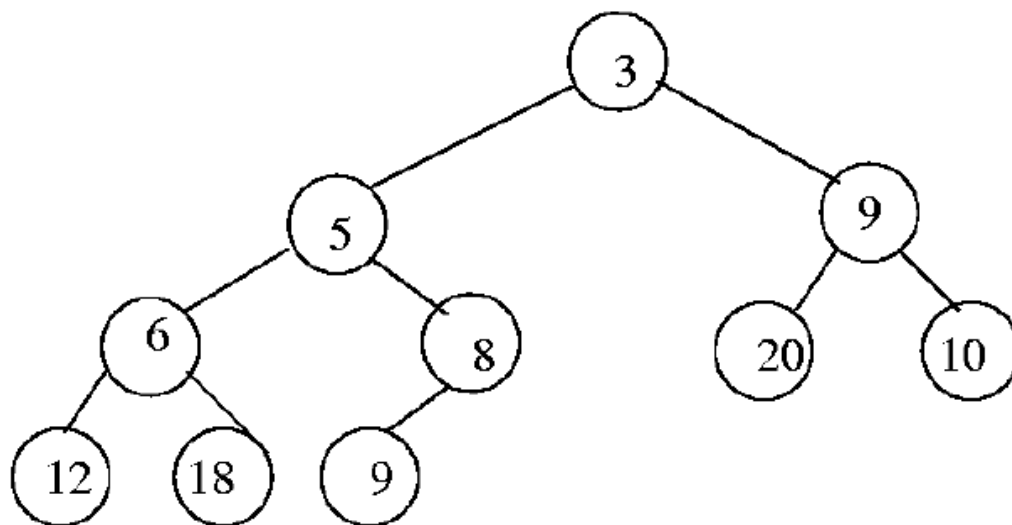
- **Hip** (**heap**, **gomila**) je struktura podataka bazirana na stablu, koja zadovolja sledeće osobine: ako je ključ A elementa roditelj elementa sa ključem B, tada je redosled koji se javlja između bilo koja dva elementa koja su u odnosu roditelj-potomak, isti kao redosled između elementa sa ključem A i elementa sa ključem B
- Osnovna podela hipova: **max heap** i **min heap**
- Kod **max hipa**, ključ elementa roditelj je uvek veći ili jednak od ključeva elemenata potomaka, a najveći ključ je korenski
- Kod **min hipa**, gde je korenski čvor stabla element sa najmanjim ključem, a roditelj je uvek manji ili jednak od svih ključeva elemenata potomaka
- Ovde se detaljnije proučava min hip
- Hipovi se najčešće koriste u nekom od algoritama gde su potrebne određene karakteristike prioritetnih redova

Hip - vrste

- **Binarni hip:** osnovna verzija hipa i najjednostavnija za implementaciju, vremenska složenost operacija **insert**, **extract_min** i **decrease_key** su $O(\log n)$
- **D-array hip:** generalizacija binarnog hipa, kod kojeg se umesto čvorova sa dva potomka javljaju čvorovi sa **d** potomaka
- **Binomijalni hip:** predstavlja skup binomijalnih stabala, tako da svako binomijalno stablo zadovoljava min hip uslov, prosečna vremenska složenost **insert** operacija je linearna dok je za ostale operacije $O(\log n)$
- **Fibonačijev hip:** koristi se u okviru Dijkstra algoritma, operacija **decrease_key** se izvršava u konstantnoj vremenskoj složenosti, dok je za operacije **extract_min** i **delete** vremenska složenost $O(\log n)$
- **Pairing hip:** jednostavnija varijanta Fibonačijevog hipa, jednostavne implementacije, vremenska složenost **extract_min** operacija je $O(\log n)$, operacije **min**, **union** i **insert** imaju konstantnu vremensku složenost
- **2 – 3 hip:** ova vrsta hipa je osmišljena od strana Tadao Takaoka 1999 godine i slična je Fibonačijevom hipu

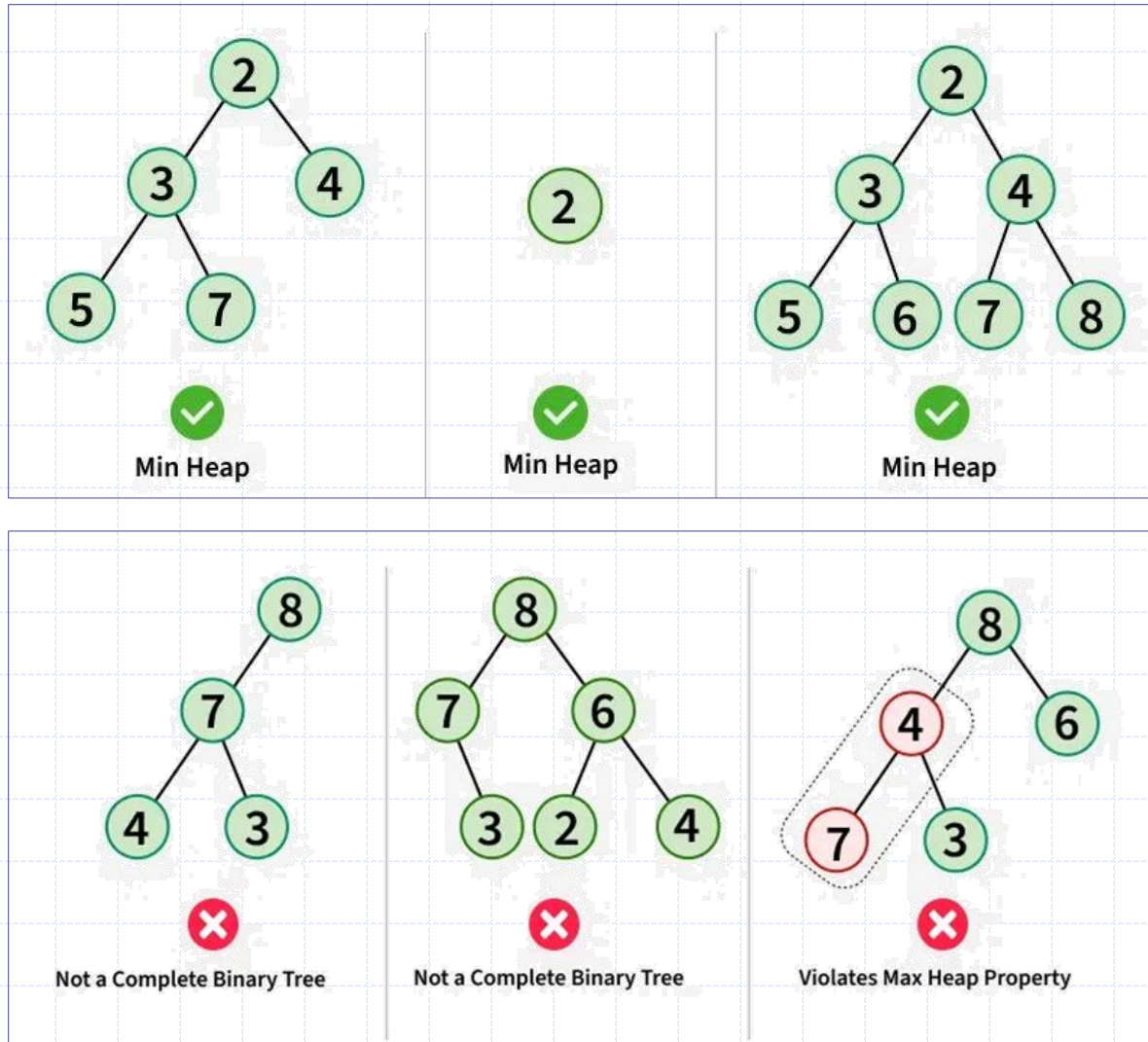
Binarni hip

- **Binarni hip** (binary heap) je kompletno binarno stablo sa elementima u parcijalno uređenom redosledu, tako da je svaki ključ nekog elementa koje je roditelj, manji od ključa elementa levog, odnosno desnog potomka

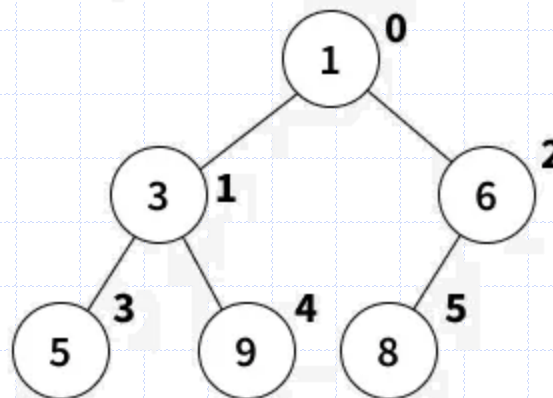
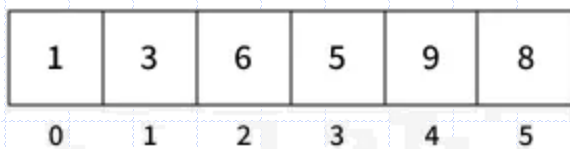


3	5	9	6	8	20	10	12	18	9	.	.	.	.
1	2	3	4	5	6	7	8	9	10	.	.	.	.

Binarni hip - vrste



Binarni hip



- Binarni hip je kompletno binarno stablo čiji se elementi predstavljaju nizom:
 - Element korena je član niza **arr[0]**
 - Ostali elementi niza, sa indeksom **i** ili **arr[i]** su:

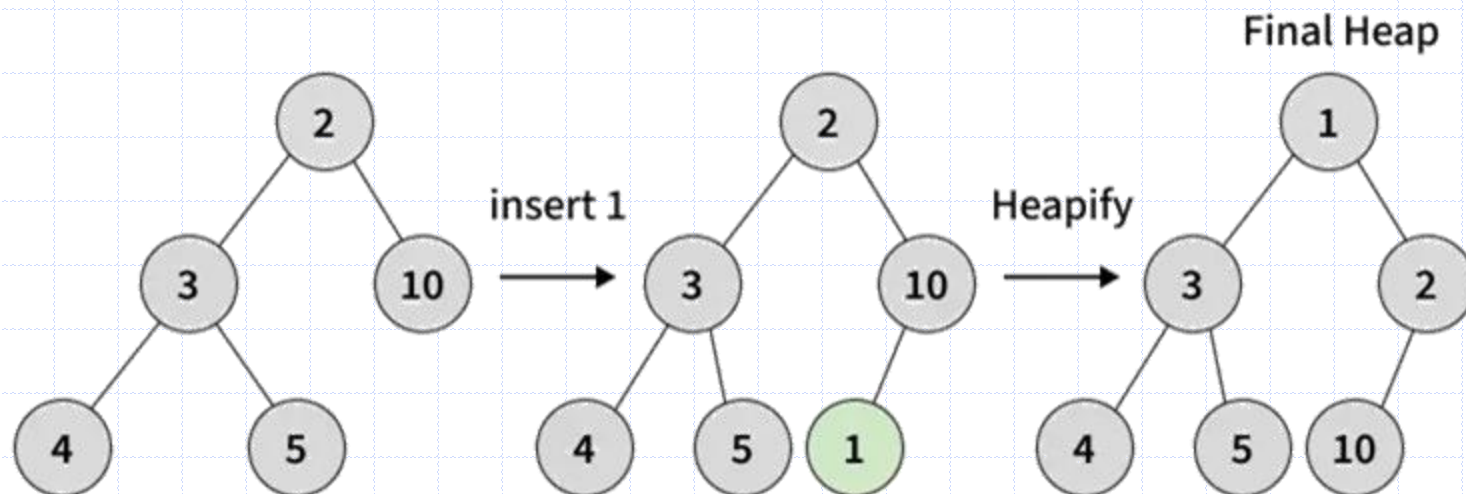
$\text{arr}[(i-1)/2]$	čvor roditelja
$\text{arr}[2*i+1]$	čvor levog potomka
$\text{arr}[2*i+2]$	čvor desnog potomka

Binarni hip

- Binarni hip se češće prikazuje u formi niza nego stabla jer je jednostavno odrediti indekse ekvivalentnog niza
- Ako je pozicija nekog elementa i , pozicija levog potomka je $2i+1$, a desnog potomka $2i+2$ dok se roditeljski čvor nekog elementa nalazi na poziciji $\lceil \frac{i-1}{2} \rceil$
- Hip sa visinom k može imati između 2^k i $2^{k+1}-1$ elemenata
- Hip sa n elemenata ima visinu $\lceil \log_2 n \rceil$
- Osnovne operacije binarnog hipa:
 - kreiranje binarnog hipa
 - peek top binarnog hipa
 - izdvajanje Min/Max elementa
 - prolaz kroz binarni hip
 - umetanje novog elementa u binarni hip
 - brisanje binarnog hipa

Binarni hip

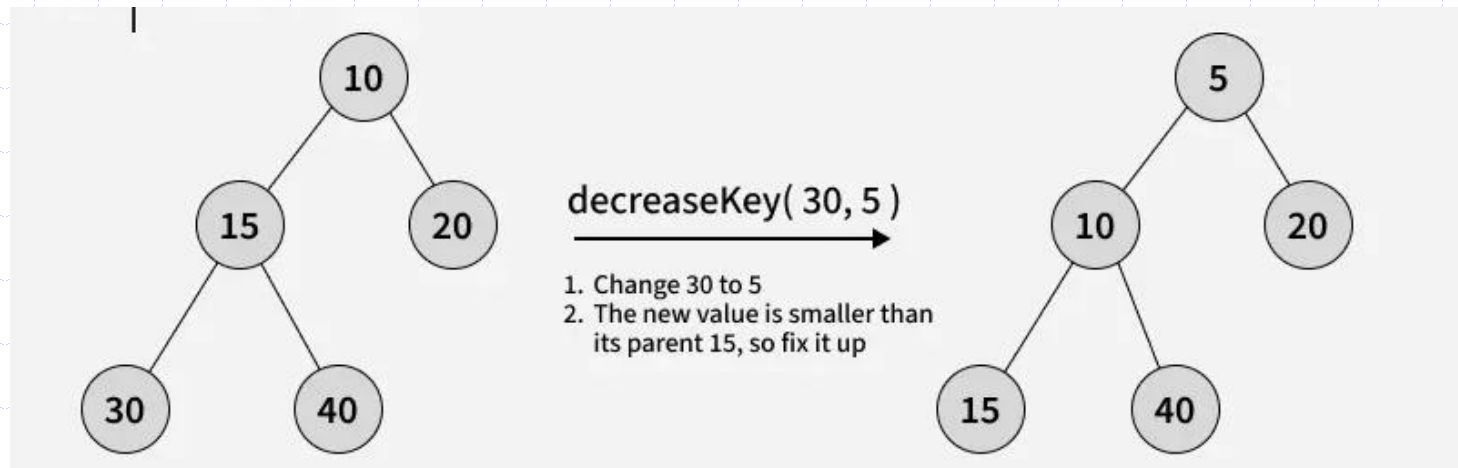
insert(): umetanje novog elementa u hip



- Za umetanje elementa sa ključem **x**, u hip sa **n** elemenata, prvo se stavlja taj element na poziciju **n+1**, a nakon toga se proverava da li je ključ roditelja manji ili jednak ključu potomka
- Ako jeste, pronađeno je odgovarajuće mesto za taj element, u suprotnom, umetnuti element treba zameniti sa elementom roditelja (**zamena na gore**) i nakon toga opet proveriti stanje uslova za hip strukturu

Binarni hip

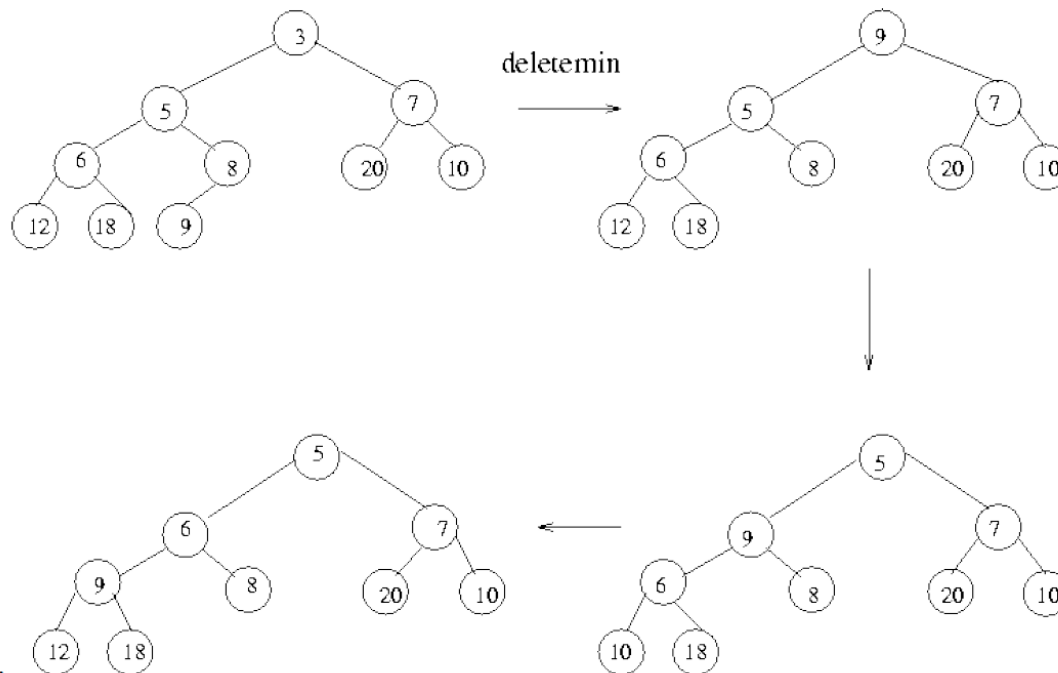
decreaseKey(): smanjivanje vrednosti ključa



- Prvi korak nakon umanjenja ključa je provera da li hip u tom stanju zadovoljava svojstva hipa, ako zadovoljava, postupak se završava, u suprotnom ponavlja se proces kao kod operacije umetanja, gde se radi "zamena na gore", odnosno ključ se propagira ka korenu stabla
- S obzirom da ključ koji se menja može biti i na poslednjem nivou stabla, a propagiranja se može obavljati do korena, vremenska složenost ovog algoritma je logaritamska, odnosno $O(\log n)$

Binarni hip

Brisanje elementa sa najmanjim ključem



- Prvi korak je fizičko uklanjanje korena, odnosno elementa sa najmanjim ključem, a zatim tu poziciju zauzima poslednji element iz hipa. Prebacivanjem poslednjeg elementa u koren, potrebno je proveriti da li u tom trenutku hip zadovoljava svojstva. Ako zadovoljava, brisanje elementa sa najmanjim ključem je završeno, dok se u suprotnom radi zamena novog korena sa elementom potomka koji ima manji ključ (zamena na dole)
- Nakon zamene dolazi do provere da li trenutno stanje zadovoljava svojstva hipa

Vremenska i prostorna složenost operacija binarnog hipa

	Vremenska složenost	Prostorna složenost
Kreiranje binarnog hipa	$O(1)$	$O(n)$
Operacija peek binarnog hipa	$O(1)$	$O(1)$
Prolazak kroz binarni hip	$O(n)$	$O(1)$
Umetanje čvora u binarni hip	$O(\log n)$	$O(\log n)$
Izdvajanje čvora u binarnom hipu	$O(\log n)$	$O(\log n)$
Brisanje binarnog hipa	$O(1)$	$O(1)$

Binarni hip - primene

- **Prioritetni redovi:** Hip se koristi u prioritetnim redovima redovima (elementi većeg prioriteta se izdvajaju prvi), što se primenjuje u operativnim sistemima za raspoređivanje zadataka (*scheduling tasks*), upravljanje prekidima ili procesiranju događaja
- **Algoritmi sortiranja:** **Heapsort** je algoritam sortiranja zasnovan na strukturi binarnog hipa i vremenskom složenosti $O(n \log n)$, pa je pogodan za velike skupove podataka
- **Algoritmi grafova:** Binarni hip se koristi u algoritmima grafova kao npr. Dijkstra algoritam, Primov algoritam ili A* algoritam pretrage
- **Lossless kompresija:** U algoritmima kompresije bez gubitaka kao npr. Huffman kôd, koji koristi prioritetni red implementiran kao min-heap
- **Medicinske aplikacije:** U medicinskom softveru i aplikacijama hip se koristi za upravljanje i čuvanje podataka o pacijentima u formi prioritetnih redova baziranih na npr. vitalnim znacima, rezultatima testova ili lečenju pacijenata

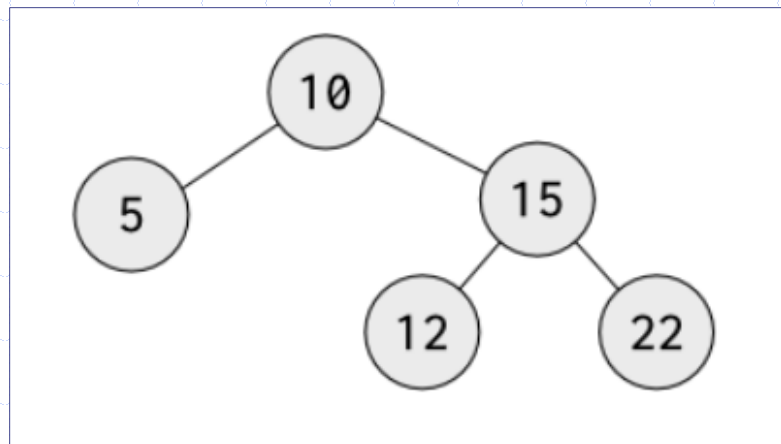
Binarni hip - primene

- **Balansiranje opterećenja** (load balancing): Hip se koristi u mrežama servera za algoritme balansiranja opterećenja zadataka ili zahteva servera, prioritetskim procesiranjem elemenata sa najmanjim opterećenjem
- **Statistika nizova/redova**: Hip struktura podataka se može efikasno koristiti za pronalaženje najmanjeg ili najvećeg elementa niza
- **Alokacija resursa**: Hip se može efikasno koristiti za alokaciju resursa računara, kao npr. memorisjkih blokova ili CPU vremena, dodelom prioriteta za svaki od resursa i procesiranjem zahteva prema prioritetu
- **Raspoređivanje poslova** (job scheduling): Hip struktura podataka koristi se u algoritmima raspoređivanja poslova, kod kojih se zadaci raspoređuju prema prioritetu ili vremenskom ograničenju, omogućujući efikasan pristup zadacima najvećeg prioriteta

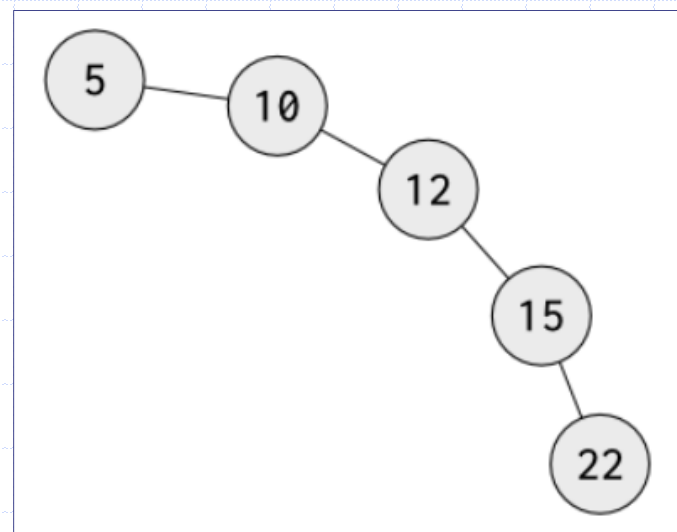
3. Binarna balansirana stabla: AVL i RBT

Balansirana binarna stabla

- U binarnom stablu za efikasno pronalaženje elemenata mora postojati optimalan raspored elemenata ili "balans" koji se prvo treba uspostaviti, a zatim održavati



Binarno stablo koje je **visinski balansirano**, može se koristiti za pretragu. Sve vrednosti u levom podstablu manje su od vrednosti datog čvora, dok su u desnom podstablu sve vrednosti veće

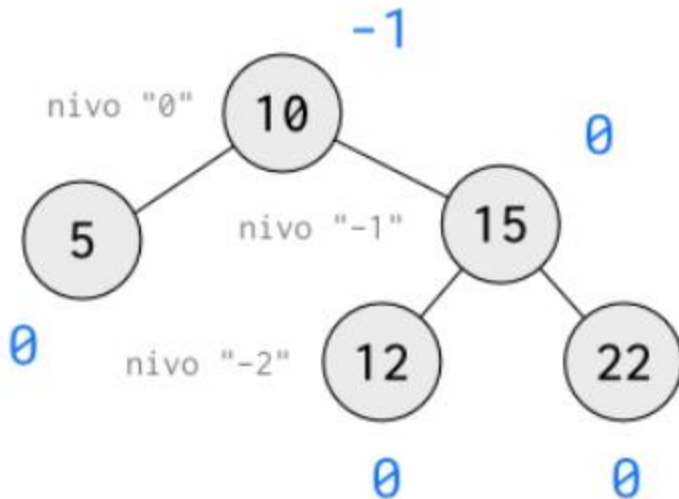


Ako se pri kreiranju stabla čvorovi samo dodaju po rastućem redosledu i ne vodi se računa o balansu stabla, nastaje struktura koja nije balansirana i predstavlja ulančanu listu

Faktor balansiranosti

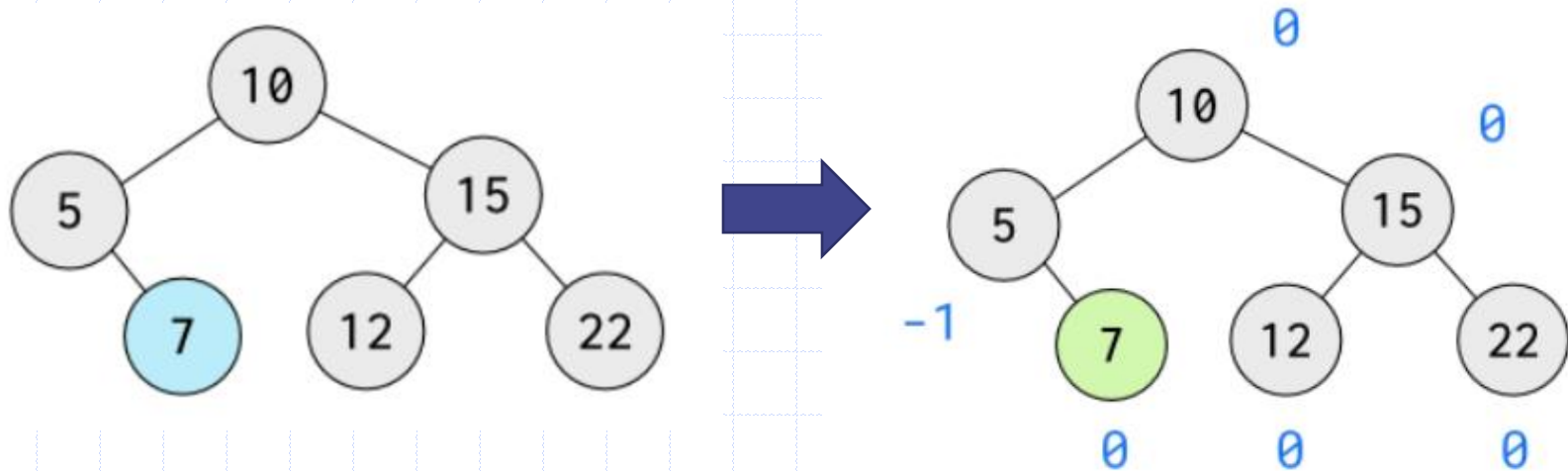
Ako je stablo **visinski balansirano**, tada na svakom čvoru postoji visinska ravnoteža između levog i desnog podstabla (koja su povezana sa datim čvorom), tj. za svaki čvor važi da je razlika između visine levog podstabla i visine desnog podstabla (datog čvora), manja ili jednaka 1 po apsolutnoj vrednosti (tj. dozvoljeno je da razlika bude -1, 0 ili 1)

Faktor_Balansiranosti = VisinaLevogPodstabla - VisinaDesnogPodstabla



- Listovi su čvorovi #5, #12 i #22, imaju faktor balansiranosti **0** ($0-0=0$)
- Oba podstabla čvora #15 imaju visinu 1, faktor balansiranosti je **0** ($1-1=0$)
- Za čvor #10, levo podstablo je visine 1, a desno visine 2 (na I nivou je čvor #15, na II nivou su čvorovi #12 i #22), pa je faktor balansiranosti **-1** ($1-2=-1$)

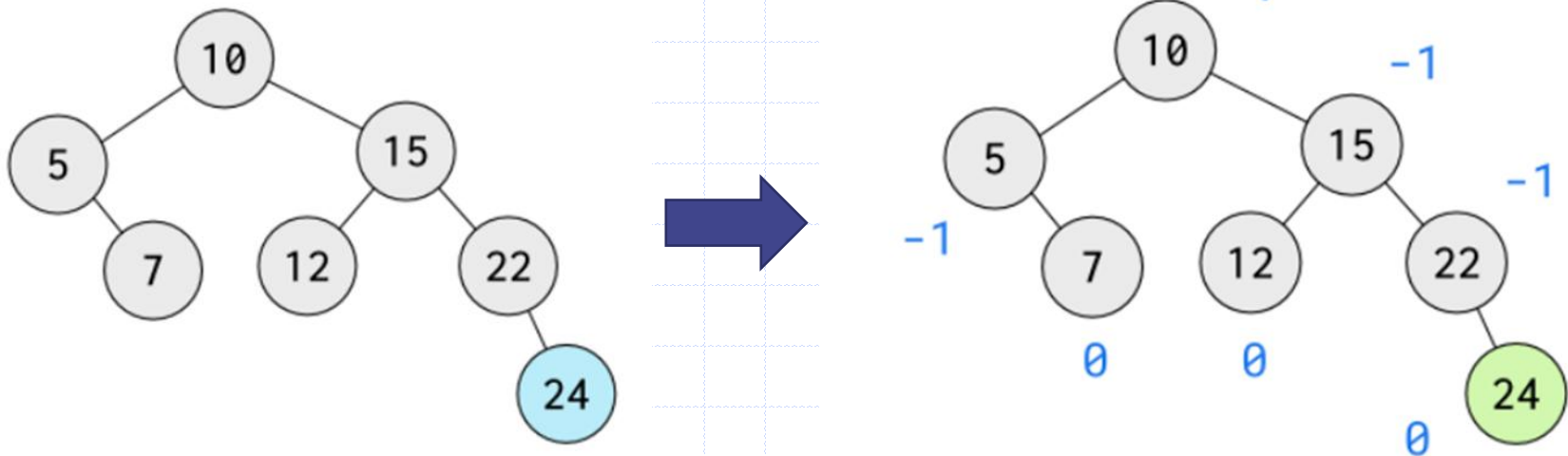
Faktor balansiranosti



Dodavanje čvora #7:

- Po pravilima za dodavanje novih čvorova u binarno stablo, čvor #7 se dodaje na označenu poziciju jer je nj. vrednost manja od 10 pa prelazi se u levo podstablo, i veća od 5, pa se unosi desno posle čvora #5
- Visina levog podstabla je sada 1, faktor balansiranosti čvora #5 je sada -1 ($0-1=-1$), faktor balansiranosti korenog čvora #10 je 0 ($2-2=0$)
- **Balans stabla nije narušen dodavanjem čvora #7**

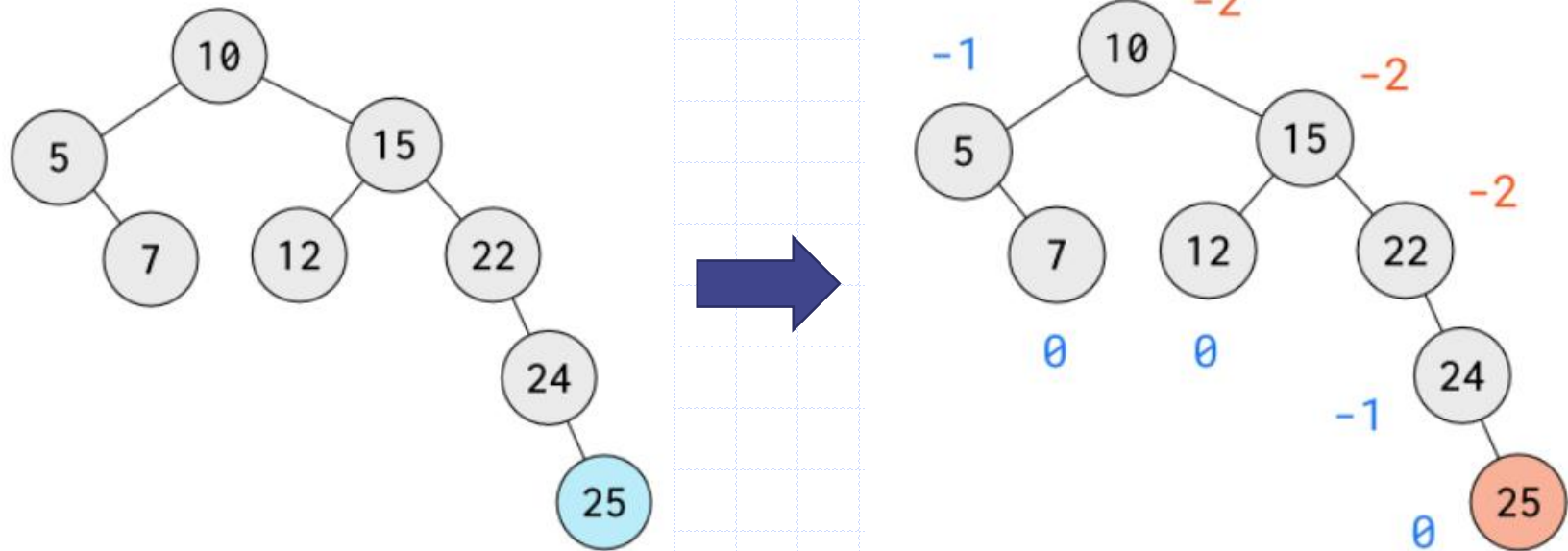
Faktor balansiranosti



Dodavanje čvora #24:

- Po pravilima za dodavanje novih čvorova u binarno stablo, čvor #24 se dodaje na označenu poziciju jer je nj. vrednost veća od 22 pa prelazi u desno podstablo čvora #22, i čvor #24 je ima faktor balansiranosti 0
- Faktor balansiranosti čvora #15 je sada -1 ($1-2=-1$), a faktor balansiranosti korenog čvora #10 je sada -1 ($2-3=-1$)
- **Balans stabla nije narušen dodavanjem čvora #24**

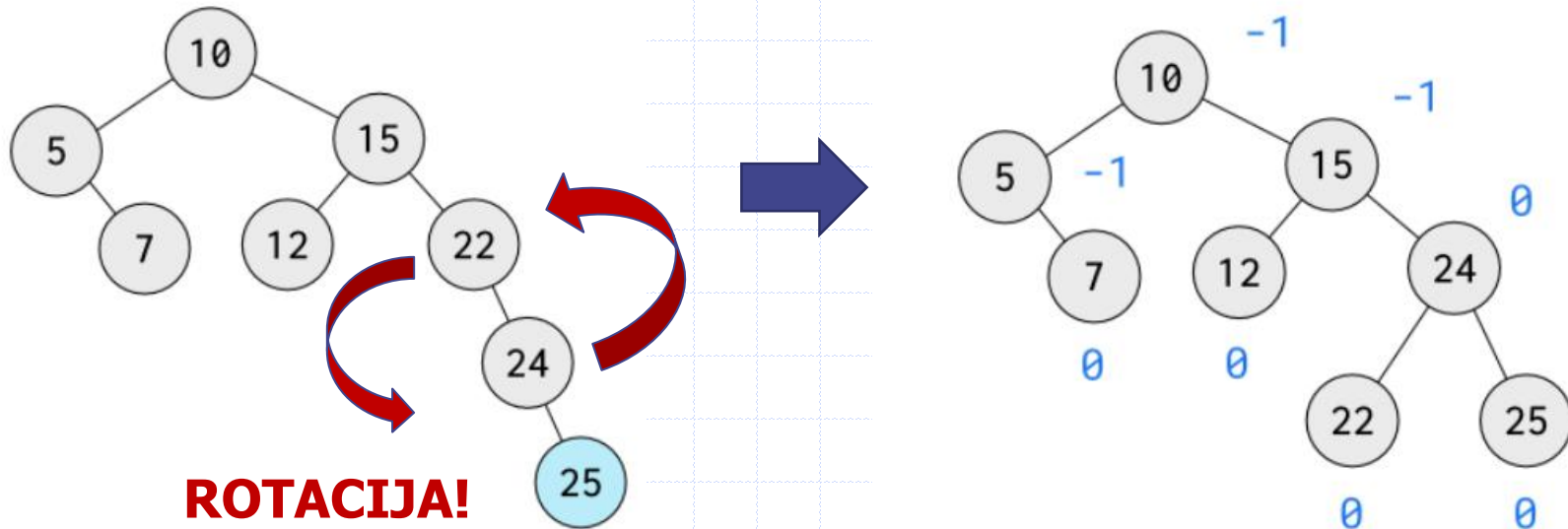
Faktor balansiranosti



Dodavanje čvora #25:

- Čvor #25 se dodaje u desno podstablo čvora #24, i čvor #25 je ima faktor balansiranosti 0
- Faktori balansiranosti: za čvor #24 je sada -1 ($0-1=-1$), za čvor #22 je sada -2 ($0-2=-2$), za čvor #15 je sada -2 ($1-3=-2$), i za čvor #10 je sada -2 ($2-4=-2$)
- **Balans stabla je narušen (nije u ravnoteži) dodavanjem čvora #25**

Faktor balansiranosti



Rotacija čvora #22:

- Da bi stablo bilo vraćeno u stanje ravnoteže, potrebno je povratiti balans 'najnižeg' čvora čiji je balans poremećen
- Traži se 'prvi' tj. najbliži predak dodatog čvora koji ima faktor balansiranosti 2 ili -2 (čvor #22), koji se rotira ulevo, i na njegovo mesto dolazi čvor #24 koji sada ima dva lista #22 i #25
- **Balans stabla je vraćen (stablo opet u ravnoteži) rotacijom čvora #22**

Balansirana binarna stabla

- Uređena binarna stabla su pogodna za implementaciju asocijativnih struktura podataka poput uređenih mapa i uređenih skupova (asocijativne strukture podataka koriste pristup elementima na osnovu vrednosti ključa)
- Ukoliko binarno stablo nije balansirano, operacije pretrage, umetanja i brisanja elemenata mogu zahtevati linearno vreme u odnosu na broj elemenata u stablu (ako u prazno stablo vršimo redom umetanje n elemenata čije su vrednosti ključeva strogo rastuće ili opadajuće, dobija se uređeno binarno stablo visine n)
- **Ideja:** Kreirati uređeno binarno stablo koje bi imalo garantovanu visinu $O(\log n)$ bez obzira na redosled umetanja i brisanja elemenata
- Za takvo binarno stablo bi vremenska složenost operacija pretrage, umetanja i brisanja elementa u najgorem slučaju bila logaritamska u odnosu na broj elemenata u stablu
- Kako bi se stablo održavalo balansiranim, prilikom izvršavanja operacija koje menjaju njegovu strukturu (umetanje ili brisanje elemenata), potrebno je sprovesti dodatne operacije balansiranja stabla

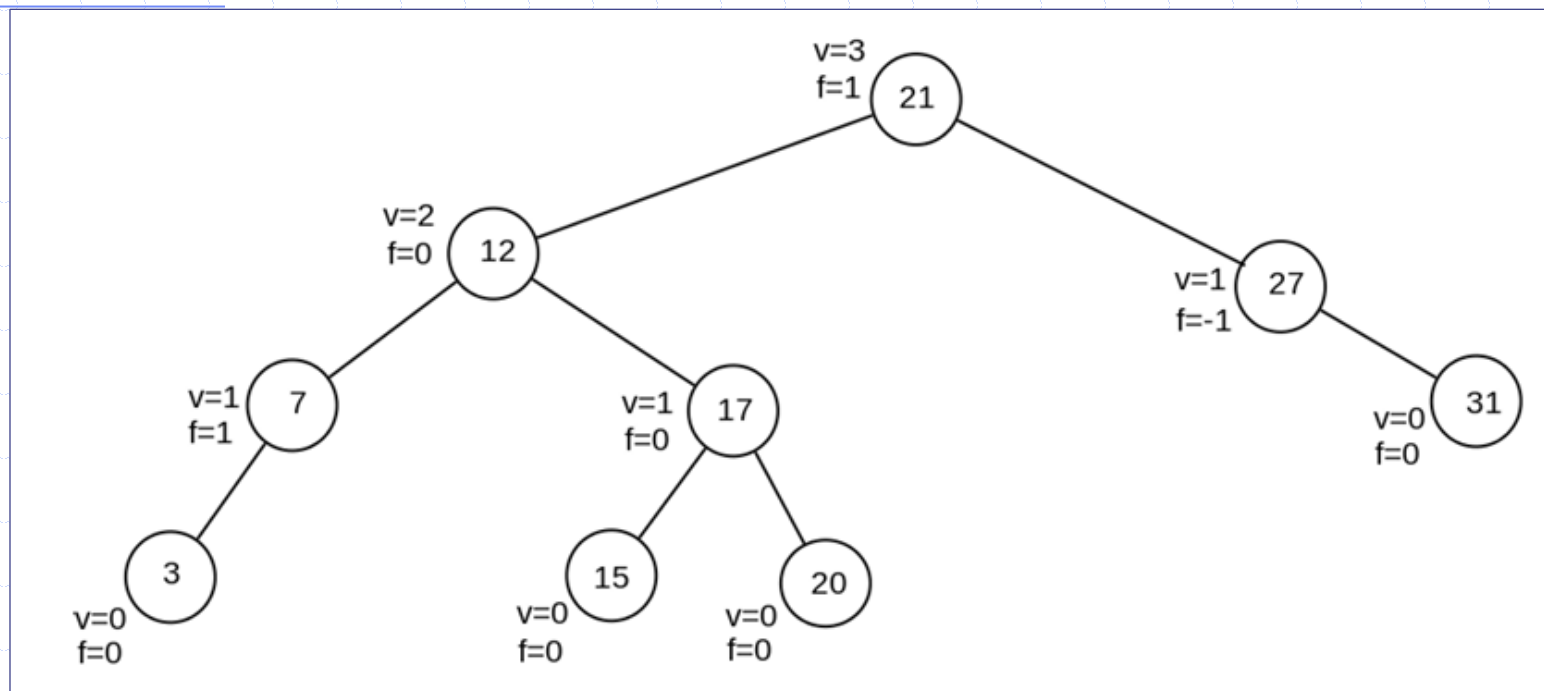
AVL stabla

- **AVL stablo** predstavlja vrstu **balansiranih uređenih binarnih stabala**
- AVL stablo (naziv po prvim slovima prezimena autora, Georgij **A**delson-**V**elski i Jevgenij **L**andis), algoritam objavljen 1962.
- Za AVL stablo važi da složenost operacija traženja, umetanja i brisanja u najgorem slučaju nije veća od $O(\log n)$ gde je n broj elemenata stabla
- To se postiže tako što se posle svake od operacija umetanja i brisanja elemenata ulaže dodatni napor da se stablo balansira, tako da visina stabla uvek bude $O(\log n)$
- Pri tome se balansiranost stabla definiše tako da se može lako održavati
- Osnovna ideja AVL stabla je da za svaki čvor važi da su njegovo levo i desno podstablo približnih visina

AVL stabla

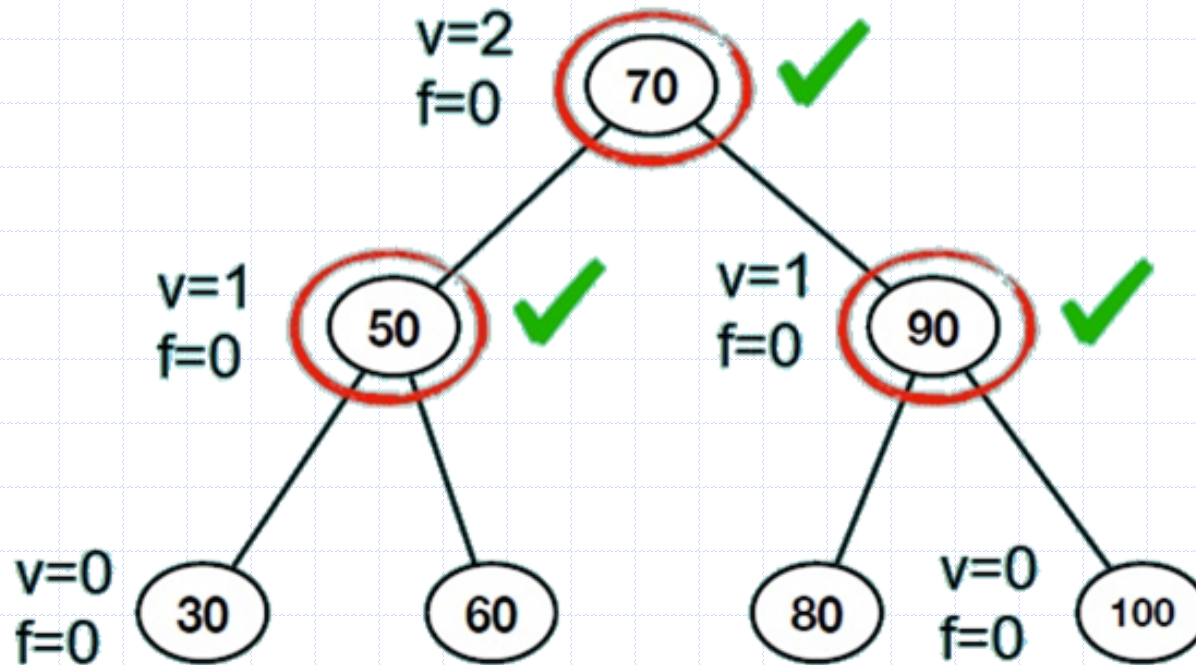
- **Visina stabla** je najveće rastojanje korena do nekog lista i ona se može definisati i kao broja čvorova ili broj grana na najdužem putu od korena do lista
- Po drugoj konvenciji, visina stabla predstavlja maksimalni broj grana na putu od korena do lista, pa stablo koje sadrži jedan čvor ima visinu 0, stablo koje sadrži dva čvora ima visinu 1, dok je prazno stablo visine -1
- **Faktor ravnoteže čvora** je razlika visina levog i desnog podstabla tog čvora
- Ako čvor nema levi (desni) potomak, visina levog (desnog) podstabla je jednaka -1 , kao visina praznog stabla
- AVL stablo se definiše kao uređeno binarno stablo kod koga je za svaki čvor apsolutna vrednost faktora ravnoteže manja ili jednaka od jedan (≤ 1) pa za svaki čvor važi da se visine levog i desnog podstabla mogu razlikovati najviše za jedan

AVL stabla



- **Primer AVL stabla:** uz svaki čvor prikazana je njegova visina v i faktor ravnoteže f čvora
- Faktor ravnoteže čvora sa ključem 7 je 1 jer je visina njegovog levog podstabla 0, a desnog -1, dok je faktor ravnoteže čvora sa ključem 21 jednak 1, jer je visina njegovog levog podstabla 2, a desnog 1
- Faktori ravnoteže svih čvorova datog stabla su iz skupa $\{-1, 0, 1\}$

AVL stabla



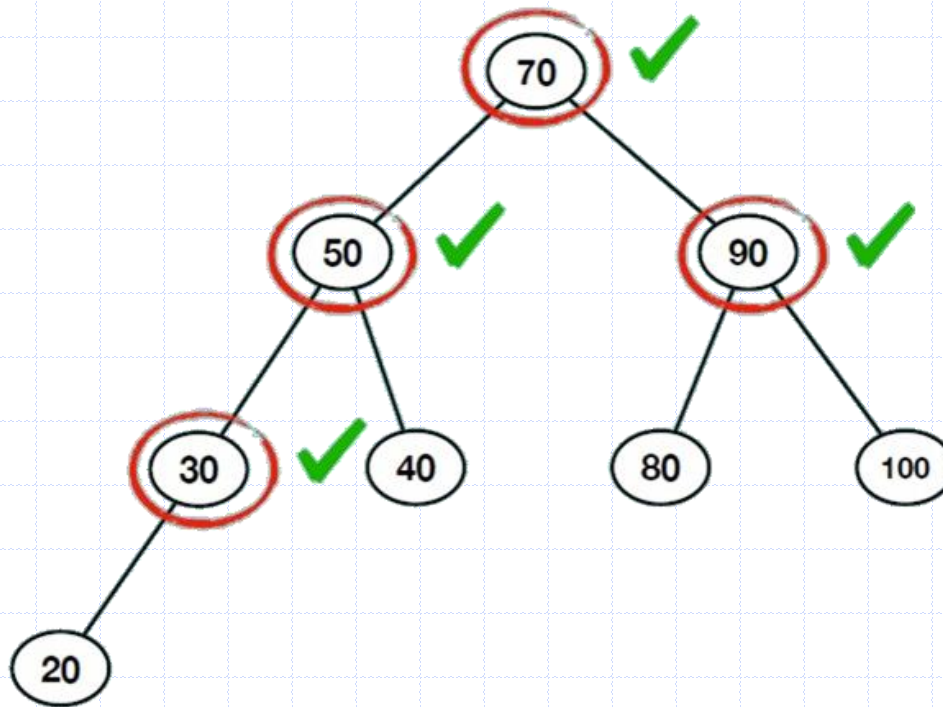
čvor 50: $v_{\text{levog_podstabla}}=1$, $v_{\text{desnog_podstabla}}=1$, $f=0$

čvor 90: $v_{\text{levog_podstabla}}=1$, $v_{\text{desnog_podstabla}}=1$, $f=0$

čvor 70: $v_{\text{levog_podstabla}}=2$, $v_{\text{desnog_podstabla}}=2$, $f=0$

**AVL
stablo**

AVL stabla



čvor 30: $v_{\text{levog_podstabla}}=1$, $v_{\text{desnog_podstabla}}=0$, $f=1$

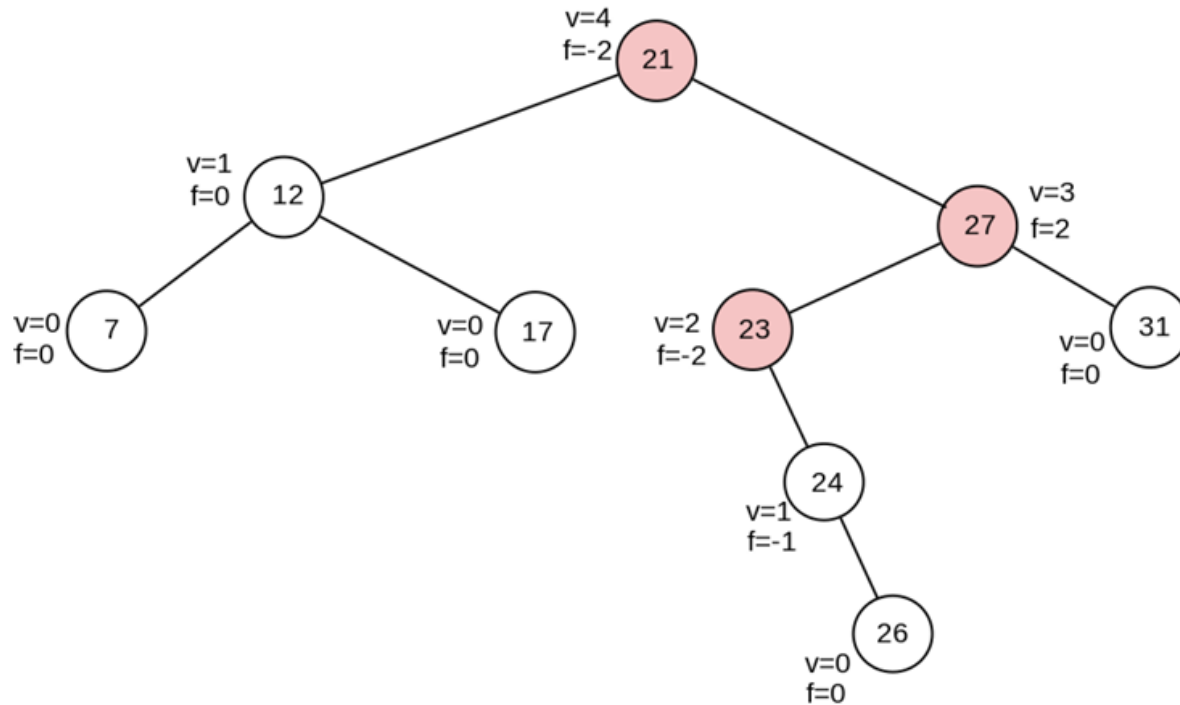
čvor 50: $v_{\text{levog_podstabla}}=2$, $v_{\text{desnog_podstabla}}=1$, $f=1$

čvor 90: $v_{\text{levog_podstabla}}=1$, $v_{\text{desnog_podstabla}}=1$, $f=0$

čvor 70: $v_{\text{levog_podstabla}}=3$, $v_{\text{desnog_podstabla}}=2$, $f=1$

**AVL
stablo**

AVL stabla



čvor 23: $v_{\text{levog_podstabla}}=0, v_{\text{desnog_podstabla}}=2, f=0-2=-2$

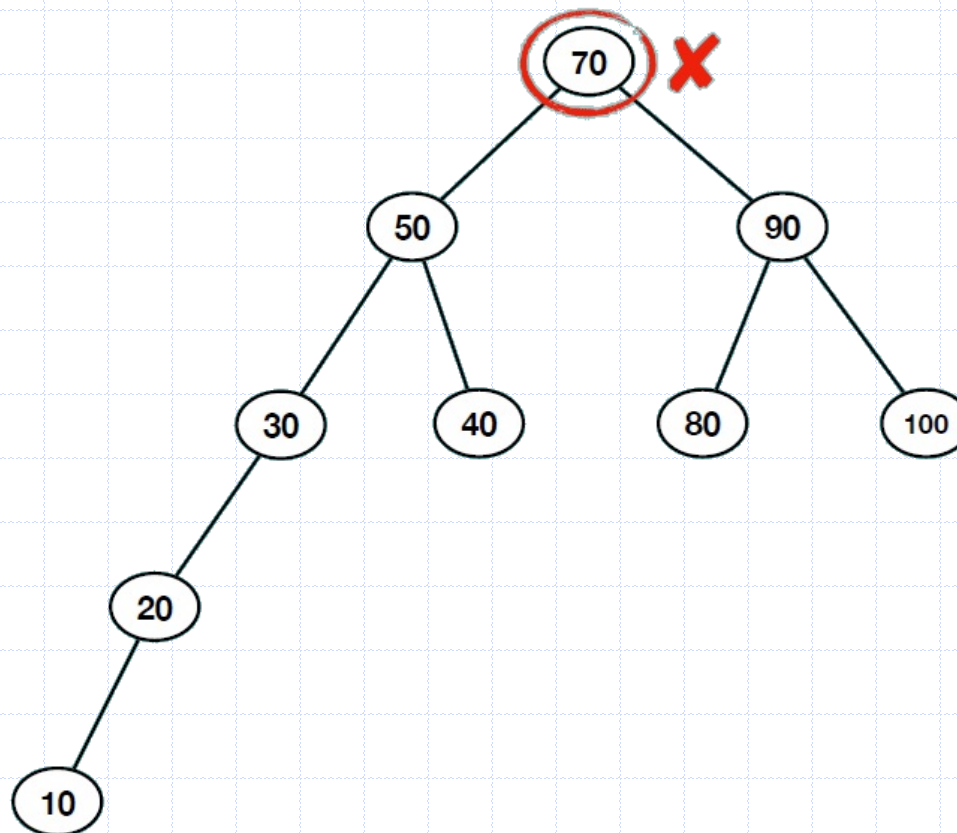
čvor 27: $v_{\text{levog_podstabla}}=3, v_{\text{desnog_podstabla}}=1, f=3-1=2$

čvor 21: $v_{\text{levog_podstabla}}=1, v_{\text{desnog_podstabla}}=3, f=1-3=-2$

Faktori ravnoteže čvorova datog stabla nisu iz skupa $\{-1, 0, 1\}$

**NIJE
AVL
stablo**

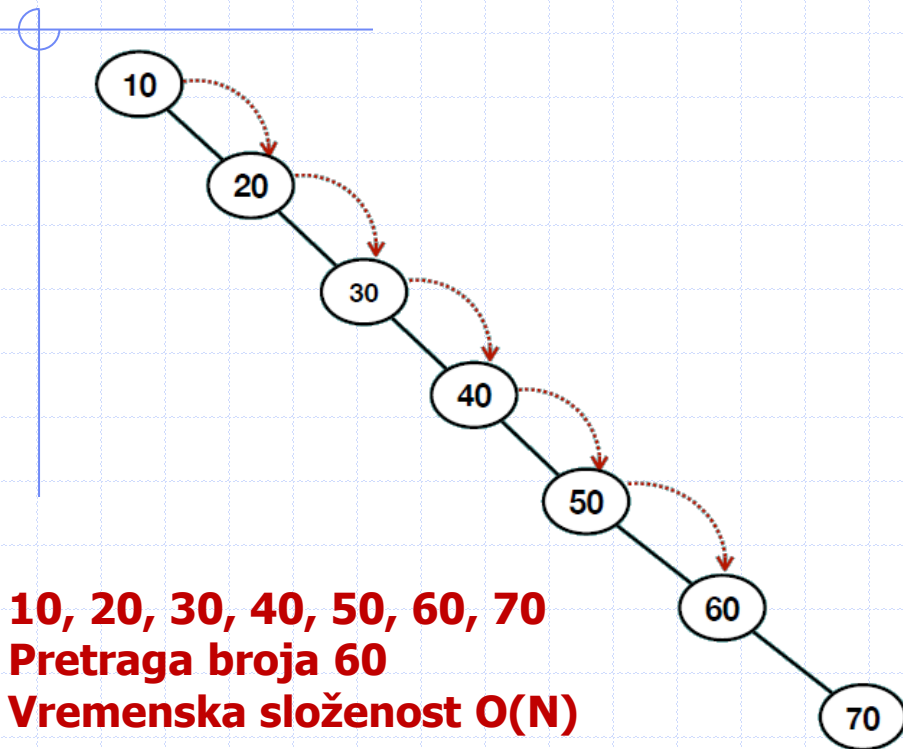
AVL stabla



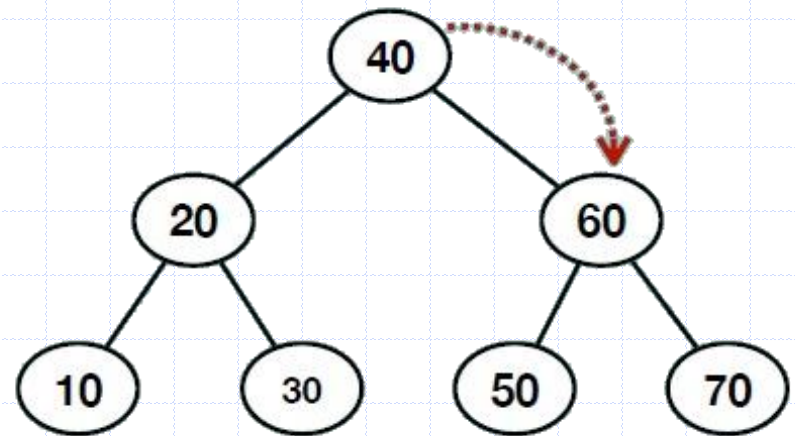
**NIJE
AVL stablo**

čvor 70: $v_{\text{levog_podstabla}}=4$, $v_{\text{desnog_podstabla}}=2$, $f=2$

Zbog čega AVL stabla?

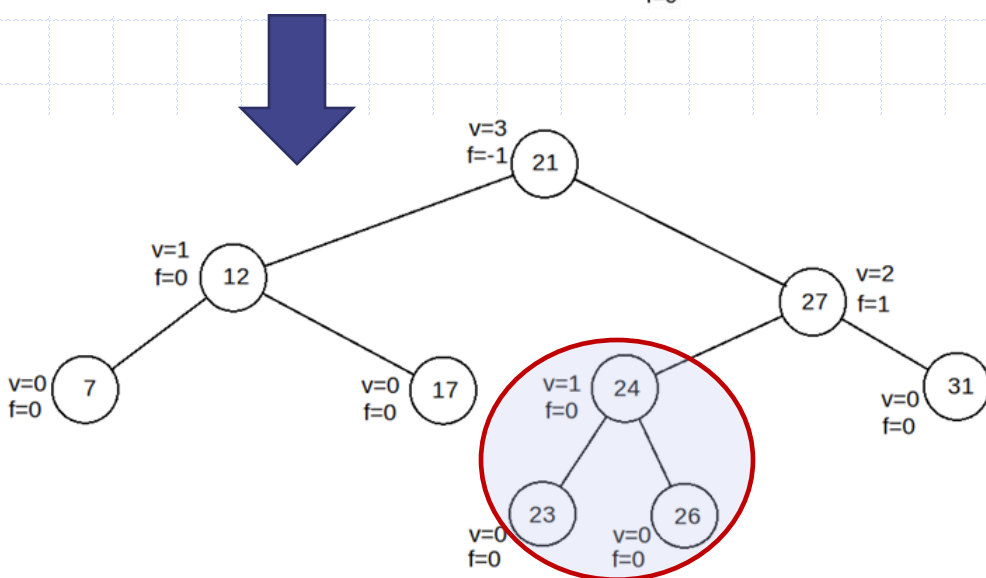
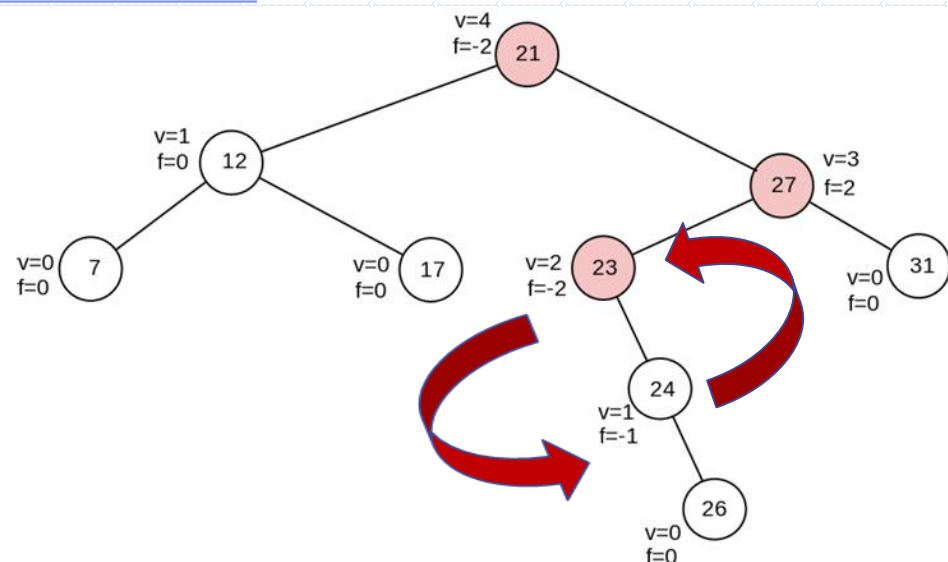


10, 20, 30, 40, 50, 60, 70
Pretraga broja 60
Vremenska složenost $O(\log N)$



- U stablu sa npr. milion elemenata, binarno stablo može da ima visinu 20, dok nebalansirano binarno stablo može imati visinu 1.000.000
- Pretraga ovako velikih ulančanih lista može trajati više minuta, dok pretraga odgovarajućeg binarnog stable traje najviše nekoliko desetina mikrosekundi ili manje

Rotacija AVL stabla



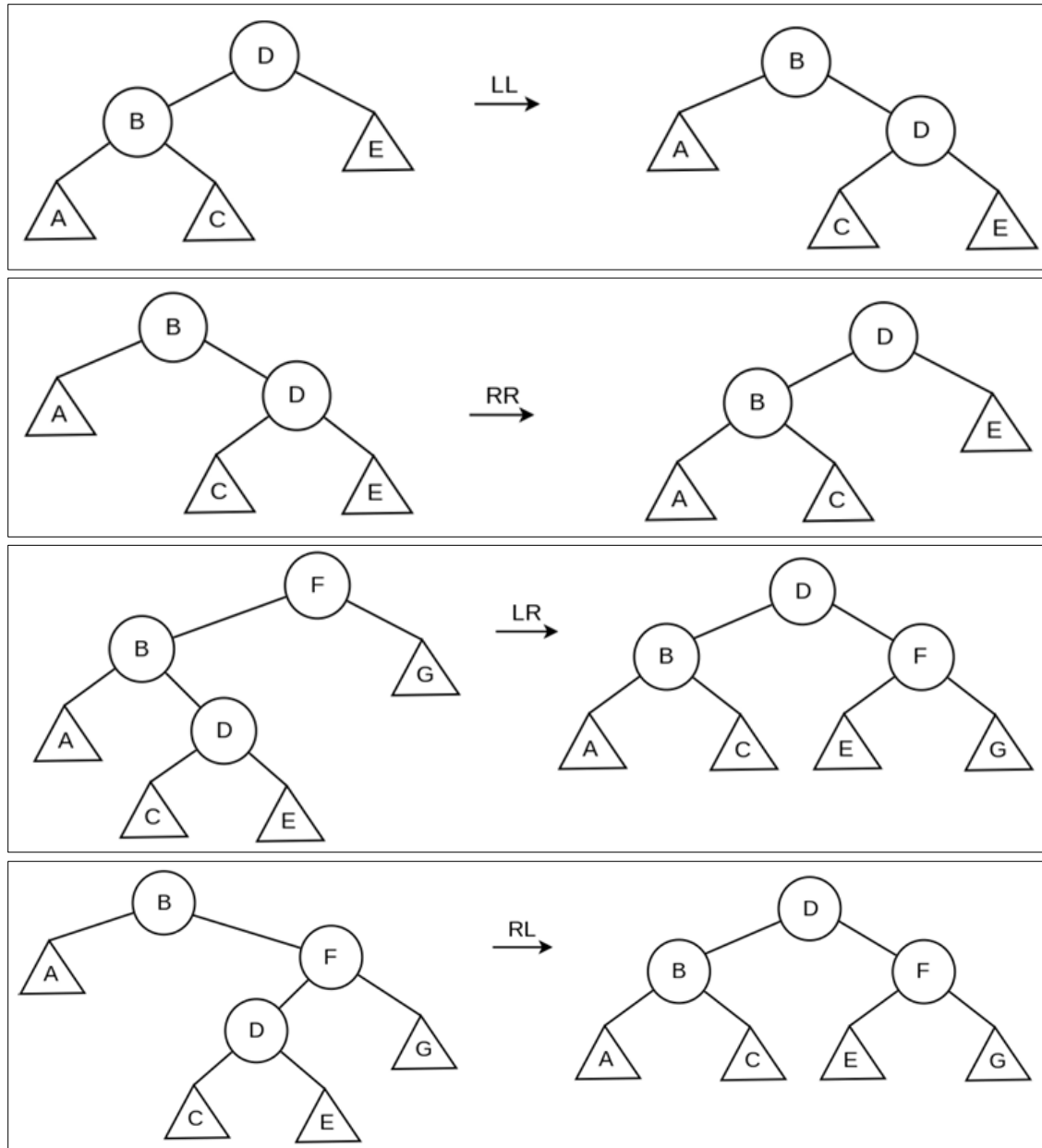
Transformacija ne-AVL stabla u AVL stablo operacijom balansiranja stabla (rotacijom čvora 23)

Teorema: Visina h AVL stabla sa n čvorova zadovoljava uslov $h \leq 2\log_2 n$

Rotacije AVL stabla

- Prilikom umetanja elementa u AVL stablo dovoljno je izbalansirati podstablo sa korenom u kritičnom čvoru (koji ima faktor ravnoteže različit od 0)
- To se postiže različitim tipovima rotacija stabla, koje vrše reorganizaciju čvorova u stablu, održavajući pritom uređenost stabla
- Primjenjuju se 4 tipa rotacija stabla:
 - LL rotacija (novi čvor se dodaje levom podstablu levog podstabla)
 - LR rotacija (novi čvor se dodaje desnom podstablu levog podstabla)
 - RR rotacija (novi čvor se dodaje desnom podstablu desnog podstabla)
 - RL rotacija (novi čvor se dodaje levom podstablu desnog podstabla)
- U implementaciji AVL stabla uz svaki čvor stabla čuva se njegova visina i njegov faktor ravnoteže, jednak razlici visina njegovog levog i desnog podstabla, a te razlike su elementi skupa $\{-1, 0, 1\}$
- U svakoj od razmatranih rotacija važi da visina podstabla kome je koren kritični čvor posle balansiranja ostaje ista kao što je bila pre umetanja novog elementa

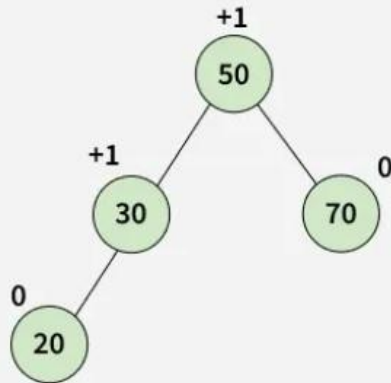
Rotacije AVL stabla



AVL stablo - primer

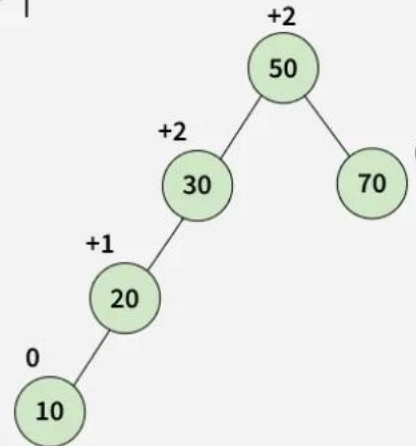
01
Step

Početno BST stablo



02
Step

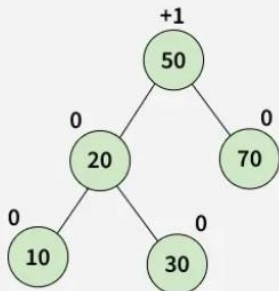
Unos čvora #10



- unos čvora 10 levo od čvora 20
- za čvor 30 je $f=2$ ($2-0=0$)
- potrebna LL rotacija na čvoru 30

03
Step

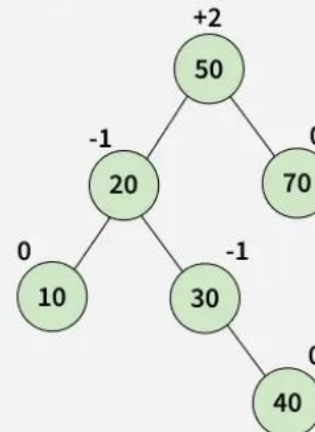
Izvršavanje LL rotacije



Faktor balansiranosti za svaki čvor stabla je u opsegu od -1 do 1

04
Step

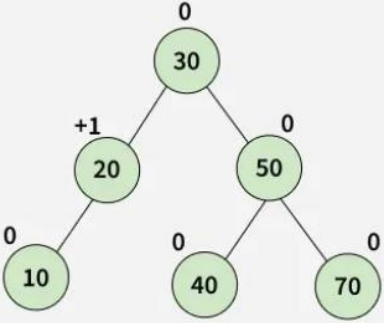
Unos čvora 40



- unos čvora 40 desno od čvora 30
- za čvor 50 je $f=2$ ($3-1=2$)
- potrebna LR rotacija na čvoru 50

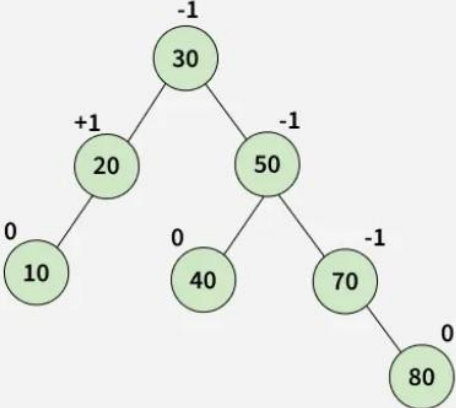
AVL stablo - primer

05 Step | Izvršavanje LR rotacije



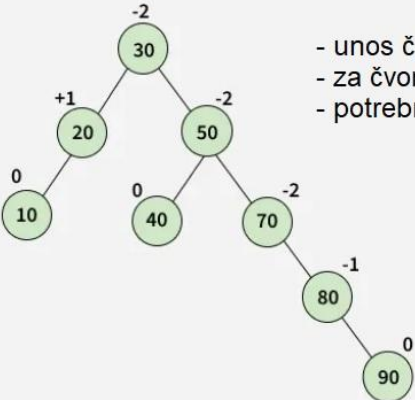
Faktor balansiranosti za svaki čvor stabla je u opsegu od -1 do 1

06 Step | Unos čvora 80



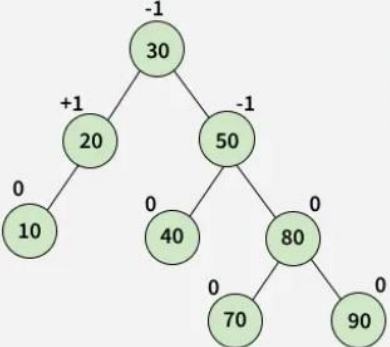
- unos čvora 80 desno od čvora 70

07 Step | Unos čvora 90



- unos čvora 90 desno od čvora 80
- za čvor 70 je $f=-2$ ($0-2=-2$)
- potrebna je RR rotacija na čvoru 70

08 Step | Izvršavanje RR rotacije

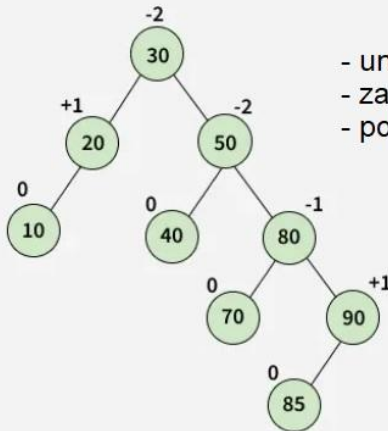


Faktor balansiranosti za svaki čvor stabla je u opsegu od -1 do 1

AVL stablo - primer

09
Step

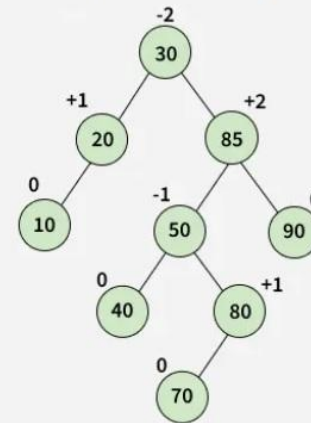
Unos čvora 85



- unos čvora 85 levo od čvora 90
- za čvor 50 je $f=-2$ ($1-3=-2$)
- potrebna je RL rotacija na čvoru 50

10
Step

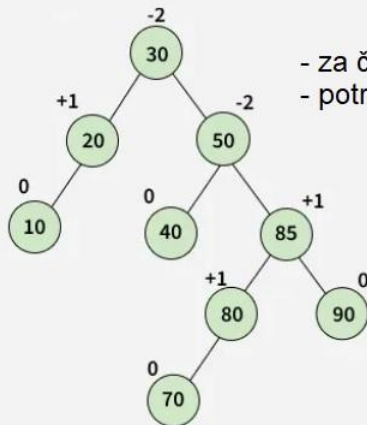
Izvršavanje RL rotacije



- za čvor 85 je $f=2$ ($3-1=2$)
- potrebna je LL rotacija na čvoru 85

11
Step

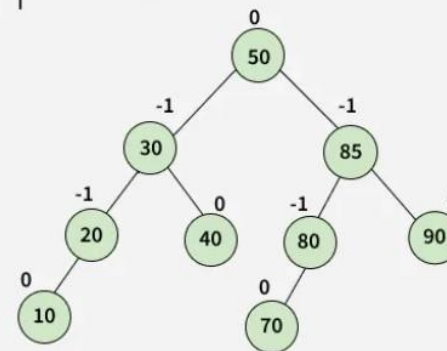
Izvršavanje LL rotacije



- za čvor 50 je $f=-2$ ($1-3=-2$)
- potrebna je RR rotacija na čvoru 50

12
Step

Izvršavanje RR rotacije



KONAČNO AVL STABLO

Faktor balansiraności za svaki čvor stabla je u opsegu od -1 do 1

Vremenska i prostorna složenost operacija u AVL stablu

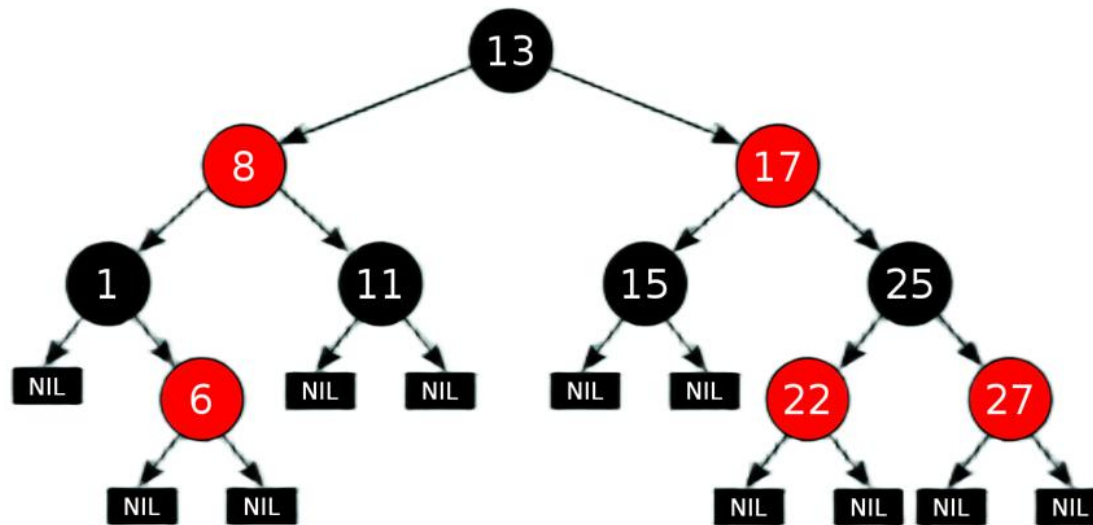
	Vremenska složenost	Prostorna složenost
Kreiranje AVL stabla	$O(1)$	$O(1)$
Unos novog čvora u AVL stablo	$O(\log n)$	$O(\log n)$
Prolazak kroz AVL stablo	$O(n)$	$O(n)$
Pretraga čvora u AVL stablu	$O(\log n)$	$O(\log n)$
Brisanje čvora u AVL stablu	$O(\log n)$	$O(\log n)$
Brisanje AVL stabla	$O(1)$	$O(1)$

Primena AVL stabala

- AVL stabla su najstarija samobalansirajuća uređena binarna stabla
- Nastala su iz potrebe efikasnije pretrage i dobijanja podataka u bazama podataka, indeksiranjem podataka korišćenjem balansiranih binarnih stabala
- AVL stabla su prevashodno korisna za organizaciju i čuvanje velike količine podataka kada se operacije pretrage podataka sprovode često, dok su umetanja i brisanja ređa
- AVL stabla se koriste u primenama koje zahtevaju efikasnu pretragu: implementacija raznih tipova internih kolekcija, kao što su uređeni skupovi i uređene mape, omogućavaju efikasno indeksiranje podataka u pretraživačima, koriste se u fajl-sistemima za čuvanje strukture direktorijuma i za implementaciju tabela simbola u kompajlerima i interpreterima (gde se uz identifikator čuvaju i pridružene informacije: tip podataka, opseg i sl), imaju primenu u sistemima koji rade u realnom vremenu, kao što su sistemi za kontrolu letova ili u video igrima, kada je potrebno brzo sprovesti operacije nad podacima

3. Struktura podataka: RBT stabla

Crveno-crna stabla



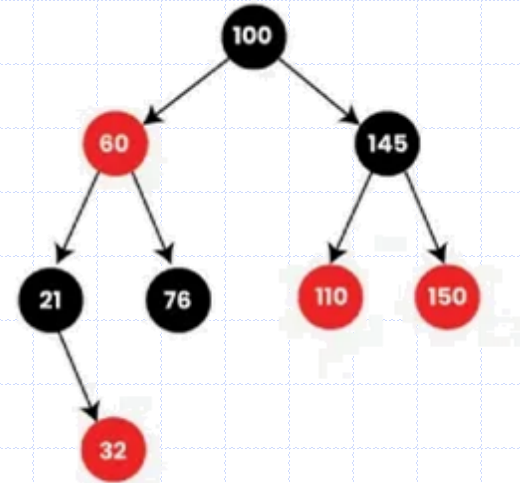
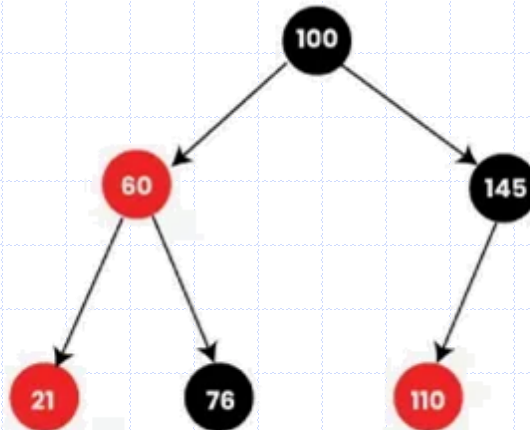
- **Crveno-crno stablo RBT** (Red-Black Tree) je uređeno binarno stablo koje dodatno zadovoljava sledeće uslove:
 - Koren je uvek crn, a svaki drugi čvor je ili crven ili crn
 - Svi listovi su crni i ne sadrže vrednosti (označeni su sa NIL ili NULL)
 - Svi crveni čvorovi imaju crnog roditelja i tačno dva crna potomka
 - Crne čvorovi mogu imati dva crvena, dva crna, ili crveni i crni potomak
 - Sve putanje od nekog čvora do listova u njegovom poddrvetu sadrže isti broj crnih čvorova

RBT stabla

- RBT stabla je 1972. izumeo Rudolf Bayer, L. Guibas i R. Sedgwick su uveli crveno-crnu konvenciju bojenja čvorova, dok je Arne Andersson 1993. osmislio varijantu RBT stabla sa pojednostavljenim operacijama umetanja i brisanja
- Boje se koriste za održavanje ravnoteže tokom umetanja i brisanja, obezbeđujući efikasan pristup i upravljanje podacima stabla
- RBT su samobalansirajuća binarna stabla pretraživanja gde visina stabla nikada nije veća od $O(\log n)$
- U RBT stablu osiguravamo da visina stabla ostane uvek $O(\log n)$ nakon svakog umetanja i brisanja, tako da se garantuje gornja granica od $O(\log n)$ za ove operacije

RBT algoritam	Vremenska složenost
Pretraga	$O(\log n)$
Umetanje	$O(\log n)$
Brisanje	$O(\log n)$

RBT stabla



- **Pravilno RBT stablo** (leva slika) osigurava da svaka putanja od korena do čvora lista ima isti broj crnih čvorova, a u ovom slučaju postoji jedan (isključujući čvor korena)
- **Nepravilno RBT stablo** (desna slika) ima dva crvena čvora jedan pored drugog, a dodatni problem je što jedna od putanja do lista nema crnih čvorova, dok druge dve sadrže crni čvor
- Uslovi RBT stabla garantuju da će putanja od korena do njemu najudaljenijeg lista biti najviše duplo duža nego putanja do njemu najbližeg lista

RBT stabla

- Za RBT stablo se definiše sledeće:
 - $h(v)$ – visina podstabla čiji je koren čvor v , tj. broj čvorova od čvora v do najudaljenijeg lista (ne računajući čvor v)
 - $h_b(v)$ – crna visina podstabla čiji je koren čvor v , tj. broj crnih čvorova od čvora v do proizvoljnog lista (ne računajući čvor v ako je on crn)
- Pod unutrašnjim čvorovima stabla podrazumevaju se svi čvorovi sem listova (NIL čvorova) koji sadrže vrednosti ključeva
- Crna dubina čvora je definisana kao broj crnih čvorova od korena do tog čvora, tj. broj crnih predaka
- **Lema:** RBT stablo sa korenom u čvoru v ima bar $2^{h_b(v)} - 1$ unutrašnjih čvorova
- **Teorema:** Visina h RBT stabla koje sadrži n unutrašnjih čvorova zadovoljava uslov $h \leq 2 \log_2(n + 1)$

RBT stabla

- Direktna posledica prethodne teoreme je da je visina RBT stabla koje sadrži n čvorova $O(\log n)$
- Operacija pretrage u RBT stablu će biti logaritamske složenosti i u funkciji broja elemenata u stablu
- Poseban akcenat je na tome (kao i kod AVL stabla) da se nakon umetanja, odnosno brisanja elementa iz RBT stabla mogu „pokvariti” pojedina svojstva stabla i tada je neophodno izmeniti boje čvorova i/ili preusmeriti neke od pokazivača, tako da dobijeno stablo i dalje ostane crveno-crno
- Preusmeravanje pokazivača se vrši operacijama rotacije stabla
- Postoje leva i desna rotacija i one odgovaraju LL i RR rotaciji u AVL stablu

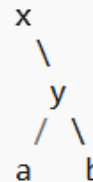
RBT rotacije čvorova

- Rotacije su fundamentalne operacije u održavanju uravnotežene strukture RBT stabla jer pomažu u očuvanju svojstava stabla, osiguravajući da najduži put od korena do bilo kog lista nije veći od dvostruke dužine najkraćeg puta.
- Rotacije postoje kao dve vrste: rotacije ulevo i rotacije udesno

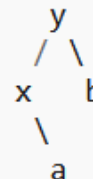
Rotacija ulevo:

- Postaviti **y** da bude desni potomak od **x**
- Prebaciti levo podstablo od **y** u desno podstablo od **x**
- Ažurirati roditelje od **x** i **y**
- Ažurirati roditelja od **x** da bude **y**
- Postaviti levi potomak od **y** da bude **x**
- Ažurirati roditelja od **x** da bude **y**

Before Rotation:



After Left Rotation:



Leva rotacija čvora **x** pomera **x** nadole levo, a njegov desni potomak **y** nagore da zauzme njegovo mesto **x**

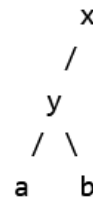
RBT rotacije čvorova

- Rotacije u RBT stablima se obično izvode tokom umetanja i brisanja čvorova kako bi se održala svojstva stabla
- Popravka narušavanja RBT principa stabla nakon umetanja ili brisanja elementa stabla

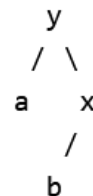
Rotacija udesno:

- Postaviti **y** da bude levi potomak od **x**
- Prebaciti desno podstablo od **y** na levo podstablo od **x**
- Ažurirati roditelje od **x** i **y**
- Ažurirati da roditelj od **x** bude **y**
- Postaviti desni potomak od **y** da bude **x**
- Ažurirati roditelja od **x** da bude **y**

Before Right Rotation:



After Right Rotation:

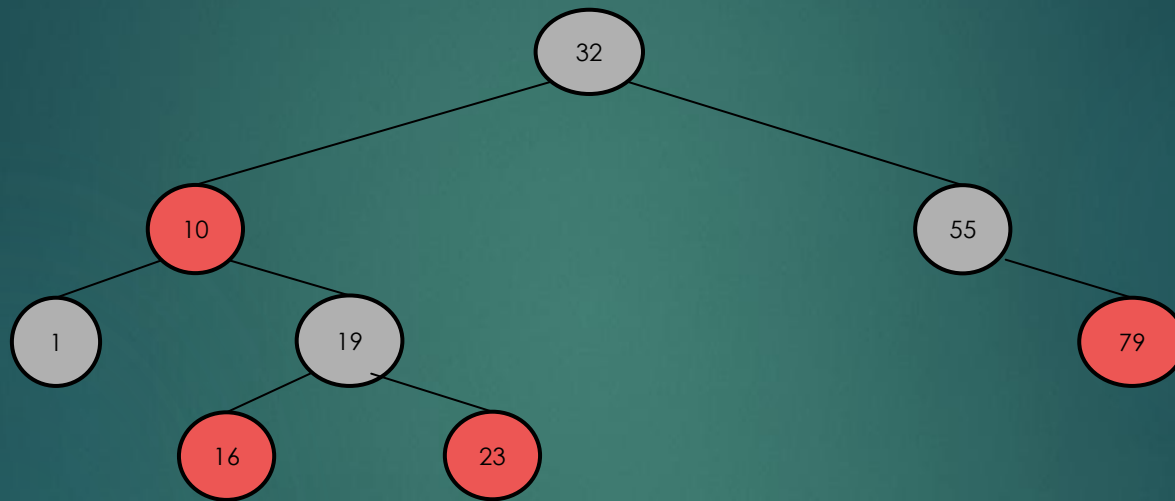


Desna rotacija čvora **x** ga pomera nadole udesno, a njegovog levog potomka **y** na gore da zauzme mesto **x**

Vremenska i prostorna složenost operacija u RBT stablu

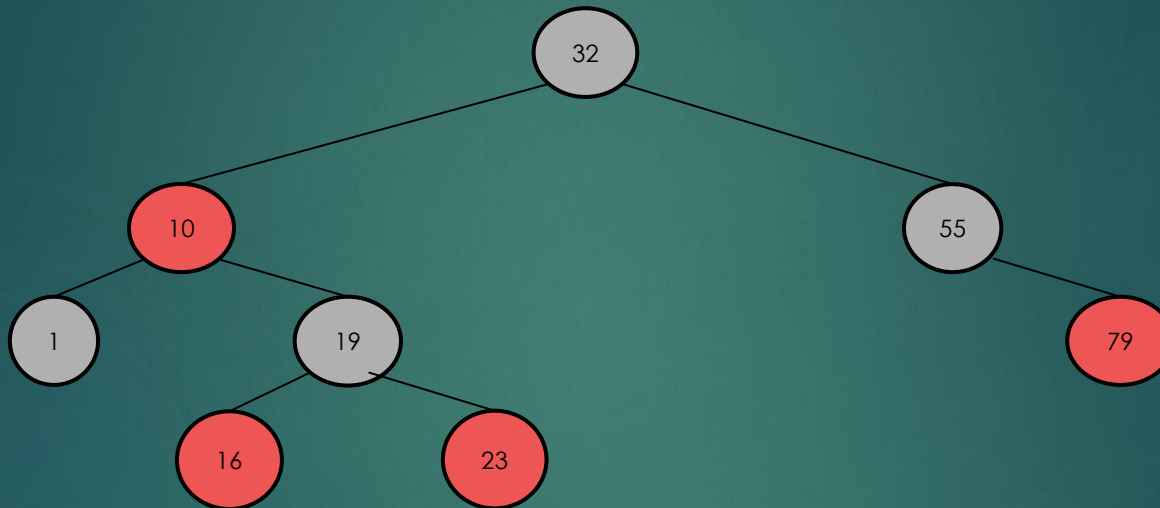
	Vremenska složenost	Prostorna složenost
Unos novog čvora u RBT stablo	$O(\log n)$	$O(\log n)$
Pretraga čvora u RBT stablu	$O(\log n)$	$O(\log n)$
Brisanje čvora u RBT stablu	$O(\log n)$	$O(\log n)$
Brisanje RBT stabla	$O(1)$	$O(1)$

PRIMER:



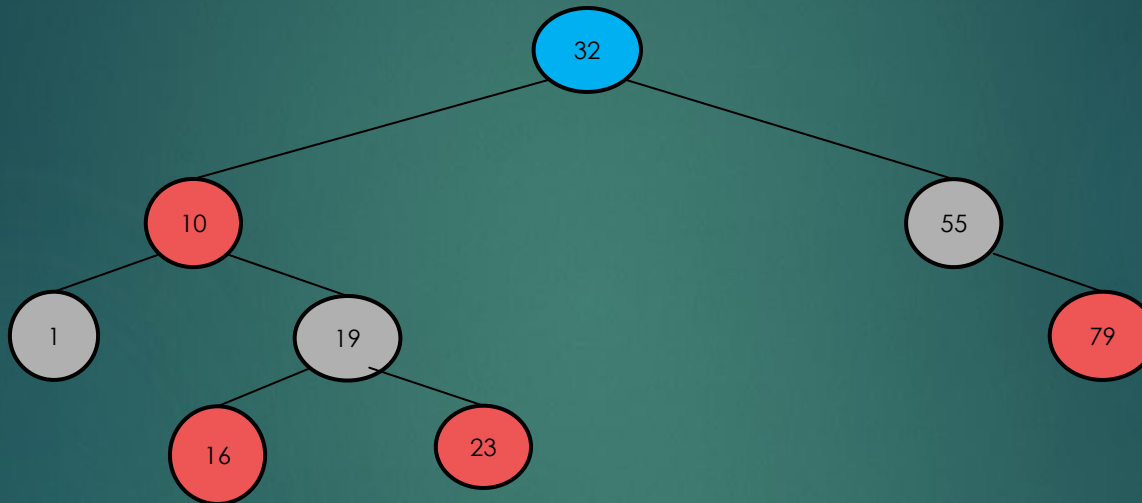
PRIMER:

UNOSI SE BROJ 12



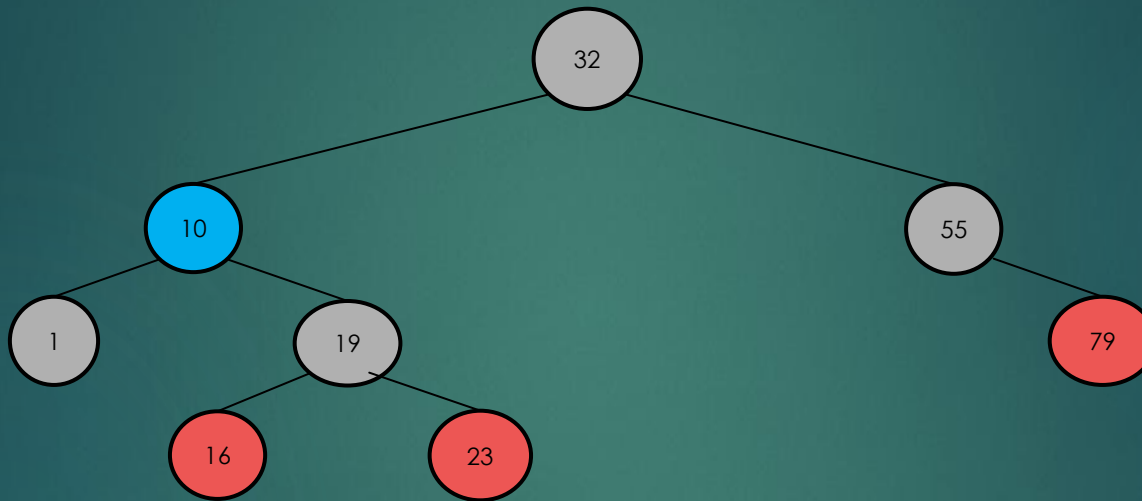
PRIMER:

UNOSI SE BROJ 12



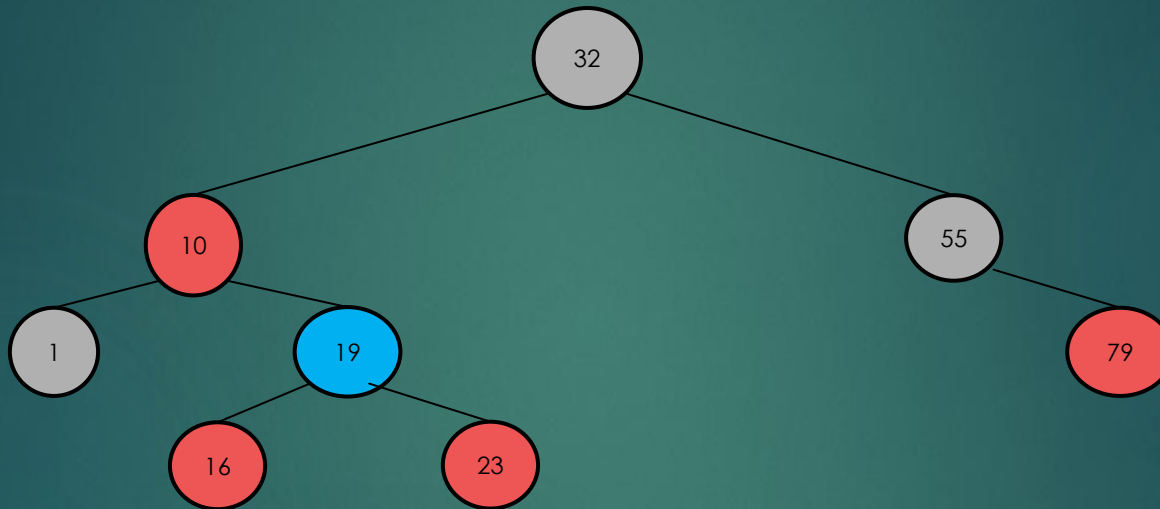
Example:

UNOSI SE BROJ 12



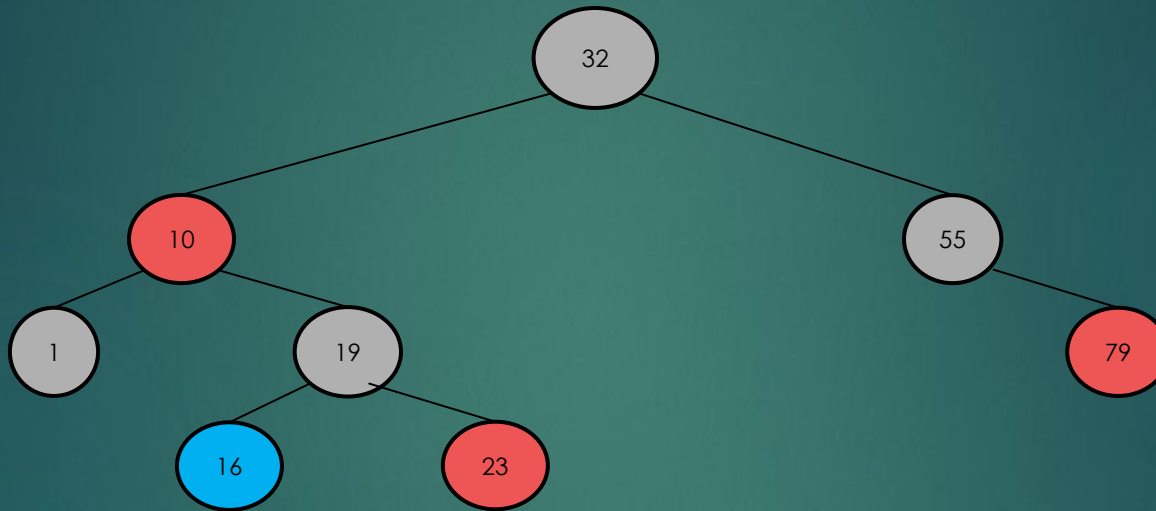
PRIMER:

UNOSI SE BROJ 12

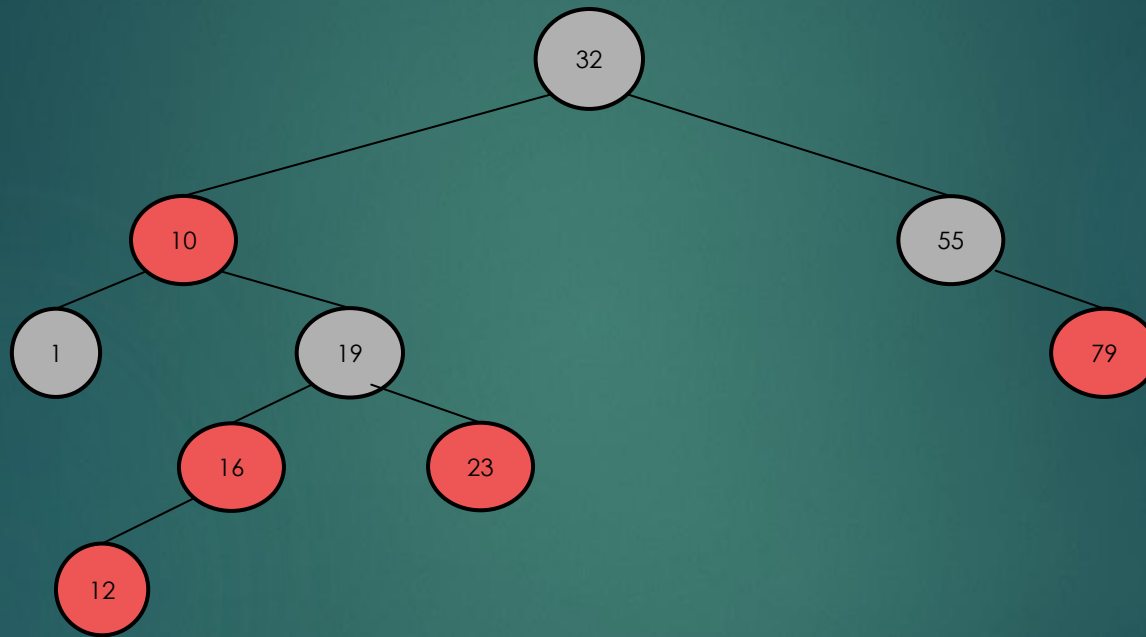


PRIMER:

UNOSI SE BROJ 12

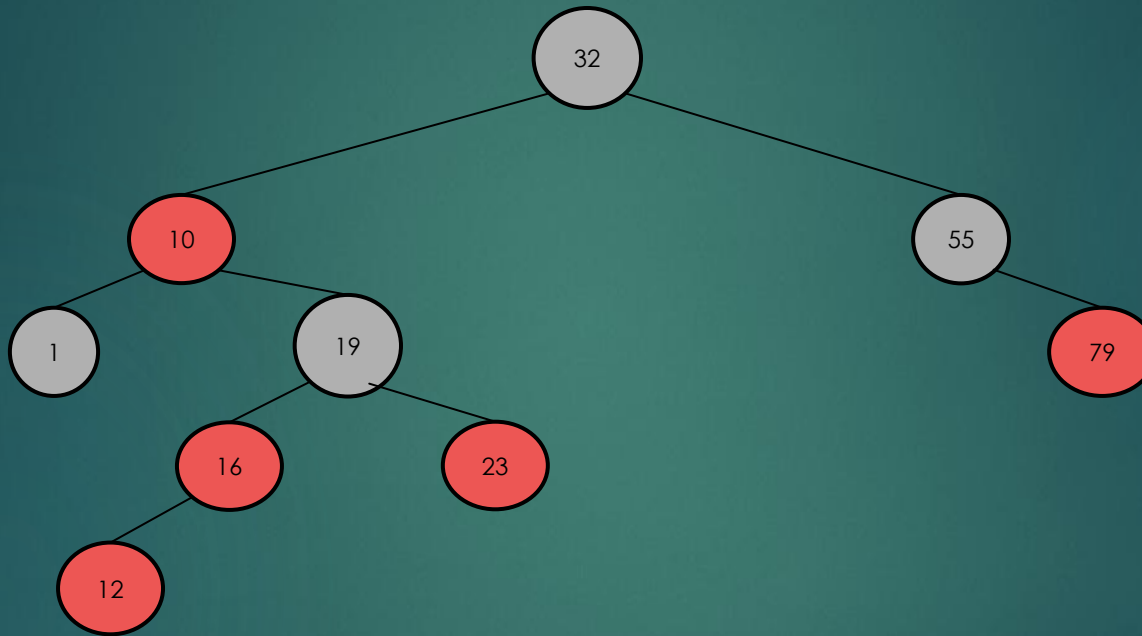


PRIMER:



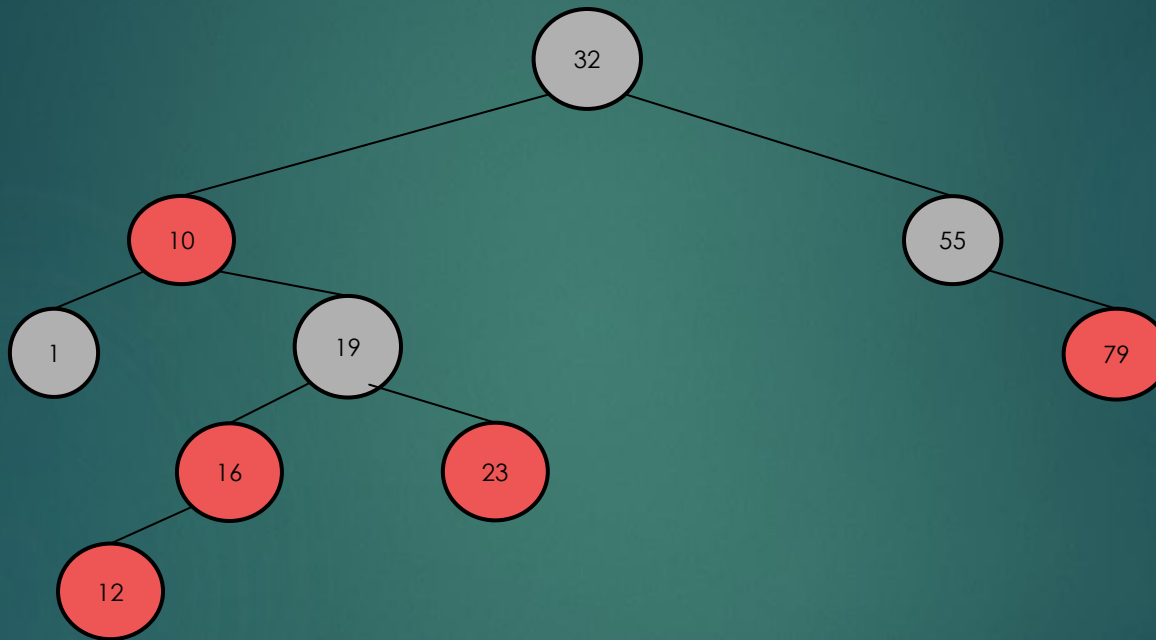
PRIMER:

ZA SVAKI UNOS NOVOG ELEMENTA PROVERAVA SE
DA LI JE NARUŠEN RBT PRINCIP ILI NE



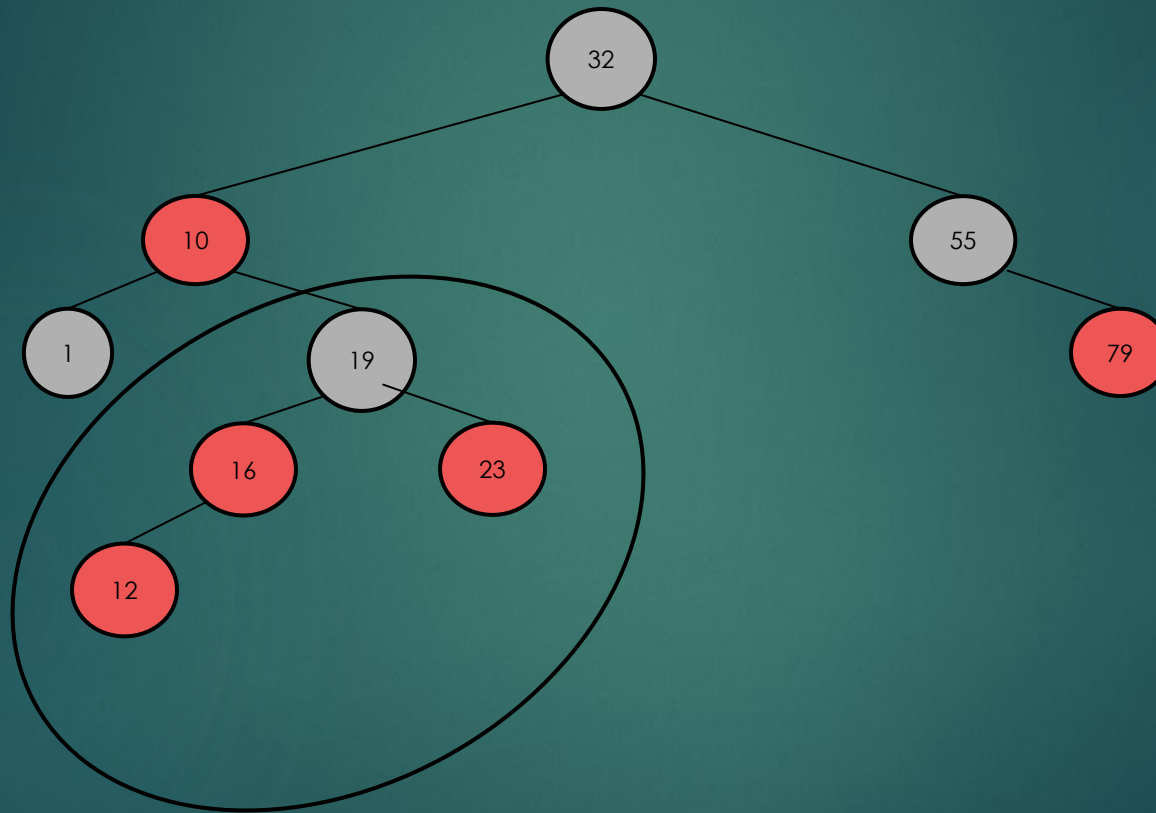
PRIMER:

ZA SVAKI UNOS NOVOG ELEMENTA PROVERAVA SE
DA LI JE NARUŠEN RBT PRINCIP ILI NE



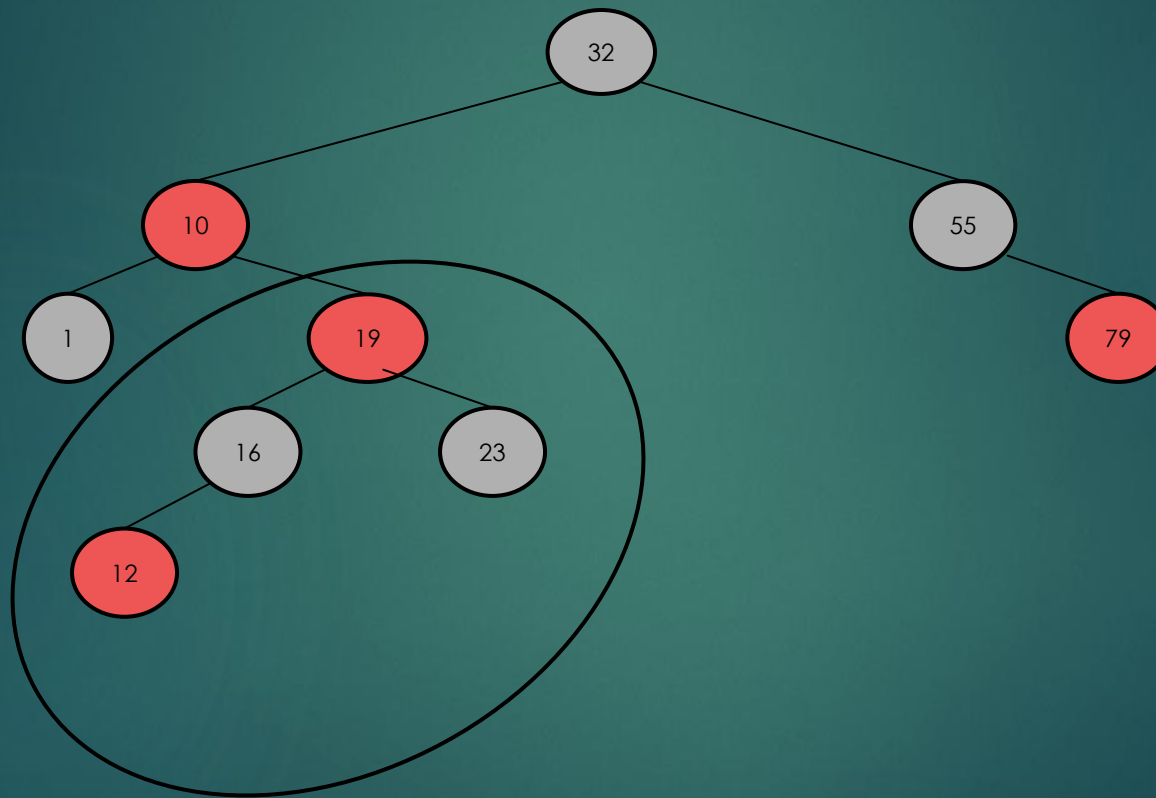
NARUŠEN RBT PRINCIP JER CRVENI ČVOR IMA
JEDNOG CRVENOG POTOMKA

PRIMER:



POTREBNO JE PROMENITI BOJE ČVOROVA

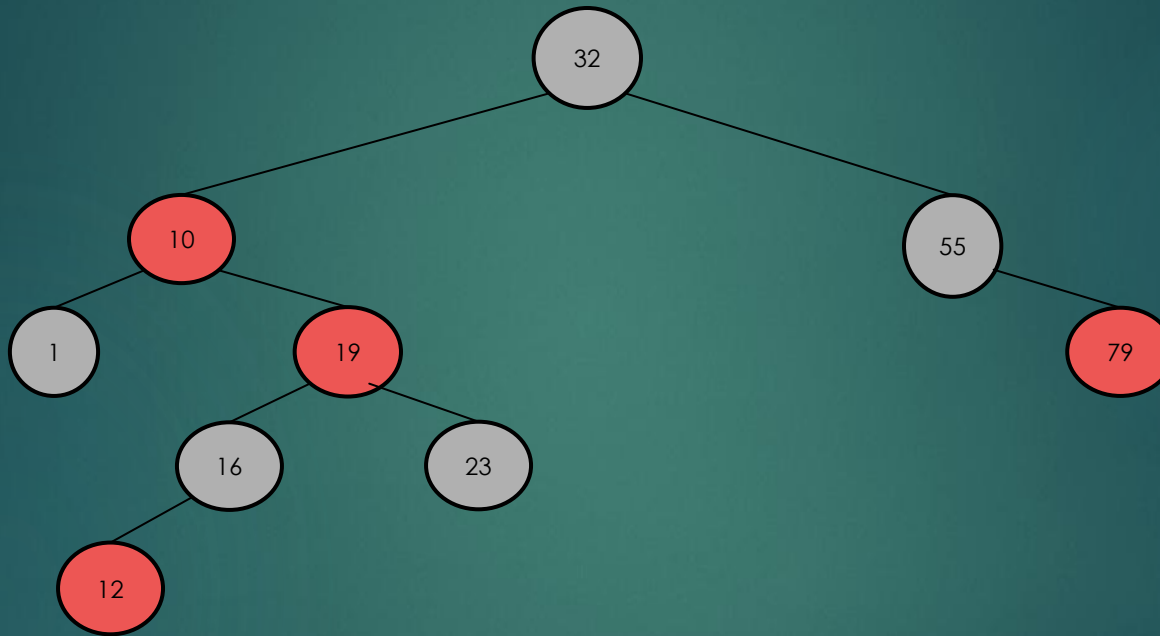
PRIMER:



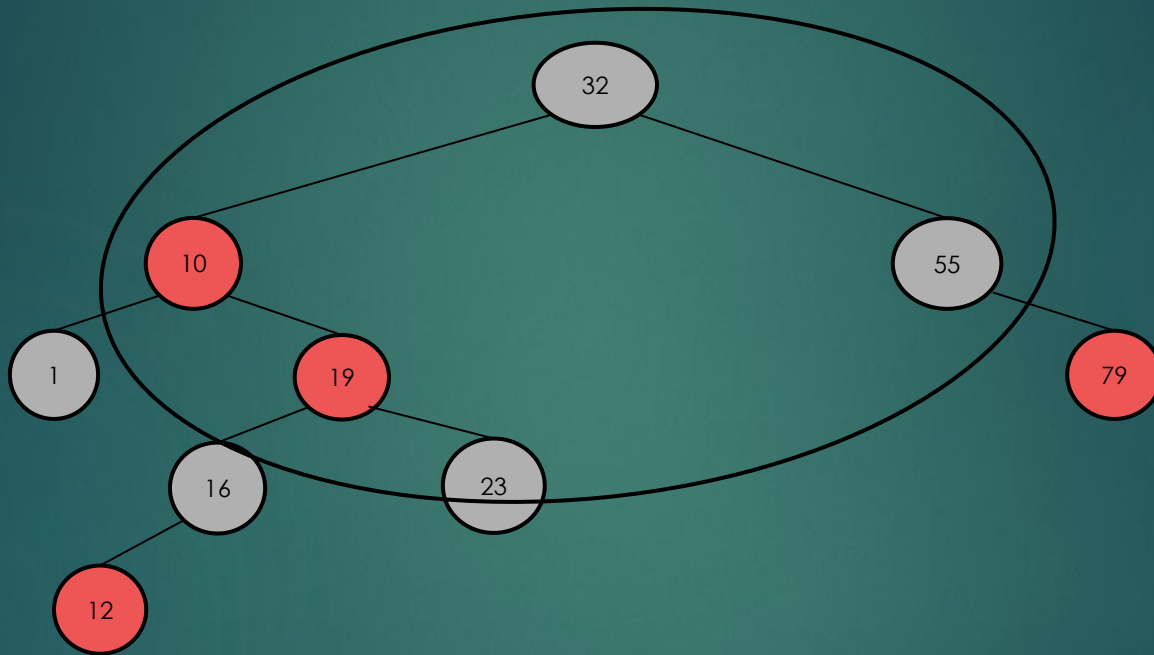
POTREBNO JE PROMENITI BOJE ČVOROVA

PRIMER:

OPET SE PROVERAVA DA LI JE NARUŠEN RBT PRINCIP ILI NE

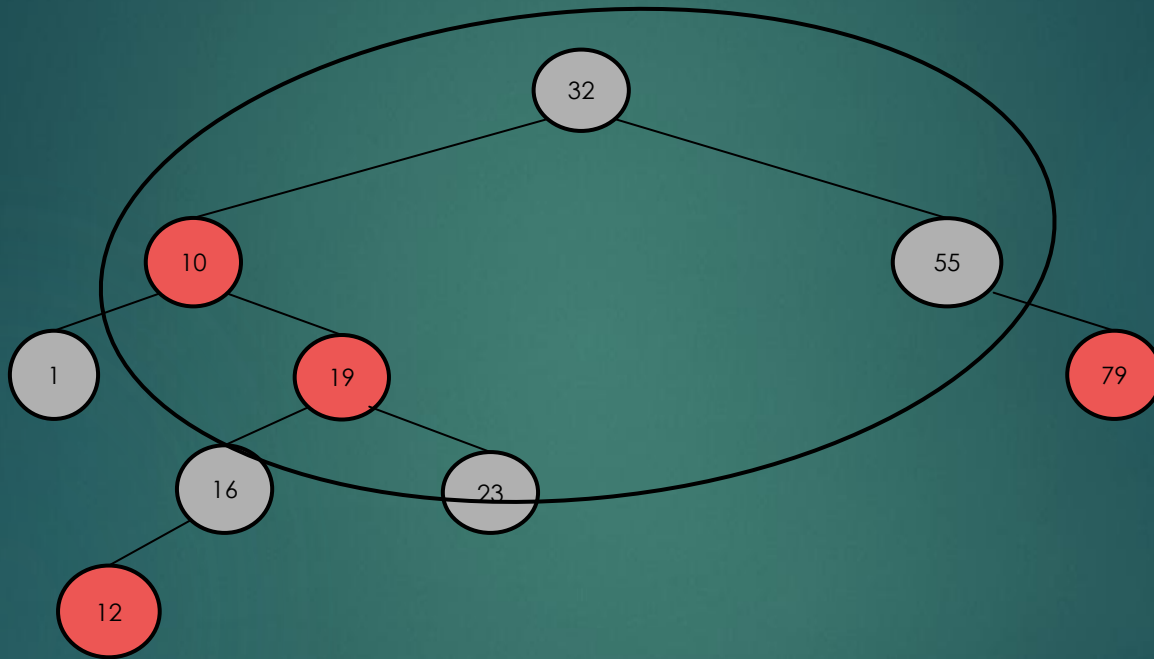


PRIMER:



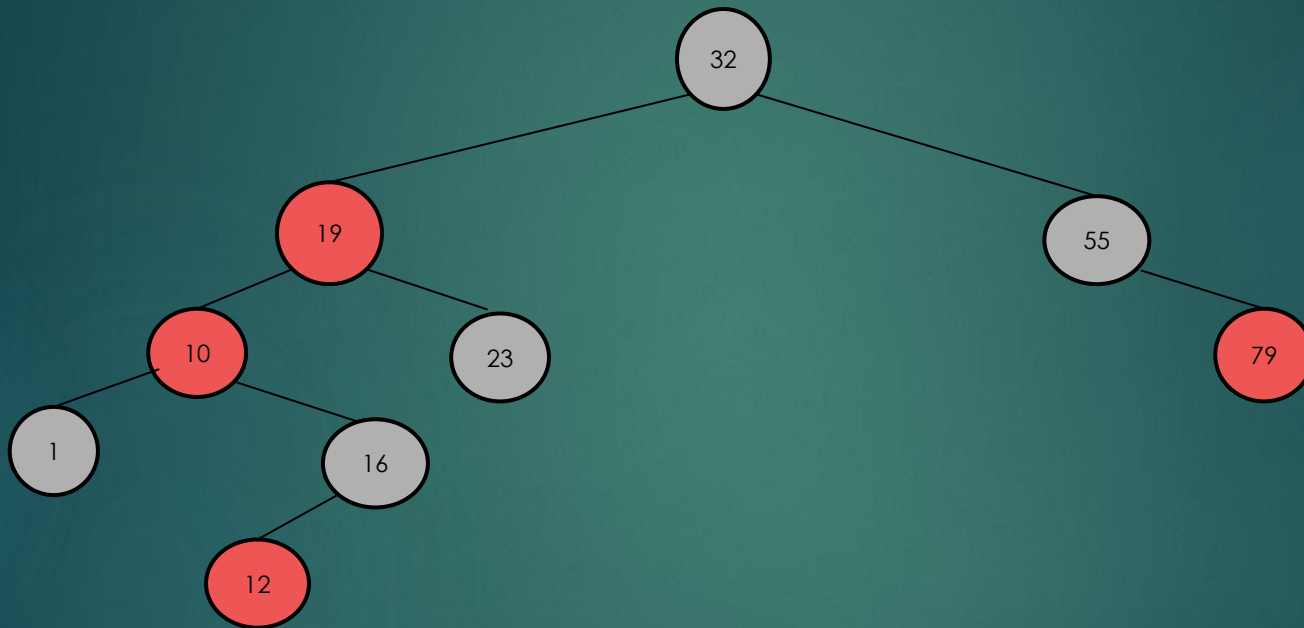
OPET NARUŠEN PRINCIP RBT STABLA

PRIMER:



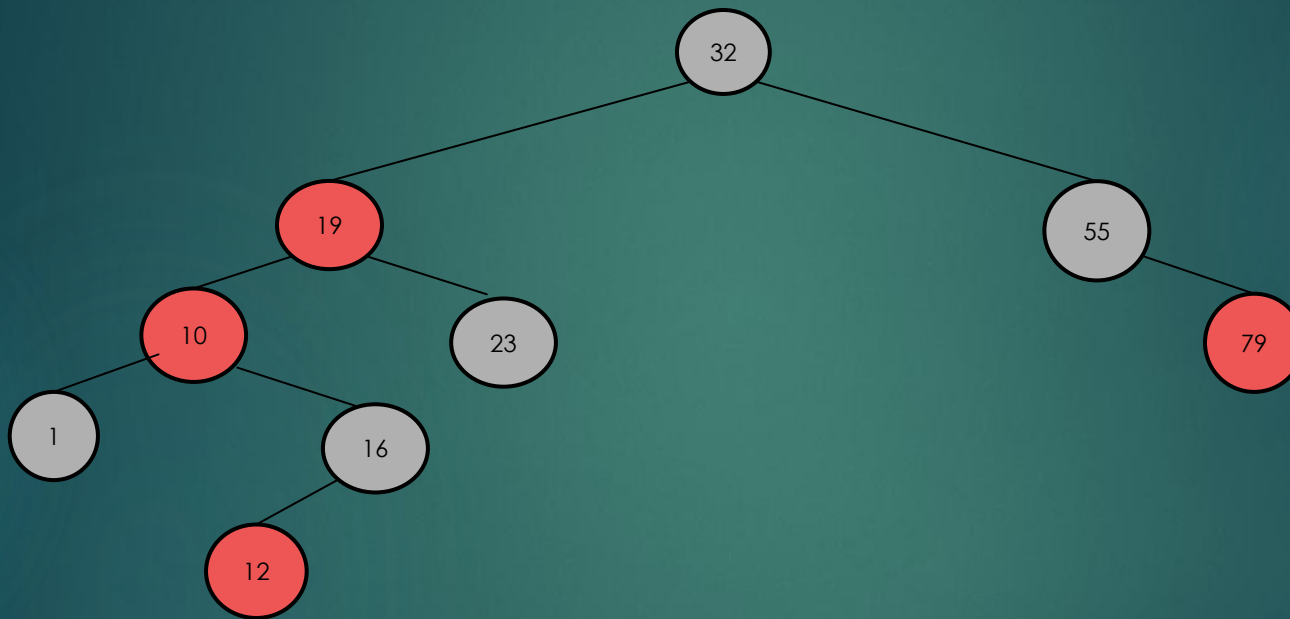
POTREBNO JE IZVRŠITI LEVU ROTACIJU ČVORA 10

PRIMER:



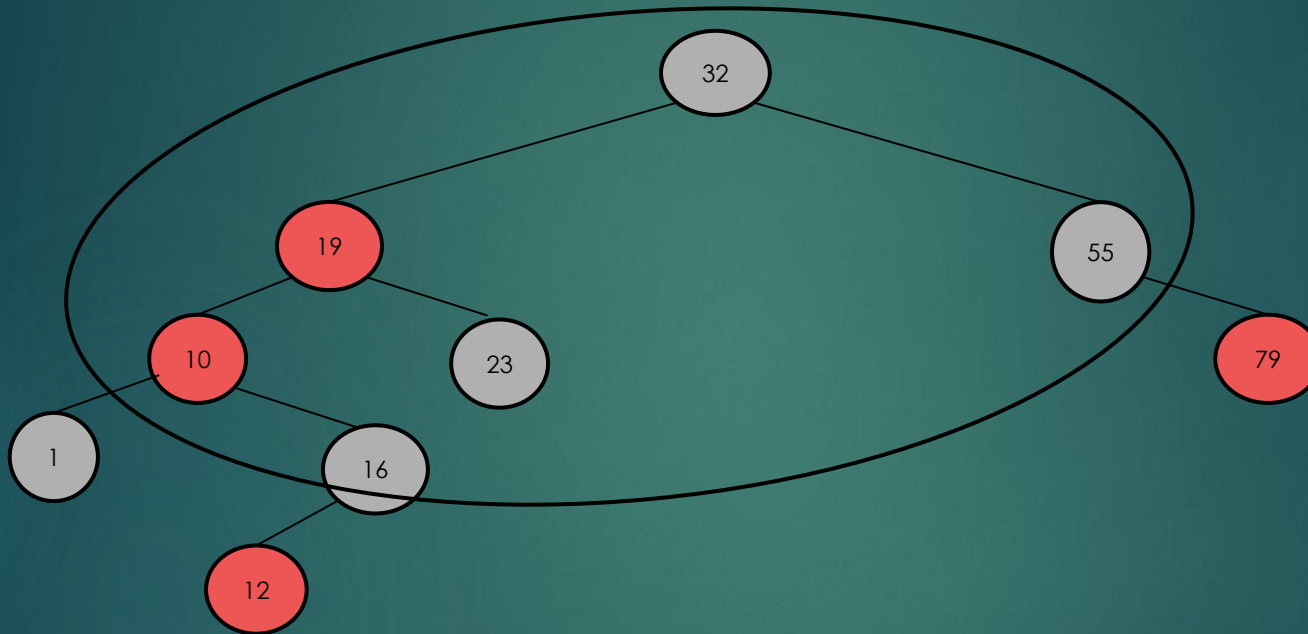
PRIMER:

OPET SE PROVERAVA DA LI JE NARUŠEN RBT PRINCIP ILI NE



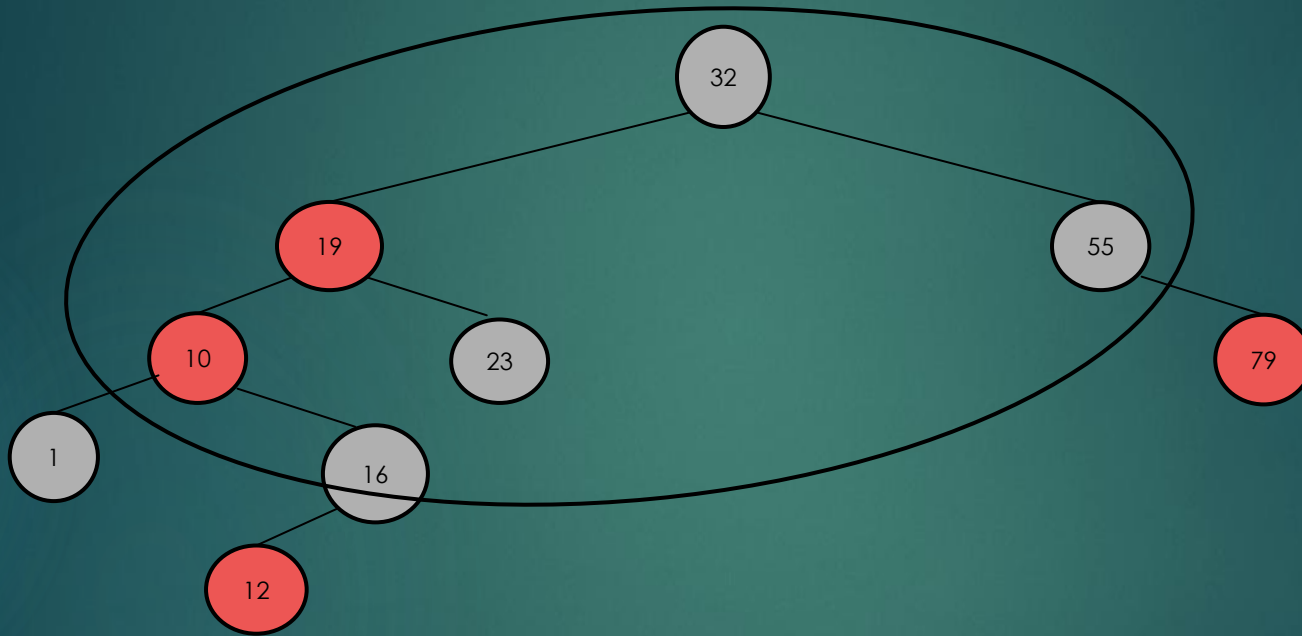
Example:

OPET SE PROVERAVA DA LI JE NARUŠEN RBT PRINCIP ILI NE



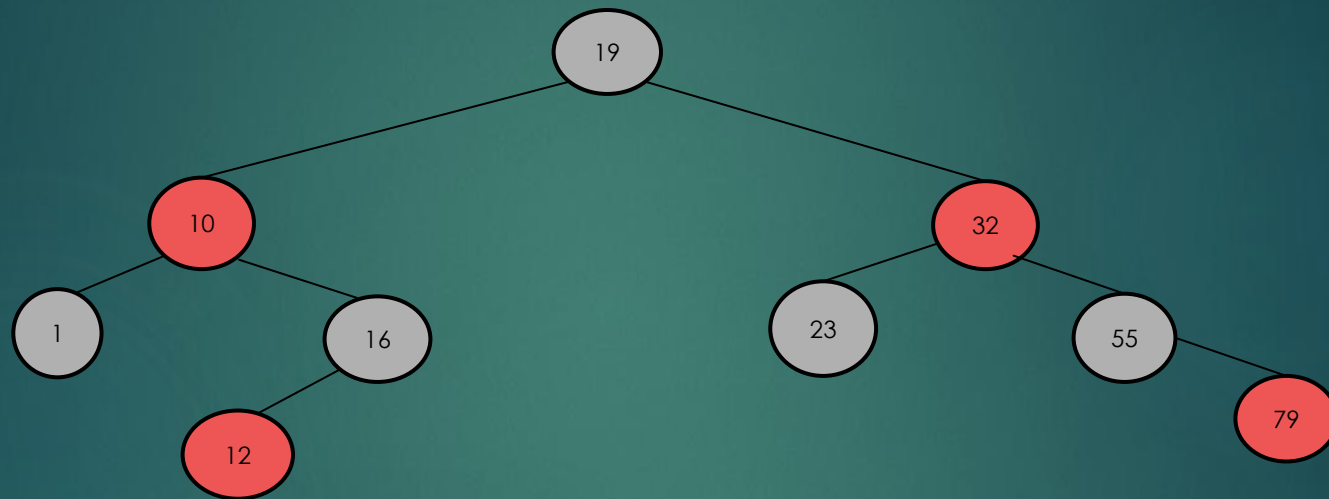
PRIMER:

OPET SE PROVERAVA DA LI JE NARUŠEN RBT PRINCIP ILI NE



POTREBNO JE IZVRŠITI DESNU ROTACIJU ROOT ČVORA

PRIMER:



AVL stabla vs. RBT stabla

- AVL stabla su strožije izbalansirana od RBT, jer je visina AVL stabla uglavnom manja, pa se pretraga u AVL izvodi efikasnije nego u RBT
- AVL stabla su bolja opcija kada je očekivani broj operacija pretraživanja značajno veći od broja operacija umetanja
- Boje (crvena i crna) čvorova u RBT stablima omogućavaju izbegavanje ili smanjenje broja operacija rebalansiranja
- AVL stabla imaju veće troškove rotacije nego RBT, što će u situacijama velikog broja umetanja, dovesti do velikog broja rotacija i tada RBT predstavlja bolji izbor
- Svako AVL stablo se može transformisati u RBT bojenjem čvorova redom u crveno i crno, dok postoje RBT stabla koja nisu AVL

